APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

KTU CST414 — DEEP LEARNING HANDWRITTEN LECTURE NOTES & DERIVATION MANUSCRIPT

B.Tech Computer Science and Engineering · Scheme 2019

Course Code: CST414	Credits: 3
Course Name: Deep Learning	Lecture Hours: 36
Faculty In-Charge: Jane Rose	Department: Computer Science & Engineering
Design: High-Contrast Printable Notebook	Font: Authentic Chalk & Kalam Ink

Prepared for students and faculty for classroom lectures, print study, and university examination preparation.

COURSE LECTURE INDEX & MAPPING

Day	Topic ID	Lecture Title	Module	CO	Date
#01	1.1	Introduction to Neural Networks - Single...	M1	CO1	18-Aug-2025
#02	1.2	Multi-Layer Perceptrons (MLPs) and Repre...	M1	CO1	20-Aug-2025
#03	1.3	Activation Functions, Risk Minimization ...	M1	CO1	22-Aug-2025
#04	1.4	Training MLPs with Backpropagation	M1	CO1	25-Aug-2025
#05	1.5	Illustration of the Backpropagation Algo...	M1	CO1	29-Aug-2025
#06	1.6	Practical Issues in Training: Overfittin...	M1	CO1	01-Sep-2025
#07	1.7	Convergence Difficulties, Local and Spur...	M1	CO1	10-Sep-2025
#08	2.1	Introduction to Deep Learning and Deep F...	M2	CO2	12-Sep-2025
#09	2.2	Training Deep Models: Optimization and R...	M2	CO2	15-Sep-2025
#10	2.3	Gradient Descent (GD) and GD with Moment...	M2	CO2	17-Sep-2025
#11	2.4	Nesterov Accelerated Gradient (NAG) and ...	M2	CO2	19-Sep-2025
#12	2.5	Adaptive Learning Rate Methods: AdaGrad,...	M2	CO2	22-Sep-2025
#13	2.6	Regularization Techniques: L1, L2, Early...	M2	CO2	24-Sep-2025
#14	2.7	Parameter Sharing, Parameter Tying, and ...	M2	CO2	26-Sep-2025
#15	2.8	Ensemble Methods, Model Averaging, Baggi...	M2	CO2	29-Sep-2025
#16	2.9	Dropout and Parameter Initialization	M2	CO2	03-Oct-2025
#17	3.1	Convolutional Neural Networks: The Convo...	M3	CO3	06-Oct-2025
#18	3.2	Motivation for CNNs and Pooling Operatio...	M3	CO3	08-Oct-2025
#19	3.3	Convolution and Pooling as an Infinitely...	M3	CO3	10-Oct-2025
#20	3.4	Variants of the Convolution Function	M3	CO3	13-Oct-2025
#21	3.5	Structured Outputs and Diverse Data Type...	M3	CO3	15-Oct-2025
#22	3.6	Efficient Convolution Algorithms	M3	CO3	17-Oct-2025
#23	4.1	Recurrent Neural Networks: Computational...	M4	CO4	27-Oct-2025
#24	4.2	Encoder-Decoder Sequence-to-Sequence Arc...	M4	CO4	29-Oct-2025
#25	4.3	Deep Recurrent Networks and Recursive Ne...	M4	CO4	31-Oct-2025
#26	4.4	Modern RNNs: Long Short-Term Memory (LST...	M4	CO4	03-Nov-2025
#27	4.5	Gated Recurrent Unit (GRU) and Modern Se...	M4	CO4	05-Nov-2025
#28	5.1	Deep Learning Applications: Computer Vis...	M5	CO5	07-Nov-2025
#29	5.2	Deep Learning Applications: Speech Recog...	M5	CO5	10-Nov-2025
#30	5.3	Deep Learning Applications: Natural Lang...	M5	CO5	12-Nov-2025
#31	5.4	Common Word Embedding: CBOW and Word2Vec	M5	CO5	14-Nov-2025
#32	5.5	Common Word Embedding: Global Vectors fo...	M5	CO5	17-Nov-2025
#33	5.6	Current Research Areas: Autoencoders and...	M5	CO5	19-Nov-2025
#34	5.7	Current Research Areas: Representation L...	M5	CO5	21-Nov-2025
#35	5.8	Current Research Areas: Boltzmann Machin...	M5	CO5	24-Nov-2025
#36	5.9	Current Research Areas: Deep Belief Netw...	M5	CO5	26-Nov-2025

LECTURE #01 · MODULE 1 (Topic 1.1)

Date: 18-Aug-2025
Period: Period 2

Title: Introduction to Neural Networks – Single Layer Perceptrons

CO: CO1 Bloom's: Understand / Apply

Prerequisites: Linear Algebra (vectors, dot products), Basic Calculus, Python basics
Key Terminology: Artificial Neuron, Threshold Logic Unit (TLU), Perceptron Learning Rule, Linear Separability, Hyperplane Decision Boundary, Bias Weight

Introduction

Artificial Neural Networks (ANNs) are computational models inspired by the human biological nervous system. The fundamental computational unit is the artificial neuron, first conceptualized as the McCulloch-Pitts neuron (1943) and later modernized into the trainable Perceptron by Frank Rosenblatt (1958).

Mathematical Model

A Single Layer Perceptron receives an n-dimensional input vector x = [x1, x2, ..., xn]^T and computes the net input z (pre-activation) as the inner product with weight vector w = [w1, w2, ..., wn]^T plus bias b: z = sum_{i=1}^n (w_i * x_i) + b = w^T * x + b The output y is determined by a step activation function: y = f(z) = { +1 if z >= 0 (or 1 in unipolar) { 0 if z < 0 (or -1 in bipolar) Geometrically, w^T * x + b = 0 defines an (n-1)-dimensional hyperplane that partitions the input space into two decision regions.

Perceptron Learning Algorithm

Perceptron Learning Algorithm (PLA): 1. Initialize weights w and bias b to 0 or small random values. 2. For each training sample (x_k, d_k), where d_k is the target label: a. Compute predicted output: y_k = f(w^T * x_k + b) b. Compute error: e_k = d_k - y_k c. Update parameters using learning rate eta (0 < eta <= 1): w <- w + eta * (d_k - y_k) * x_k b <- b + eta * (d_k - y_k) 3. Repeat until all training patterns are classified correctly (zero error). Convergence Theorem (Novikoff, 1962): If the training data is linearly separable with margin gamma and bounded by ||x|| <= R, the PLA is guaranteed to converge in at most (R/gamma)^2 update steps.

Linear Separability And Limitation

Minsky & Papert (1969) proved that single-layer perceptrons can only classify linearly separable patterns. For non-linearly separable problems like the XOR function, no single linear decision boundary exists that can separate the classes, necessitating Multi-Layer Perceptrons.

Exam Important Points

- Derivation and geometric interpretation of the hyperplane decision boundary w^T * x + b = 0.
- Step-by-step weight updates using the Perceptron Learning Rule.
- Proof of linear separability for AND/OR and inability to solve XOR.
- Novikoff's Perceptron Convergence Theorem condition and upper bound on iterations.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Manual Output Calculation of Single Neuron

Statement: Given a 3-dimensional input vector x = [2, 2, 1]^T connected to a single neuron with weight vector w = [0.3, -0.4, 0.5]^T and bias b = -0.1. Calculate the net input (pre-activation) z and the final output y using (a) a unipolar step function and (b) a binary sigmoid activation function sigma(z) = 1 / (1 + e^(-z)).

Given: x = [2, 2, 1]^T, w = [0.3, -0.4, 0.5]^T, b = -0.1

Governing Formula: z = sum(w_i * x_i) + b; y_step = 1 if z >= 0 else 0; y_sigmoid = 1 / (1 + exp(-z))

Step-by-Step Calculation:

Step 1: Compute weighted sum: sum = (0.3 * 2) + (-0.4 * 2) + (0.5 * 1) = 0.6 - 0.8 + 0.5 = 0.3.

Step 2: Add bias: z = sum + b = 0.3 + (-0.1) = 0.2.

Step 3: Unipolar step function: Since z = 0.2 >= 0, y_step = 1.

Step 4: Binary sigmoid function: y_sigmoid = 1 / (1 + e^(-0.2)) = 1 / (1 + 0.81873) = 1 / 1.81873 = 0.5500.

[FINAL EVALUATED ANSWER: Net input z = 0.2, Step output = 1, Sigmoid output = 0.5500]

Pedagogical Note: The positive pre-activation indicates that the input vector lies on the positive side of the separating hyperplane. The sigmoid yields an activated confidence probability of 55%.

Problem 2 (Advanced (KTU Exam Level)): Perceptron Learning Rule Step-by-Step Weight Update

Statement: Design a single-layer perceptron to compute the 2-input NAND function. Initial weights are w1 = 0.0, w2 = 0.0 with bias b = 0.0. The learning rate is eta = 0.5. Perform the first epoch of training using the Perceptron Learning Algorithm for the sequential training samples: (0,0)->1, (0,1)->1, (1,0)->1, (1,1)->0. Show all intermediate calculations.

Given: Initial w = [0.0, 0.0], b = 0.0, eta = 0.5. Activation: step function (y=1 if z>=0 else 0). Samples: x1=(0,0), d1=1; x2=(0,1), d2=1; x3=(1,0), d3=1; x4=(1,1), d4=0.

Governing Formula: z = w1*x1 + w2*x2 + b; y = 1 if z >= 0 else 0; delta_w = eta * (d - y) * x; delta_b = eta * (d - y)

Step-by-Step Calculation:

Sample 1: x=(0,0), d=1. z = 0*0 + 0*0 + 0 = 0. Pred y = 1. Error = 1 - 1 = 0. No update. w=[0.0, 0.0], b=0.0.

Sample 2: x=(0,1), d=1. z = 0*0 + 0*1 + 0 = 0. Pred y = 1. Error = 1 - 1 = 0. No update. w=[0.0, 0.0], b=0.0.

Sample 3: x=(1,0), d=1. z = 0*1 + 0*0 + 0 = 0. Pred y = 1. Error = 1 - 1 = 0. No update. w=[0.0, 0.0], b=0.0.

Sample 4: x=(1,1), d=0. z = 0*1 + 0*1 + 0 = 0. Pred y = 1. Error = 0 - 1 = -1 (MISCLASSIFIED!).

Update on Sample 4: delta_w1 = 0.5 * (-1) * 1 = -0.5; delta_w2 = 0.5 * (-1) * 1 = -0.5; delta_b = 0.5 * (-1) = -0.5.

New weights after Epoch 1: w1 = -0.5, w2 = -0.5, b = -0.5.

[FINAL EVALUATED ANSWER: Weights after Epoch 1: w1 = -0.5, w2 = -0.5, b = -0.5.]

Pedagogical Note: The perceptron adjusted its hyperplane to penalize the false positive at (1,1). Further epochs will refine bias and weights until the decision boundary separates (1,1) from the remaining vertices.

Real-World Industry Case (Smart Manufacturing & Automated Quality Control): Industrial Defect Screening via Linear Separator

Pipeline Workflow: High-speed camera captures manufactured ceramic tiles -> Edge detection extracts contrast ratio (x1) and surface roughness variance (x2) -> Calibrated single-layer threshold neuron performs millisecond binary pass/fail classification.

Industrial Impact & Benefit: Enables ultra-low-latency (sub-millisecond) hardware defect filtering on microcontrollers without requiring deep GPU hardware.

Key Oral Viva Voce Checkpoints

Q1: What is the biological analogue of weights in an artificial neuron?

Ans: Synaptic connection strength (synapse efficacy).

Q2: Define linear separability in n-dimensional Euclidean space.

Ans: Two sets of points are linearly separable if there exists an (n-1)-dimensional hyperplane that cleanly divides the space with all points of one set on one side and the other set on the opposite side.

Q3: Why can a single-layer perceptron not solve the XOR problem?

Ans: Because the XOR truth table produces non-linearly separable patterns in 2D space where true and false instances lie on alternating diagonal vertices.

Q4: What role does the bias term play in a neuron?

Ans: The bias allows the decision hyperplane to shift away from the coordinate origin, enabling flexible positioning of the decision threshold.

Q5: State Novikoff's Perceptron Convergence Theorem.

Ans: If training samples are linearly separable with margin gamma and bounded norm R, the PLA will terminate with a separating solution in at most (R/gamma)^2 weight updates.

LECTURE #02 · MODULE 1 (Topic 1.2)

Date: 20-Aug-2025
Period: Period 4

Title: Multi-Layer Perceptrons (MLPs) and Representation Power of MLPs

CO: CO1 Bloom's: Understand / Analyze

Prerequisites: Single Layer Perceptron, Matrix multiplication, Linear Separability
Key Terminology: Input Layer, Hidden Layer, Output Layer, Universal Approximation Theorem, Feature Space Warping, Feedforward Architecture

Introduction

A Multi-Layer Perceptron (MLP) is a class of feedforward artificial neural network consisting of at least three layers of nodes: an input layer, one or more hidden layers, and an output layer. Except for input nodes, each node is a neuron that uses a non-linear activation function.

Architecture And Forward Pass

For an MLP with L layers: Layer 0: Input vector a^(0) = x For layer l = 1, 2, ..., L: z^(l) = W^(l) * a^(l-1) + b^(l) a^(l) = sigma(z^(l)) where W^(l) is an (n_l x n_{l-1}) weight matrix, b^(l) is an (n_l x 1) bias vector, and sigma is an element-wise non-linear activation function. The final network prediction is y_hat = a^(L).

Representation Power And Xor

How MLPs resolve the XOR problem: By mapping original 2D input space (x1, x2) through a hidden layer with 2 neurons (h1, h2), the input space is transformed into a new feature space where the points become linearly separable. Specifically: h1 = ReLU(x1 + x2 - 0.5) (acts as an OR-like detector) h2 = ReLU(x1 + x2 - 1.5) (acts as an AND-like detector) Output y = h1 - 2 * h2 produces exactly the XOR truth table!

Universal Approximation Theorem

Cybenko (1989) and Hornik (1991) proved the Universal Approximation Theorem: A standard feedforward neural network with a single hidden layer containing a finite number of neurons with non-linear activation functions can approximate any continuous function f: R^n -> R^m on compact subsets of R^n to arbitrary precision epsilon > 0. Crucial Caveat: The theorem guarantees the EXISTENCE of such an approximation, but it does NOT guarantee that the required weights can be discovered by gradient-based learning algorithms, nor does it guarantee that a single hidden layer is computationally or parameter-efficient compared to deeper architectures.

Exam Important Points

- Full architectural diagram of MLP with matrix dimensions for weights and biases.
- Algebraic demonstration of XOR resolution using hidden representations.
- Formal statement and engineering implications of the Universal Approximation Theorem.
- Calculation of total parameters (weights + biases) across multi-layer networks.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Trainable Parameter Count in Multi-Layer Perceptron

Statement: An MLP has an input layer with 784 features, a first hidden layer with 256 neurons, a second hidden layer with 128 neurons, and an output layer with 10 classes. Assuming all layers are fully connected with individual biases, calculate: (a) Number of weight parameters between each layer, (b) Number of bias parameters in each layer, and (c) Total trainable parameters in the entire network.

Given: Layer 0 (Input) n0 = 784, Layer 1 n1 = 256, Layer 2 n2 = 128, Layer 3 (Output) n3 = 10.

Governing Formula: Weights between layer (l-1) and l: W_l = n_l * n_{l-1}; Biases in layer l: B_l = n_l; Total = sum(W_l + B_l)

Step-by-Step Calculation:

Layer 1 (784 -> 256): Weights = 784 * 256 = 200,704. Biases = 256.
Layer 2 (256 -> 128): Weights = 256 * 128 = 32,768. Biases = 128.
Layer 3 (128 -> 10): Weights = 128 * 10 = 1,280. Biases = 10.
Total Weights = 200,704 + 32,768 + 1,280 = 234,752.
Total Biases = 256 + 128 + 10 = 394.
Total Trainable Parameters = 234,752 + 394 = 235,146.

[FINAL EVALUATED ANSWER: Total Trainable Parameters = 235,146]

Pedagogical Note: The majority of parameters (~85.3%) reside in the initial fully connected transition from input to the first hidden layer.

Problem 2 (Advanced (KTU Exam Level)): Feedforward Output Calculation of 2-Hidden-Layer MLP

Statement: Consider a 2-2-1 MLP with input x = [1.0, -1.0]^T. Hidden Layer 1 has weight matrix W^(1) = [[0.5, -0.2], [0.8, 0.1]] and bias b^(1) = [0.1, -0.2]^T with ReLU activation. Output layer has weight vector W^(2) = [0.7, -0.4] and bias b^(2) = 0.3 with Sigmoid activation. Calculate the step-by-step activations and the final network output.

Given: x = [1.0, -1.0]^T, W^(1) = [[0.5, -0.2], [0.8, 0.1]], b^(1) = [0.1, -0.2]^T, W^(2) = [0.7, -0.4], b^(2) = 0.3

Governing Formula: z^(1) = W^(1)*x + b^(1); a^(1) = max(0, z^(1)); z^(2) = W^(2)*a^(1) + b^(2); y = 1 / (1 + exp(-z^(2)))

Step-by-Step Calculation:

Step 1: Compute z1^(1) = (0.5 * 1.0) + (-0.2 * -1.0) + 0.1 = 0.5 + 0.2 + 0.1 = 0.8.
Step 2: Compute z2^(1) = (0.8 * 1.0) + (0.1 * -1.0) + (-0.2) = 0.8 - 0.1 - 0.2 = 0.5.
Step 3: Apply ReLU: a1^(1) = max(0, 0.8) = 0.8; a2^(1) = max(0, 0.5) = 0.5.
Step 4: Compute z^(2) = (0.7 * 0.8) + (-0.4 * 0.5) + 0.3 = 0.56 - 0.20 + 0.3 = 0.66.
Step 5: Apply Sigmoid: y = 1 / (1 + e^(-0.66)) = 1 / (1 + 0.51685) = 1 / 1.51685 = 0.6593.

[FINAL EVALUATED ANSWER: Hidden activations a^(1) = [0.8, 0.5]^T, Final Output y = 0.6593]

Pedagogical Note: Both hidden neurons are active (positive pre-activations), allowing information to propagate forward and yielding a positive prediction confidence of 65.93%.

Real-World Industry Case (Fintech & Credit Risk Assessment): Retail Loan Default Prediction Engine

Pipeline Workflow: Customer credit bureau features (debt-to-income, credit score, delinquencies, monthly inquiries) fed into a 4-layer MLP -> Hidden layers synthesize non-linear interaction patterns -> Output sigmoid neuron outputs probability of default within 90 days.

Industrial Impact & Benefit: Detects complex non-linear repayment risk signals that linear logistic regression models miss, cutting default rates by up to 18%.

Key Oral Viva Voce Checkpoints

Q1: What is the primary architectural difference between an SLP and an MLP?

Ans: An MLP contains one or more hidden layers between the input and output layers with non-linear activation functions.

Q2: Why is a non-linear activation function strictly required in hidden layers?

Ans: Without non-linear activations, a composition of linear layers collapses mathematically into a single linear transformation: W2*(W1*x + b1) + b2 = (W2*W1)*x + (W2*b1 + b2), removing the benefit of depth.

Q3: What does the Universal Approximation Theorem state?

Ans: A single hidden layer feedforward network with non-linear activations can approximate any continuous function on compact subsets of R^n to arbitrary accuracy, given sufficient hidden units.

Q4: If a shallow network can approximate any function, why do we use deep networks?

Ans: Shallow networks may require an exponentially large number of hidden neurons. Deep networks represent hierarchical compositional features with exponentially fewer parameters.

Q5: How many parameters are in a dense layer connecting 100 inputs to 50 outputs?

Ans: 100 * 50 = 5,000 weights + 50 biases = 5,050 trainable parameters.

LECTURE #03 · MODULE 1 (Topic 1.3)

Date: 22-Aug-2025
Period: Period 1

Title: Activation Functions, Risk Minimization and Loss Functions

CO: CO1 Bloom's: Understand / Apply

Prerequisites: Calculus (derivatives, partial derivatives), Probability basics
Key Terminology: Activation Function, Sigmoid, Hyperbolic Tangent (Tanh), Rectified Linear Unit (ReLU), Softmax, Empirical Risk Minimization, Cross-Entropy Loss, Negative Log-Likelihood

Activation Functions

1. Sigmoid (Logistic): sigma(z) = 1 / (1 + e^(-z)), Range: (0, 1) Derivative: sigma'(z) = sigma(z) * (1 - sigma(z)), Max derivative = 0.25 at z = 0. Drawbacks: Vanishing gradient for |z| >> 0; Not zero-centered (causes zig-zagging gradient updates). 2. Hyperbolic Tangent (Tanh): tanh(z) = (e^z - e^(-z)) / (e^z + e^(-z)), Range: (-1, 1) Derivative: tanh'(z) = 1 - tanh^2(z), Max derivative = 1.0 at z = 0. Advantage: Zero-centered output; faster convergence than sigmoid, but still saturates at extremes. 3. Rectified Linear Unit (ReLU): f(z) = max(0, z), Range: [0, infinity) Derivative: f'(z) = 1 if z > 0, else 0. Advantages: Computationally ultra-efficient, does not saturate in positive regime (mitigates vanishing gradients), induces sparse representations. Drawback: 'Dying ReLU' problem when neurons permanently output 0. 4. Softmax (Multiclass Output): softmax(z_i) = e^(z_i) / sum_{j=1}^K e^(z_j), Range: (0, 1), sum_{i=1}^K softmax(z_i) = 1. Converts raw logits into a valid categorical probability distribution.

Risk Minimization

Statistical Learning Theory defines expected risk R(theta) as the expected loss over the true data distribution p_data(x, y): R(theta) = E_{(x,y)~p_data} [L(f(x; theta), y)] Since p_data is unknown, Empirical Risk Minimization (ERM) replaces true expectation with sample average over N training examples: R_emp(theta) = (1 / N) * sum_{i=1}^N L(f(x_i; theta), y_i) Structural Risk Minimization (SRM) adds a regularization penalty Omega(theta) to control model capacity and avoid overfitting: R_struct(theta) = R_emp(theta) + lambda * Omega(theta)

Loss Functions

1. Mean Squared Error (Regression): L_MSE = (1 / 2) * (y - y_hat)^2 (or 1/N * sum) Derived from Maximum Likelihood Estimation under Gaussian noise assumption. 2. Binary Cross-Entropy (Binary Classification): L_BCE = - [y * log(y_hat) + (1 - y) * log(1 - y_hat)] 3. Categorical Cross-Entropy (Multiclass Classification): L_CCE = - sum_{k=1}^K y_k * log(y_hat_k) Pairing Softmax with Cross-Entropy prevents gradient saturation during backpropagation because derivative dL/dz = y_hat - y.

Exam Important Points

- Formulas and graphical sketches for Sigmoid, Tanh, ReLU, and Softmax along with their derivatives.
- Why ReLU accelerates deep network convergence compared to Sigmoid.
- Mathematical proof that d/dz [sigma(z)] = sigma(z) * (1 - sigma(z)).
- Derivation of the softmax derivative and why cross-entropy is the natural loss partner.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Softmax Probability and Cross-Entropy Loss Calculation

Statement: A deep neural network outputs logits z = [2.0, 1.0, -1.0]^T for a 3-class classification problem. The true class label is Class 1 (one-hot vector y = [1, 0, 0]^T). (a) Calculate the Softmax probability distribution, and (b) Calculate the Categorical Cross-Entropy loss.

Given: z = [2.0, 1.0, -1.0]^T, true label y = [1, 0, 0]^T

Governing Formula: softmax(z_i) = e^(z_i) / sum(e^z); L = - sum(y_i * ln(p_i))

Step-by-Step Calculation:

Step 1: Compute exponentials: e^2.0 = 7.3891, e^1.0 = 2.7183, e^(-1.0) = 0.3679.

Step 2: Compute denominator sum = 7.3891 + 2.7183 + 0.3679 = 10.4753.

Step 3: Compute probabilities: p1 = 7.3891 / 10.4753 = 0.7054 (70.54%) p2 = 2.7183 / 10.4753 = 0.2595 (25.95%) p3 = 0.3679 / 10.4753 = 0.0351 (3.51%).

Step 4: Compute Cross-Entropy Loss: L = - [1 * ln(0.7054) + 0 + 0] = - (-0.3490) = 0.3490.

[FINAL EVALUATED ANSWER: Softmax probabilities = [0.7054, 0.2595, 0.0351]^T; Loss L = 0.3490]

Pedagogical Note: The model assigns the highest probability (70.54%) to the ground truth class, resulting in a moderate cross-entropy loss of 0.3490.

Problem 2 (Advanced (KTU Exam Level)): Derivative Derivation & Vanishing Gradient Verification

Statement: Given a 4-layer network where each hidden neuron uses the Sigmoid activation function. At each layer, the pre-activation magnitude |z| >= 4.0. (a) Compute the maximum possible value of the activation derivative at each layer, and (b) Calculate the upper bound on the gradient propagation factor across the 4 layers: product_{l=1}^4 sigma'(z_l). Contrast this with ReLU.

Given: Number of layers L = 4, |z| >= 4.0, Sigmoid activation.

Governing Formula: sigma'(z) = sigma(z) * (1 - sigma(z)); For |z| >= 4, sigma(4) = 1 / (1 + e^(-4)) = 0.9820, sigma'(4) = 0.9820 * (1 - 0.9820) = 0.0177.

Step-by-Step Calculation:

Step 1: In the best case (|z| = 0), sigma'(0) = 0.25. But for |z| >= 4.0, sigma'(z) <= 0.0177.

Step 2: Calculate product of gradients across 4 layers under Best-Case (z=0): (0.25)^4 = 0.003906.

Step 3: Calculate product of gradients under Saturated Case (|z| >= 4): (0.0177)^4 = 9.80 x 10^(-8).

Step 4: Comparison with ReLU: For ReLU, for any positive input z > 0, f'(z) = 1.0. Thus, product = (1.0)^4 = 1.0.

[FINAL EVALUATED ANSWER: Saturated gradient product = 9.80 x 10^(-8) (vanishes to near zero). ReLU gradient product = 1.0 (retained).]

Pedagogical Note: This illustrates mathematically why Sigmoid networks suffer from catastrophic vanishing gradients in deep architectures, whereas ReLU preserves gradient magnitude.

Real-World Industry Case (Computer Vision & Edge Automotive Systems): Autonomous Vehicle Traffic Sign Recognition

Pipeline Workflow: Input camera frame -> Deep network extracts visual features -> Softmax layer outputs probability across 43 traffic sign categories (Speed limit, Stop, Yield) -> Cross-entropy loss trains system to minimize classification risk.

Industrial Impact & Benefit: Softmax probabilities allow the automotive decision control unit to establish confidence thresholds (e.g. only brake if Stop Sign confidence > 98%).

Key Oral Viva Voce Checkpoints

Q1: What is the maximum value of the derivative of the standard logistic sigmoid?

Ans: 0.25, occurring at z = 0.

Q2: Why is Tanh generally preferred over Sigmoid in hidden layers?

Ans: Tanh is zero-centered (range -1 to 1), which prevents systematic directional bias in weight updates across successive layers.

Q3: What is the 'Dying ReLU' problem?

Ans: When a neuron's pre-activation is consistently negative, its gradient becomes permanently 0, preventing the weights from ever updating again during training.

Q4: How does Leaky ReLU solve the dying ReLU issue?

*Ans: By introducing a small positive slope (e.g., 0.01*z) for negative inputs so gradients never become completely zero.*

Q5: Why is Softmax combined with Cross-Entropy instead of Mean Squared Error?

Ans: Cross-Entropy cancels out the exponential term in Softmax, yielding a clean linear error term (y_hat - y) that prevents gradient saturation during backpropagation.

LECTURE #04 · MODULE 1 (Topic 1.4)

Date: 25-Aug-2025
Period: Period 2

Title: Training MLPs with Backpropagation

CO: CO1 Bloom's: Understand / Apply

Prerequisites: Chain Rule of Multivariable Calculus, Vector and Matrix Calculus, Forward Propagation
Key Terminology: Backpropagation, Chain Rule, Error Sensitivity (Delta), Gradient Vector, Jacobian Matrix, Forward Pass, Backward Pass

Introduction

Backpropagation (Rumelhart, Hinton, Williams, 1986) is the foundational training algorithm for multi-layer neural networks. It efficiently computes the partial derivative of the scalar loss function L with respect to every weight and bias parameter in the network by applying the chain rule in reverse order.

Mathematical Derivation

Consider layer l with activation a^(l) = sigma(z^(l)), where z^(l) = W^(l) * a^(l-1) + b^(l). Define the Error Sensitivity (Delta) vector for layer l: delta^(l) = del L / del z^(l) 1. Fundamental Equation 1 (Output Layer Delta): delta^(L) = (del L / del a^(L)) elementwise_prod sigma'(z^(L)) For Cross-Entropy Loss with Softmax: delta^(L) = y_hat - y 2. Fundamental Equation 2 (Hidden Layer Delta Backpropagation): delta^(l) = [(W^(l+1))^T * delta^(l+1)] elementwise_prod sigma'(z^(l)) This equation propagates error signals backward from layer l+1 to layer l. 3. Fundamental Equation 3 (Rate of change of Loss with respect to Bias): del L / del b^(l) = delta^(l) 4. Fundamental Equation 4 (Rate of change of Loss with respect to Weights): del L / del W^(l) = delta^(l) * (a^(l-1))^T Parameter Update Rule: W^(l) <- W^(l) - eta * (del L / del W^(l)) b^(l) <- b^(l) - eta * (del L / del b^(l))

Computational Efficiency

Naive numerical gradient estimation (perturbing each weight by epsilon) requires O(P) forward passes, leading to O(P^2) time complexity for P parameters. Backpropagation evaluates all P gradients in a single backward pass with computational cost roughly equivalent to 2 forward passes: O(P) complexity!

Exam Important Points

- State and derive the four fundamental equations of backpropagation.
- Define the error vector delta^(l) and explain its physical significance.
- Explain why backpropagation runs backward rather than forward.
- Matrix dimensional consistency across all weight gradient equations.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Output Layer Error Delta Calculation

Statement: An output neuron with Sigmoid activation has pre-activation z = 1.5, giving output a = 0.8176. The ground truth target is y = 1.0. If the loss function is Squared Error L = 0.5 * (a - y)^2, calculate: (a) Error derivative dL/da, (b) Activation derivative da/dz, and (c) Error sensitivity delta.

Given: z = 1.5, a = 0.8176, y = 1.0, L = 0.5*(a - y)^2

Governing Formula: dL/da = a - y; da/dz = a*(1 - a); delta = (dL/da) * (da/dz)

Step-by-Step Calculation:

Step 1: dL/da = 0.8176 - 1.0 = -0.1824.

Step 2: da/dz = 0.8176 * (1.0 - 0.8176) = 0.8176 * 0.1824 = 0.1491.

Step 3: delta = (-0.1824) * 0.1491 = -0.02720.

[FINAL EVALUATED ANSWER: dL/da = -0.1824, da/dz = 0.1491, delta = -0.0272]

Pedagogical Note: The negative delta indicates that increasing the pre-activation z will decrease the loss, which guides the gradient update in the proper direction.

Problem 2 (Advanced (KTU Exam Level)): Hidden Layer Weight Gradient via Error Backpropagation

Statement: In a 2-layer network, hidden neuron h1 has activation a1 = 0.6. It is connected to output neuron y with weight w_out = 0.8. The output neuron has computed error delta_out = -0.15. The hidden neuron h1 receives an input x1 = 0.5 through weight w_in, and its activation function is Sigmoid. Calculate: (a) The backpropagated error delta_h1, and (b) The gradient dL/dw_in.

Given: a1 = 0.6, w_out = 0.8, delta_out = -0.15, x1 = 0.5, hidden activation = Sigmoid.

Governing Formula: delta_h = (w_out * delta_out) * a_h * (1 - a_h); dL/dw_in = delta_h * x1

Step-by-Step Calculation:

Step 1: Compute activation derivative of hidden neuron: sigma'(z1) = a1 * (1 - a1) = 0.6 * (1 - 0.6) = 0.24.

Step 2: Backpropagate error through w_out: backprop_sum = w_out * delta_out = 0.8 * (-0.15) = -0.12.

Step 3: Compute hidden delta: delta_h1 = backprop_sum * sigma'(z1) = -0.12 * 0.24 = -0.0288.

Step 4: Compute gradient with respect to input weight: dL/dw_in = delta_h1 * x1 = -0.0288 * 0.5 = -0.0144.

[FINAL EVALUATED ANSWER: Hidden delta_h1 = -0.0288, Gradient dL/dw_in = -0.0144]

Pedagogical Note: The negative gradient means increasing w_in will decrease the overall network error. The weight update rule will increment w_in by eta * 0.0144.

Real-World Industry Case (Digital Signal Processing & Real-time Acoustics): Acoustic Noise Cancellation in Aviation Communication

Pipeline Workflow: Cockpit ambient microphone picks up jet engine roar -> Filtered through deep MLP -> Output produces anti-phase acoustic cancellation wave -> Error microphone measures residual decibel leak -> Backpropagation updates weights at 100 Hz.

Industrial Impact & Benefit: Enables crystal-clear air traffic controller communication in high-decibel turbofan cockpit environments.

Key Oral Viva Voce Checkpoints

Q1: Who popularized the Backpropagation algorithm for neural networks in 1986?

Ans: David Rumelhart, Geoffrey Hinton, and Ronald Williams.

Q2: What is the primary mathematical principle underlying backpropagation?

Ans: The chain rule of multivariable calculus applied in reverse topological order.

Q3: What does the term delta^(l) represent?

Ans: The partial derivative of the scalar loss with respect to the pre-activation vector z of layer l: del L / del z^(l).

Q4: Why is caching intermediate activations during the forward pass necessary?

Ans: Because weight gradients depend directly on the input activation from the preceding layer: dL/dW^(l) = delta^(l) * (a^(l-1))^T.

Q5: What is the time complexity of backpropagation compared to finite difference numerical gradient calculation?

Ans: Backpropagation is O(P) with respect to parameter count P, whereas finite differences require O(P^2) or O(P) full forward passes.

LECTURE #05 · MODULE 1 (Topic 1.5)

Date: 29-Aug-2025
Period: Period 1

Title: Illustration of the Backpropagation Algorithm

CO: CO1 Bloom's: Apply / Evaluate

Prerequisites: Backpropagation Equations, Matrix Calculus, Day 4 concepts
Key Terminology: End-to-End Walkthrough, Forward Evaluation, Backward Sensitivity, Gradient Update, Loss Reduction Verification, Learning Step

Network Specification

*Standard 2-2-1 Fully Connected MLP Architecture: Input vector: x = [x1, x2]^T = [0.05, 0.10]^T Target output: y = 0.01 (or target vector for 2 outputs) Initial weights from Input to Hidden layer: V = [[v11, v12], [v21, v22]] = [[0.15, 0.20], [0.25, 0.30]] Hidden biases: b1 = [0.35, 0.35]^T Weights from Hidden to Output layer: W = [w1, w2] = [0.40, 0.45] Output bias: b2 = 0.60 Activation function: Logistic Sigmoid everywhere: sigma(z) = 1 / (1 + e^(-z)) Loss Function: Squared Error L = 0.5 * (y_hat - y)^2 Learning rate: eta = 0.5*

Step By Step Execution Phases

*Phase 1: Forward Pass 1. Hidden Neuron 1: z_h1 = (0.15 * 0.05) + (0.20 * 0.10) + 0.35 = 0.0075 + 0.020 + 0.35 = 0.3775 a_h1 = sigma(0.3775) = 1 / (1 + e^(-0.3775)) = 0.59327 2. Hidden Neuron 2: z_h2 = (0.25 * 0.05) + (0.30 * 0.10) + 0.35 = 0.0125 + 0.030 + 0.35 = 0.3925 a_h2 = sigma(0.3925) = 1 / (1 + e^(-0.3925)) = 0.59688 3. Output Neuron: z_o = (0.40 * 0.59327) + (0.45 * 0.59688) + 0.60 = 0.23731 + 0.26860 + 0.60 = 1.10591 a_o = sigma(1.10591) = 1 / (1 + e^(-1.10591)) = 0.75137 4. Loss Calculation: L = 0.5 * (0.75137 - 0.01)^2 = 0.5 * (0.74137)^2 = 0.27481 Phase 2: Backward Pass 1. Output Error Delta: dL/da_o = a_o - y = 0.75137 - 0.01 = 0.74137 da_o/dz_o = a_o * (1 - a_o) = 0.75137 * (1 - 0.75137) = 0.18681 delta_o = (dL/da_o) * (da_o/dz_o) = 0.74137 * 0.18681 = 0.13849 2. Output Weight Gradients: dL/dw1 = delta_o * a_h1 = 0.13849 * 0.59327 = 0.08216 dL/dw2 = delta_o * a_h2 = 0.13849 * 0.59688 = 0.08266 dL/db2 = delta_o = 0.13849 3. Hidden Neuron Deltas: delta_h1 = (delta_o * w1) * a_h1 * (1 - a_h1) = (0.13849 * 0.40) * 0.59327 * (1 - 0.59327) = 0.05540 * 0.24130 = 0.01337 delta_h2 = (delta_o * w2) * a_h2 * (1 - a_h2) = (0.13849 * 0.45) * 0.59688 * (1 - 0.59688) = 0.06232 * 0.24061 = 0.01499 4. Input-to-Hidden Weight Gradients: dL/dv11 = delta_h1 * x1 = 0.01337 * 0.05 = 0.0006685 dL/dv12 = delta_h1 * x2 = 0.01337 * 0.10 = 0.001337 Phase 3: Parameter Updates (eta = 0.5) w1_new = 0.40 - (0.5 * 0.08216) = 0.35892 w2_new = 0.45 - (0.5 * 0.08266) = 0.40867 v11_new = 0.15 - (0.5 * 0.0006685) = 0.14967*

Exam Important Points

- *Full 14-mark KTU examination standard problem solving template.*
- *Clear notation distinguishing layer indices and neuron indices.*
- *Retention of at least 4 decimal places throughout all calculations.*
- *Demonstration of post-update forward pass verifying loss reduction.*

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Single Step Weight Update with Learning Rate

Statement: An output layer weight w has current value 0.45. The computed gradient of the loss function with respect to w is dL/dw = 0.08266. If the learning rate is eta = 0.5, calculate: (a) The parameter update step delta_w, and (b) The new updated weight w_new.

Given: w_current = 0.45, dL/dw = 0.08266, eta = 0.5

Governing Formula: delta_w = - eta * (dL/dw); w_new = w_current + delta_w

Step-by-Step Calculation:

Step 1: Compute parameter update step: delta_w = - 0.5 * 0.08266 = -0.04133.

Step 2: Compute updated weight: w_new = 0.45 + (-0.04133) = 0.40867.

[FINAL EVALUATED ANSWER: delta_w = -0.04133, w_new = 0.40867]

Pedagogical Note: The positive gradient signifies that increasing w increases loss; hence, the gradient descent step reduces w by 0.04133.

Problem 2 (Advanced (KTU Exam Level)): KTU Model Paper Q11: Parameter Update in MLP using ReLU

Statement: Update the parameters in the given MLP using gradient descent with learning rate eta = 0.5 and activation function as ReLU. Input is x = [1.0, 1.0]^T, target y = 1.0. Initial weights: Hidden layer V = [[0.1, 0.2], [0.1, 0.1]] with hidden bias b1 = [0.0, 0.0]^T. Output layer W = [0.1, 0.1] with output bias b2 = 0.0. Loss function is L = 0.5 * (y_hat - y)^2. Perform one full step of gradient descent.

Given: x = [1.0, 1.0]^T, y = 1.0, eta = 0.5, V = [[0.1, 0.2], [0.1, 0.1]], b1=[0,0]^T, W = [0.1, 0.1], b2 = 0.0. ReLU activation everywhere.

Governing Formula: z1 = V*x + b1; a1 = max(0, z1); zo = W*a1 + b2; y_hat = max(0, zo); dL/da = y_hat - y; delta_o = dL/da * (1 if zo > 0 else 0)

Step-by-Step Calculation:

Step 1: Hidden layer forward pass: z1_1 = 0.1*1.0 + 0.2*1.0 = 0.3 -> a1_1 = max(0, 0.3) = 0.3 z1_2 = 0.1*1.0 + 0.1*1.0 = 0.2 -> a1_2 = max(0, 0.2) = 0.2.

Step 2: Output layer forward pass: zo = 0.1*0.3 + 0.1*0.2 + 0.0 = 0.03 + 0.02 = 0.05. y_hat = max(0, 0.05) = 0.05.

Step 3: Loss calculation: L = 0.5 * (0.05 - 1.0)^2 = 0.5 * (-0.95)^2 = 0.45125.

Step 4: Output layer gradients: dL/dy_hat = 0.05 - 1.0 = -0.95. dy_hat/dzo = 1.0 (since zo = 0.05 > 0). delta_o = -0.95 * 1.0 = -0.95. dL/dW1 = delta_o * a1_1 = -0.95 * 0.3 = -0.285. dL/dW2 = delta_o * a1_2 = -0.95 * 0.2 = -0.190.

Step 5: Hidden layer gradients: Since z1_1, z1_2 > 0, ReLU derivatives are 1.0. delta_h1 = (delta_o * W1) * 1.0 = (-0.95 * 0.1) * 1.0 = -0.095. delta_h2 = (delta_o * W2) * 1.0 = (-0.95 * 0.1) * 1.0 = -0.095. dL/dV11 = delta_h1 * x1 = -0.095 * 1.0 = -0.095. dL/dV12 = delta_h1 * x2 = -0.095 * 1.0 = -0.095. dL/dV21 = delta_h2 * x1 = -0.095 * 1.0 = -0.095. dL/dV22 = delta_h2 * x2 = -0.095 * 1.0 = -0.095.

Step 6: Parameter Updates (eta = 0.5): W1_new = 0.1 - 0.5 * (-0.285) = 0.1 + 0.1425 = 0.2425. W2_new = 0.1 - 0.5 * (-0.190) = 0.1 + 0.0950 = 0.1950. V11_new = 0.1 - 0.5 * (-0.095) = 0.1 + 0.0475 = 0.1475. V12_new = 0.2 - 0.5 * (-0.095) = 0.2 + 0.0475 = 0.2475. V21_new = 0.1 - 0.5 * (-0.095) = 0.1 + 0.0475 = 0.1475. V22_new = 0.1 - 0.5 * (-0.095) = 0.1 + 0.0475 = 0.1475.

[FINAL EVALUATED ANSWER: Updated W = [0.2425, 0.1950], Updated V = [[0.1475, 0.2475], [0.1475, 0.1475]]]

Pedagogical Note: Because the model severely underpredicted the target (y_hat=0.05 vs y=1.0), all weight gradients are negative, leading to significant positive weight increments to boost future predictions.

Real-World Industry Case (Electric Vehicles & Battery Management Systems (BMS)): Real-Time Battery State-of-Charge (SoC) Calibration in EVs

Pipeline Workflow: BMS monitors cell voltage, discharge current, and ambient temperature -> 2-hidden-layer MLP predicts battery State-of-Charge -> At charging stations, precise open-circuit voltage calibration triggers single-step backpropagation on-board microchip.

Industrial Impact & Benefit: Maintains accurate driving range estimation over 10-year battery degradation curves without requiring cloud data transfer.

Key Oral Viva Voce Checkpoints

Q1: What is the primary benefit of working through backpropagation manually?

Ans: It solidifies understanding of chain rule mechanics, tensor dimensions, and why error signals scale across layers.

Q2: How does learning rate magnitude impact the weight update step?

Ans: A high learning rate creates large updates that can overshoot the minimum and diverge; a low rate causes slow, computationally prohibitive convergence.

Q3: What guarantees that a backpropagation gradient update reduces the loss?

Ans: For an infinitesimally small learning rate eta, the update moves in the negative gradient direction (direction of steepest descent), guaranteeing non-increasing loss on smooth surfaces.

Q4: What happens to ReLU gradients when pre-activation z is exactly 0?

Ans: The mathematical derivative is technically undefined, but in practical deep learning frameworks it is defined via sub-gradient as either 0.0 or 1.0 (typically 0.0).

Q5: Why are bias gradients simply equal to the layer delta?

Ans: Because the derivative of pre-activation z with respect to bias b is identically 1 (dz/db = 1).

LECTURE #06 · MODULE 1 (Topic 1.6)

Date: 01-Sep-2025
Period: Period 2

Title: Practical Issues in Training: Overfitting, Vanishing and Exploding Gradients

CO: CO1 Bloom's: Understand / Analyze

Prerequisites: Backpropagation Algorithm, Linear Algebra (eigenvalues), Optimization basics
Key Terminology: Overfitting, Generalization Gap, Vanishing Gradients, Exploding Gradients, Product of Jacobians, Gradient Clipping, Weight Initialization

Overfitting

Overfitting occurs when a neural network memorizes training data noise and idiosyncratic artifacts rather than learning generalizable underlying patterns. Key Indicators: - Training Loss continues decreasing monotonically toward zero. - Validation / Test Loss reaches an inflection point and begins increasing steadily. - Large generalization gap: Loss_val - Loss_train >> 0. Fundamental Causes: 1. Excessive model capacity (too many parameters relative to dataset size). 2. Insufficient training samples or unrepresentative data distribution. 3. Training for excessive epochs without early stopping.

Vanishing And Exploding Gradients

Consider a deep network with L layers. By the chain rule, the gradient of the loss with respect to activation at early layer 1 is: del L / del a^(1) = (del L / del a^(L)) * product_{k=1}^{L-1} (del a^(k+1) / del a^(k)) Each Jacobian matrix is: del a^(k+1) / del a^(k) = W^(k+1) * diag(sigma'(z^(k+1))) Case 1: Vanishing Gradients If the largest singular values of weight matrices ||W^(k)|| < 1 and saturating activations have sigma'(z) <= 0.25 (e.g. Sigmoid): || del L / del a^(1) || <= || del L / del a^(L) || * product_{k=1}^{L-1} (0.25 * ||W^(k)||) -> 0 as L increases. Gradients at early layers shrink exponentially, stalling learning in shallow representations. Case 2: Exploding Gradients If weight matrices have spectral radius > 1 (e.g. weights initialized too large): || del L / del a^(1) || grows exponentially as (lambda_max)^L -> infinity. Results in numerical overflow (NaN / Inf loss), unstable oscillations, and ruined weights.

Mitigation Strategies

1. Activation Choice: Replace Sigmoid/Tanh with non-saturating activations (ReLU, Leaky ReLU, ELU). 2. Proper Initialization: Xavier/Glorot (1990) for Tanh; He/Kaiming (2015) for ReLU. 3. Gradient Clipping: Rescale gradient vector g if ||g|| exceeds threshold c: g_clipped = g * (c / ||g||) if ||g|| > c else g. 4. Normalization: Batch Normalization (Ioffe & Szegedy, 2015) and Layer Normalization. 5. Architectural: Residual skip connections (He et al., 2015) creating identity gradient highways.

Exam Important Points

- Mathematical chain rule derivation of vanishing gradient across L layers.
- Comparison of training vs validation loss curves indicating overfitting vs underfitting.
- Formula and geometric interpretation of Gradient Clipping by norm.
- How spectral radius of weight matrices dictates gradient explosion or decay.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Gradient Clipping by Norm

Statement: During backpropagation of a deep network, the computed gradient vector across all parameters is g = [3.0, -4.0, 12.0]^T. The maximum permissible gradient clipping threshold is c = 5.0. (a) Calculate the L2 norm of the gradient vector ||g||_2, (b) Determine whether clipping is required, and (c) Calculate the clipped gradient vector g_clipped.

Given: g = [3.0, -4.0, 12.0]^T, clipping threshold c = 5.0

Governing Formula: ||g||_2 = sqrt(sum(g_i^2)); if ||g|| > c then g_clipped = g * (c / ||g||_2) else g

Step-by-Step Calculation:

Step 1: Compute L2 norm: ||g||_2 = sqrt(3^2 + (-4)^2 + 12^2) = sqrt(9 + 16 + 144) = sqrt(169) = 13.0.

Step 2: Check clipping condition: Since ||g||_2 = 13.0 > c = 5.0, gradient clipping MUST be applied.

Step 3: Compute scaling factor: factor = c / ||g||_2 = 5.0 / 13.0 = 0.384615.

Step 4: Compute clipped components: g1_clipped = 3.0 * (5/13) = 15/13 = 1.1538 g2_clipped = -4.0 * (5/13) = -20/13 = -1.5385 g3_clipped = 12.0 * (5/13) = 60/13 = 4.6154.

Step 5: Verify new norm: sqrt(1.1538^2 + (-1.5385)^2 + 4.6154^2) = sqrt(1.3312 + 2.3670 + 21.3019) = sqrt(25.0) = 5.0.

[FINAL EVALUATED ANSWER: ||g|| = 13.0, Clipping required. Clipped gradient = [1.1538, -1.5385, 4.6154]^T (Norm = 5.0)]

Pedagogical Note: Clipping rescales the gradient vector to have a maximum length of 5.0 while strictly preserving its original descent direction.

Problem 2 (Advanced (KTU Exam Level)): Mathematical Estimation of Vanishing Gradient Decay Rate

Statement: A deep feedforward network has 8 layers. Each layer uses the standard sigmoid activation function where average pre-activation yields sigma'(z) = 0.20. The weight matrices at each layer have average spectral norm ||W||_2 = 1.5. (a) Derive the contraction factor per layer, (b) Calculate the total gradient attenuation factor from layer 8 back to layer 1, and (c) If the output error gradient is ||dL/da8|| = 2.0, determine the gradient magnitude arriving at layer 1.

Given: L = 8 layers, sigma'(z) = 0.20, ||W||_2 = 1.5, initial ||dL/da8|| = 2.0.

Governing Formula: Per-layer contraction factor gamma = ||W||_2 * sigma'(z); Total attenuation = gamma^(L-1); Magnitude at layer 1 = ||dL/da8|| * gamma^(L-1)

Step-by-Step Calculation:

Step 1: Compute per-layer contraction factor: gamma = 1.5 * 0.20 = 0.30.

Step 2: Number of inter-layer transitions across 8 layers = L - 1 = 7.

Step 3: Total attenuation factor = (0.30)^7 = 0.0002187 (or 2.187 x 10^(-4)).

Step 4: Compute gradient magnitude at layer 1: ||dL/da1|| = 2.0 * (0.30)^7 = 2.0 * 0.0002187 = 0.0004374 (or 4.374 x 10^(-4)).

[FINAL EVALUATED ANSWER: Per-layer factor = 0.30, Total attenuation = 2.187 x 10^(-4), Gradient at layer 1 = 0.0004374]

Pedagogical Note: The gradient signal is attenuated by over 99.97% before reaching the first hidden layer, meaning early layer weights learn thousands of times slower than output weights.

Real-World Industry Case (Healthcare Informatics & Predictive Medicine): Clinical Mortality Prediction in Intensive Care Units (ICU)

Pipeline Workflow: High-frequency ICU patient telemetry (ECG, SpO2, blood pressure, arterial blood gases) fed into an 8-layer deep network -> Overfitting prevented using early stopping based on validation cohort -> Gradient clipping prevents erratic updates during sudden patient physiological anomalies.

Industrial Impact & Benefit: Maintains reliable 24-hour mortality risk scores without overfitting to sparse hospital training records.

Key Oral Viva Voce Checkpoints

Q1: What is the key indicator of overfitting when observing learning curves?

Ans: Training loss continues to decrease while validation loss begins to increase steadily.

Q2: Mathematically, what causes the vanishing gradient problem?

Ans: The repeated multiplication of Jacobian matrices with spectral norms < 1 and activation derivatives bounded below 1 (e.g. 0.25 for sigmoid) across multiple layers.

Q3: What is the primary danger of exploding gradients?

Ans: They cause parameter updates so massive that weights overflow to NaN or infinity, completely destabilizing the network.

Q4: How does gradient clipping by value differ from gradient clipping by norm?

Ans: Clipping by value truncates each individual gradient element independently between [-c, c], which changes the gradient vector direction. Clipping by norm rescales the entire vector by c / ||g||, preserving its direction.

Q5: Why does the ReLU activation function mitigate vanishing gradients?

Ans: For any positive input z > 0, the derivative of ReLU is exactly 1.0, preventing exponential attenuation during backpropagation.

LECTURE #07 · MODULE 1 (Topic 1.7)

Date: 10-Sep-2025
Period: Period 4

Title: Convergence Difficulties, Local and Spurious Optima, Computational Challenges

CO: CO1 Bloom's: Understand / Analyze

Prerequisites: Optimization Theory, Multivariable Calculus (Hessian matrix, Eigenvalues), Day 6 concepts
Key Terminology: Non-Convex Optimization, Saddle Point, Spurious Local Minima, Hessian Matrix, Ill-Conditioned Curvature, Plateau, Condition Number

Loss Surfaces In Deep Learning

Unlike linear models whose convex loss functions guarantee a unique global minimum, deep neural networks have highly non-convex loss landscapes in millions of dimensions. The landscape is characterized by: 1. Symmetries and Permutation Invariance: Permuting neurons within a hidden layer yields identical input-output mappings, resulting in a factorial number of equivalent local minima. 2. Spurious Local Minima: Suboptimal local basins whose loss is significantly worse than the global minimum. Research (Dauphin et al., 2014; Choromanska et al., 2015) indicates that in high-dimensional deep models, true local minima are rare and generally close to global minimum loss. 3. Saddle Points: Critical points where gradient grad L = 0, but the Hessian has both positive and negative eigenvalues. In high dimensions, saddle points are exponentially more prevalent than local minima!

Ill Conditioned Hessian

*The Hessian matrix H contains second-order partial derivatives: H_ij = del^2 L / (del w_i del w_j). The condition number kappa(H) = lambda_max / lambda_min measures curvature disparity: - If kappa(H) >> 1, the loss surface forms an elongated canyon (ravine) where curvature is steep along one direction and flat along another. - First-order Gradient Descent oscillates back and forth across the steep walls while making agonizingly slow progress along the base toward the minimum. Second-order Taylor expansion of loss around current point w_0: L(w) approx L(w_0) + (w - w_0)^T * grad L + (1/2) * (w - w_0)^T * H * (w - w_0) A gradient descent step w = w_0 - eta * grad L yields expected loss change: delta L approx - eta * ||grad L||^2 + (1/2) * eta^2 * (grad L)^T * H * (grad L) If (grad L)^T * H * (grad L) is very large, gradient descent can actually INCREASE loss unless eta is miniscule.*

Computational Challenges

1. Memory Bandwidth Limits: Forward activations must be retained in GPU VRAM for the backward pass, limiting maximum batch size and model depth. 2. Computational Scale: Training modern deep architectures requires petascale floating-point calculations. 3. Vanishing Gradient Highways in Ravines: Stalling optimization on flat saddle plateaus.

Exam Important Points

- *Sketch and differentiate: Local minimum, global minimum, saddle point, and plateau.*
- *Define the Hessian condition number and explain why ill-conditioning causes gradient descent oscillation.*
- *Explain why saddle points pose a greater threat than local minima in high-dimensional deep networks.*
- *Derivation of expected loss change using second-order Taylor expansion.*

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Hessian Matrix Condition Number and Curvature Evaluation

Statement: At a critical point in weight space, the Hessian matrix of a neural network loss function is computed as H = [[200.0, 0.0], [0.0, 2.0]]. (a) Identify the eigenvalues of H, (b) Calculate the condition number kappa(H), and (c) Characterize the nature of the curvature along both principal axes.

Given: H = [[200.0, 0.0], [0.0, 2.0]]

Governing Formula: Eigenvalues are diagonal entries for diagonal matrix; kappa = lambda_max / lambda_min

Step-by-Step Calculation:

Step 1: Eigenvalues are lambda_1 = 200.0 and lambda_2 = 2.0.

Step 2: Both eigenvalues are positive (lambda > 0), confirming that the critical point is a local minimum.

Step 3: Calculate condition number: kappa(H) = lambda_max / lambda_min = 200.0 / 2.0 = 100.0.

[FINAL EVALUATED ANSWER: Eigenvalues = 200.0 and 2.0. Condition number kappa = 100.0 (Ill-conditioned).]

Pedagogical Note: Curvature along the first coordinate axis is 100 times steeper than along the second axis. Vanilla gradient descent will severely oscillate along axis 1 while progressing very slowly along axis 2.

Problem 2 (Advanced (KTU Exam Level)): Second-Order Taylor Series Step-Size Bound

Statement: The loss surface around current weights has gradient g = [4.0, 1.0]^T and Hessian H = [[10.0, 0.0], [0.0, 1.0]]. Using the second-order Taylor expansion delta L approx - eta * ||g||^2 + 0.5 * eta^2 * (g^T * H * g), (a) Calculate the optimal learning rate eta* that maximizes loss reduction, and (b) Compute the maximum learning rate eta_max beyond which the step causes the loss to INCREASE instead of decrease.

Given: g = [4.0, 1.0]^T, H = [[10.0, 0.0], [0.0, 1.0]]

Governing Formula: ||g||^2 = g^T * g; g^T * H * g = sum(g_i^2 * H_ii); eta* = ||g||^2 / (g^T * H * g); eta_max = 2 * eta*

Step-by-Step Calculation:

Step 1: Compute ||g||^2 = 4^2 + 1^2 = 16 + 1 = 17.0.

Step 2: Compute curvature term: g^T * H * g = (4^2 * 10.0) + (1^2 * 1.0) = (16 * 10) + 1 = 160 + 1 = 161.0.

Step 3: Quadratic loss change equation: delta L(eta) = - 17.0 * eta + 0.5 * 161.0 * eta^2 = - 17.0 * eta + 80.5 * eta^2.

Step 4: Differentiate with respect to eta and set to 0: d(delta L)/d(eta) = -17.0 + 161.0 * eta = 0 => eta* = 17.0 / 161.0 = 0.10559.

Step 5: Compute threshold for loss increase (delta L = 0): -17.0 * eta + 80.5 * eta^2 = 0 => eta * (-17.0 + 80.5 * eta) = 0 => eta_max = 17.0 / 80.5 = 2 * eta* = 0.21118.

[FINAL EVALUATED ANSWER: Optimal learning rate eta* = 0.1056; Maximum learning rate before divergence eta_max = 0.2112]

Pedagogical Note: If the engineer sets a learning rate eta > 0.2112, the quadratic curvature penalty dominates the linear descent term, causing the loss to explode instead of decrease.

Real-World Industry Case (Geophysics & Energy Exploration): Seismic Oil & Gas Subsurface Reservoir Inversion

Pipeline Workflow: Acoustic wave reflection sensors collect terabytes of subterranean echoes -> Deep neural acoustic inversion predicts subterranean rock density and porous oil pockets -> Highly ill-conditioned Hessian landscapes caused by varied rock strata require adaptive curvature optimization.

Industrial Impact & Benefit: Prevents million-dollar exploratory dry-well drilling by resolving deep geological formation boundaries.

Key Oral Viva Voce Checkpoints

Q1: What is a saddle point in a multivariable function?

Ans: A critical point where the gradient is zero, but the Hessian has both positive and negative eigenvalues, meaning it is a minimum along some directions and a maximum along others.

Q2: Why are saddle points more common than local minima in deep networks?

Ans: In an N-dimensional space, for a critical point to be a local minimum, all N eigenvalues of the Hessian must be simultaneously positive (probability ~ (1/2)^N for random signs). For large N, this probability vanishes, making saddle points exponentially more frequent.

Q3: What is the condition number of the Hessian matrix?

Ans: The ratio of its maximum eigenvalue to its minimum eigenvalue: kappa = lambda_max / lambda_min.

Q4: How does an ill-conditioned Hessian affect gradient descent?

Ans: It creates severe oscillations perpendicular to the ravine walls while causing agonizingly slow progress along the canyon floor.

Q5: What is a plateau in the loss surface?

Ans: A region where the loss surface is nearly flat with vanishingly small gradients across multiple dimensions, drastically slowing down training.

LECTURE #08 · MODULE 2 (Topic 2.1)

Date: 12-Sep-2025
Period: Period 1

Title: Introduction to Deep Learning and Deep Feed-Forward Networks

CO: CO2 Bloom's: Understand

Prerequisites: Multi-Layer Perceptrons, Linear Algebra, Machine Learning fundamentals
Key Terminology: Deep Learning, Deep Feedforward Network (DFN), Hierarchical Feature Extraction, Circuit Complexity, Representation Depth, Inductive Bias

Introduction

Deep learning is a subfield of machine learning based on artificial neural networks with representation learning. The adjective 'deep' refers to the use of multiple layers in the network. A deep feedforward network (DFN), also called a multilayer perceptron, defines a mapping y = f(x; theta) and learns the value of the parameters theta that result in the best function approximation.

Depth Vs Width

Theoretical justification for depth: 1. Exponential Efficiency: Certain functions requiring an exponential number of hidden units in a single-layer network can be approximated to arbitrary accuracy using a deep network with a polynomial number of units (Montufar et al., 2014; Telgarsky, 2016). 2. Compositional Hierarchy: Real-world perceptual data (vision, audio, natural language) is inherently compositional. Deep networks mirror this physics: early layers extract low-level primitives (edges, frequencies, phonemes), middle layers compose parts (textures, contours, syllables), and deep layers represent high-level semantic concepts (faces, words, objects). 3. Inductive Bias: Depth enforces the assumption that the target function is a composition of simpler functions, greatly constraining the search space and improving sample efficiency.

Mathematical Composition

*A deep network with L hidden layers defines a chain of function compositions: f(x) = f^(L)(f^(L-1)(... f^(2)(f^(1)(x)) ...)) where each layer computes: h^(l) = g^(l)((W^(l))^T * h^(l-1) + b^(l)), with h^(0) = x. The parameters are the collective set theta = { W^(1), b^(1), ..., W^(L), b^(L) }.*

Exam Important Points

- *Contrast shallow feature engineering with end-to-end deep representation learning.*
- *Why deep architectures are exponentially more parameter-efficient than shallow ones.*
- *Function composition formulation of Deep Feed-forward Networks.*
- *The three primary driving forces behind the deep learning revolution: Big Data, GPU parallelism, Algorithmic breakthroughs (ReLU, Adam, Dropout).*

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Layer Dimensions and Memory Footprint in Deep Feed-Forward Network

Statement: A deep feedforward network has architecture [100, 50, 25, 10, 2]. Each parameter and activation is stored as a 32-bit floating-point number (4 bytes). Calculate: (a) Total number of weight parameters, (b) Total number of bias parameters, (c) Parameter memory consumption in kilobytes, and (d) Activation memory required to store the forward pass of a mini-batch of 64 samples.

Given: Dims: [100, 50, 25, 10, 2], float32 (4 bytes/val), batch size B = 64.

Governing Formula: W_total = sum(n_l * n_{l-1}); B_total = sum(n_l); Params_mem = (W_total + B_total)*4 / 1024; Act_mem = B * sum(n_l)*4 / 1024

Step-by-Step Calculation:

Step 1: Weights calculation: W1 = 50 * 100 = 5,000 W2 = 25 * 50 = 1,250 W3 = 10 * 25 = 250 W4 = 2 * 10 = 20 Total Weights = 5,000 + 1,250 + 250 + 20 = 6,520.

Step 2: Biases calculation: B1 = 50, B2 = 25, B3 = 10, B4 = 2 Total Biases = 50 + 25 + 10 + 2 = 87.

Step 3: Total Parameters = 6,520 + 87 = 6,607 parameters. Parameter Memory = 6,607 * 4 bytes = 26,428 bytes = 25.81 KB.

Step 4: Activations per sample (all layers 1 to 4) = 50 + 25 + 10 + 2 = 87 activations. For batch of 64: Total activation floats = 64 * 87 = 5,568 floats. Activation Memory = 5,568 * 4 bytes = 22,272 bytes = 21.75 KB.

[FINAL EVALUATED ANSWER: Weights = 6,520, Biases = 87, Total Params = 6,607 (25.81 KB), Batch Activation Memory = 21.75 KB]

Pedagogical Note: In deep feedforward networks, parameter storage is modest for dense layers of moderate width, but activation caching scales linearly with mini-batch size.

Problem 2 (Advanced (KTU Exam Level)): Theoretical Piecewise Linear Regions Bound in Deep vs Shallow ReLU Networks

Statement: According to Montufar et al. (2014), a deep neural network with L layers of width n utilizing ReLU activations can partition the input space into Omega((n/n_in)^(n_in * (L-1)) * n^n_in) linear regions, whereas a shallow 1-hidden-layer network with total neurons N can partition the space into at most sum_{j=0}^{n_in} binom(N, j) regions. For an input dimension n_in = 2: (a) Calculate the upper bound of linear regions for a shallow network with N = 40 neurons, and (b) Compute the exponential lower bound for a deep network having L = 4 hidden layers of width n = 10 each (same total neurons N = 40).

Given: n_in = 2, Shallow: N = 40; Deep: 4 layers with n = 10 each (total 40 neurons).

Governing Formula: Shallow bound = binom(N, 0) + binom(N, 1) + binom(N, 2) = 1 + N + N*(N-1)/2; Deep bound approx (n/2)^(2*(L-1)) * n^2 = (10/2)^6 * 10^2 = 5^6 * 100

Step-by-Step Calculation:

Step 1: Compute shallow network linear regions: Regions_shallow = 1 + 40 + (40 * 39 / 2) = 1 + 40 + 780 = 821 regions.

Step 2: Compute deep network linear regions: With L=4 layers of width 10, the composition of folding piecewise linear operations scales as (n/2)^(d*(L-1)) * n^d. For d = 2, (10/2)^(2 * (4 - 1)) * 10^2 = 5^(6) * 100 = 15,625 * 100 = 1,562,500 regions!

[FINAL EVALUATED ANSWER: Shallow network (40 neurons) = 821 regions. Deep network (4x10 = 40 neurons) = ~1,562,500 regions.]

Pedagogical Note: With the exact same total budget of 40 hidden neurons, distributing them across 4 sequential layers creates over 1,900 times more expressive piecewise linear decision boundaries than arranging them in a single shallow layer.

Real-World Industry Case (Smart Industry 4.0 & Industrial Internet of Things (IIoT)): Industrial High-Dimensional Predictive Maintenance

Pipeline Workflow: High-frequency vibration, thermal, and acoustic sensor arrays on wind turbines -> 6-layer Deep Feedforward Network extracts non-linear failure precursors -> Predicts remaining useful life (RUL) in hours.

Industrial Impact & Benefit: Enables proactive component replacement prior to catastrophic gearbox seizure, saving over $250,000 per turbine incident.

Key Oral Viva Voce Checkpoints

Q1: What distinguishes deep learning from classical machine learning?

Ans: Deep learning automatically extracts hierarchical feature representations directly from raw data without manual feature engineering.

Q2: Why is a deep feedforward network called 'feedforward'?

Ans: Because information flows strictly forward from input nodes through hidden units to output nodes without feedback connections or recurrence.

Q3: What is the theoretical advantage of depth over width?

Ans: Deep architectures can compute complex functions with exponentially fewer parameters by reusing sub-features computed in earlier layers.

Q4: What is an inductive bias in neural networks?

Ans: The set of architectural assumptions the model uses to predict outputs on unseen inputs, such as hierarchical compositionality in deep networks.

Q5: Why did deep networks fail to train effectively prior to 2006-2010?

Ans: Lack of massive labeled datasets, lack of GPU compute, poor weight initialization, and vanishing gradients caused by sigmoid/tanh activations.

LECTURE #09 · MODULE 2 (Topic 2.2)

Date: 15-Sep-2025
Period: Period 2

Title: Training Deep Models: Optimization and Regularization Overview

CO: CO2 Bloom's: Understand

Prerequisites: Loss Functions, Gradient Calculus, Day 8 concepts
Key Terminology: Optimization vs Generalization, Surrogate Loss Function, Regularization, Generalization Error, Double Descent, Bias-Variance Tradeoff

Optimization Vs Machine Learning

In traditional numerical optimization, the objective is to minimize a known function J(theta). In machine learning, our true goal is to minimize the expected risk over the unseen data-generating distribution p_data(x, y): J(theta) = E_{(x,y)~p_data} [L(f(x; theta), y)] We can only compute the empirical risk on training data. Moreover, machine learning often optimizes a surrogate loss function (e.g. cross-entropy loss) because the true metric (0-1 classification error) is non-smooth, non-differentiable, and NP-hard to optimize.*

Regularization Definition

Ian Goodfellow defines regularization as: 'Any modification we make to a learning algorithm that is intended to reduce its generalization error but not its training error.' Regularization Taxonomy in Deep Learning: 1. Parameter Penalties: Adding norm constraints to the objective (L1, L2 weight decay). 2. Explicit Constraints: Hard parameter projection, weight clipping. 3. Dataset Modification: Data augmentation, input noise injection. 4. Architectural Priors: Convolutional parameter sharing, pooling. 5. Stochastic Noise: Dropout, DropConnect, label smoothing. 6. Algorithmic Dynamics: Early stopping, learning rate scheduling.

Bias Variance And Double Descent

Classical Regime: As model capacity increases, bias decreases monotonically while variance increases, producing a U-shaped test error curve. Modern Interpolation Regime (Belkin et al., 2019): Beyond the interpolation threshold (where the network perfectly memorizes training data with zero training error), test error undergoes a 'Double Descent' and decreases again! High overparameterization combined with stochastic gradient descent implicitly selects minimum-norm interpolating solutions that generalize remarkably well.

Exam Important Points

- *Formal definition of Regularization in Deep Learning.*
- *Difference between pure optimization and machine learning optimization.*
- *Why surrogate loss functions are necessary.*
- *Sketch and explanation of classical bias-variance tradeoff vs modern double descent curve.*

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Empirical Risk vs Surrogate Loss Evaluation

Statement: A binary classifier evaluates 4 test samples with true labels y in {0, 1}. The predicted probabilities p = P(y=1) are [0.90, 0.40, 0.20, 0.85] for true labels [1, 1, 0, 0]. Using a decision threshold of 0.5: (a) Calculate the 0-1 classification error (true empirical risk), and (b) Calculate the Binary Cross-Entropy surrogate loss.

Given: y = [1, 1, 0, 0], p = [0.90, 0.40, 0.20, 0.85], threshold = 0.5.

Governing Formula: 0-1 error: I(y_pred != y); L_BCE = - (1/N) * sum [y_i * ln(p_i) + (1 - y_i) * ln(1 - p_i)]

Step-by-Step Calculation:

Step 1: Compute threshold predictions: p1=0.90 -> 1 (Correct, error=0) p2=0.40 -> 0 (Incorrect, error=1) p3=0.20 -> 0 (Correct, error=0) p4=0.85 -> 1 (Incorrect, error=1). Total misclassifications = 2 out of 4.

Step 2: Calculate 0-1 error: Error = 2 / 4 = 0.50 (50%).

Step 3: Calculate individual cross-entropy losses: L1 = - ln(0.90) = - (-0.1054) = 0.1054 L2 = - ln(0.40) = - (-0.9163) = 0.9163 L3 = - ln(1 - 0.20) = - ln(0.80) = - (-0.2231) = 0.2231 L4 = - ln(1 - 0.85) = - ln(0.15) = - (-1.8971) = 1.8971.

Step 4: Average BCE loss: L_avg = (0.1054 + 0.9163 + 0.2231 + 1.8971) / 4 = 3.1419 / 4 = 0.7855.

[FINAL EVALUATED ANSWER: 0-1 Classification Error = 0.50 (50%), Cross-Entropy Surrogate Loss = 0.7855]

Pedagogical Note: Sample 4 contributed the largest penalty (1.8971) because the model was highly confident (85%) in the wrong direction, demonstrating why surrogate log loss provides a smooth, informative gradient for training.

Problem 2 (Advanced (KTU Exam Level)): Bias-Variance Decomposition of Mean Squared Error

Statement: The expected generalization error of a machine learning estimator f_hat(x) can be decomposed as E[(y - f_hat(x))^2] = Bias^2[f_hat(x)] + Var[f_hat(x)] + sigma_epsilon^2, where the true target is y = f(x) + epsilon with noise variance sigma_epsilon^2 = 0.05. An unregularized deep network has Bias = 0.10 and Variance = 0.40. An appropriately regularized network reduces Variance to 0.12 while slightly increasing Bias to 0.15. Calculate the total expected test MSE for both models and determine the percentage reduction achieved through regularization.

Given: sigma_epsilon^2 = 0.05. Model 1 (Unreg): Bias = 0.10, Var = 0.40. Model 2 (Reg): Bias = 0.15, Var = 0.12.

Governing Formula: MSE = Bias^2 + Variance + Irreducible_Noise; Reduction = (MSE1 - MSE2) / MSE1 * 100

Step-by-Step Calculation:

Step 1: Compute MSE for Unregularized Model 1: Bias^2 = (0.10)^2 = 0.010. Var = 0.40. Noise = 0.05. MSE_1 = 0.010 + 0.40 + 0.05 = 0.460.

Step 2: Compute MSE for Regularized Model 2: Bias^2 = (0.15)^2 = 0.0225. Var = 0.12. Noise = 0.05. MSE_2 = 0.0225 + 0.12 + 0.05 = 0.1925.

Step 3: Calculate percentage error reduction: Delta = 0.460 - 0.1925 = 0.2675. Percentage Reduction = (0.2675 / 0.460) * 100 = 58.15%.

[FINAL EVALUATED ANSWER: Unregularized MSE = 0.460, Regularized MSE = 0.1925 (58.15% error reduction)]

Pedagogical Note: Although regularization slightly increased the model's squared bias (+0.0125), it slashed variance by 0.28, cutting overall generalization error by over 58%.

Real-World Industry Case (Quantitative Finance & Algorithmic Execution): Real-Time Stock Market Algorithmic Trading

Pipeline Workflow: High-frequency limit order book feeds tick-level pricing into deep network -> Pure unregularized optimization overfits to market microstructure noise -> Multi-tier regularization (L2 penalty + early stopping) ensures model generalizes across volatile macroeconomic shifts.

Industrial Impact & Benefit: Prevents catastrophic portfolio drawdown during unexpected market volatility spikes.

Key Oral Viva Voce Checkpoints

Q1: What is Goodfellow's formal definition of regularization?

Ans: Any modification made to a learning algorithm intended to reduce its generalization error but not its training error.

Q2: What is a surrogate loss function?

Ans: A smooth, differentiable proxy loss (such as cross-entropy) optimized in place of the intractable true objective (such as 0-1 misclassification error).

Q3: Why is 0-1 loss intractable for gradient-based training?

Ans: Its derivative is zero almost everywhere and undefined at the decision boundary, offering no gradient direction for parameter updates.

Q4: Explain the concept of 'Double Descent'.

Ans: The observation that as model capacity increases past the interpolation threshold (where training error reaches zero), test error decreases again rather than continually climbing.

Q5: What is the difference between empirical risk and true risk?

Ans: Empirical risk is the average loss on the observed training sample; true risk is the expected loss over the entire underlying data distribution.

LECTURE #10 · MODULE 2 (Topic 2.3)

Date: 17-Sep-2025
Period: Period 4

Title: Gradient Descent (GD) and GD with Momentum

CO: CO2 Bloom's: Understand / Apply

Prerequisites: Vector Calculus (gradient vector), First-order Taylor Series, Day 7 concepts
Key Terminology: Batch Gradient Descent, Momentum, Velocity Vector, Friction Coefficient (Beta), Exponentially Weighted Moving Average, Ravine Oscillation

Gradient Descent

Standard Gradient Descent (Cauchy, 1847) updates parameters theta in the direction of the negative gradient: theta_{t+1} = theta_t - eta * grad_theta J(theta_t) Failure Modes: 1. Ravines (Ill-conditioned curvature): The gradient points across the steep valley rather than along the gentle slope toward the minimum, causing severe zig-zagging. 2. Saddle Points & Plateaus: Near critical points, ||grad J|| -> 0, causing the parameter updates to stall. 3. Local Minima: Without kinetic energy, GD is permanently trapped in any local depression.

Momentum Algorithm

Polyak (1964) introduced the heavy-ball / momentum method, simulating a physical particle with mass moving down a potential energy surface subject to friction. Mathematical Formulation: v_{t+1} = beta * v_t + (1 - beta) * grad_theta J(theta_t) [or simply: v_{t+1} = beta * v_t - eta * grad J] theta_{t+1} = theta_t + v_{t+1} (where beta in [0, 1) is the momentum coefficient, commonly set to 0.9). Mechanisms of Acceleration: - In directions where gradients alternate signs (oscillating valley walls), consecutive updates cancel out, damping oscillations. - In directions where gradients consistently point in the same direction, velocity accumulates up to a terminal factor of 1 / (1 - beta) times the base step size! - For beta = 0.9, the effective step size along consistent slopes is amplified tenfold: 1 / (1 - 0.9) = 10x.

Exam Important Points

- Full equations for Vanilla GD vs GD with Momentum.
- Physical interpretation of the momentum parameter beta as friction/inertia.
- Mathematical derivation of the terminal velocity factor 1 / (1 - beta).
- Graphical comparison of trajectories on an ill-conditioned quadratic bowl.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Two-Step Momentum Velocity and Weight Calculation

Statement: An optimizer with momentum coefficient beta = 0.9 and learning rate eta = 0.1 starts with initial parameter w_0 = 5.0 and initial velocity v_0 = 0.0. The gradients observed at step 0 and step 1 are g_0 = 2.0 and g_1 = 1.8. Using the update rule v_{t+1} = beta * v_t - eta * g_t, w_{t+1} = w_t + v_{t+1}, calculate the exact velocity and parameter values at steps t = 1 and t = 2.

Given: w_0 = 5.0, v_0 = 0.0, beta = 0.9, eta = 0.1, g_0 = 2.0, g_1 = 1.8
Governing Formula: v_{t+1} = beta * v_t - eta * g_t; w_{t+1} = w_t + v_{t+1}

Step-by-Step Calculation:

Step 1 (t = 1): v_1 = (0.9 * 0.0) - (0.1 * 2.0) = 0.0 - 0.20 = -0.20. w_1 = 5.0 + (-0.20) = 4.80.
Step 2 (t = 2): v_2 = (0.9 * -0.20) - (0.1 * 1.8) = -0.18 - 0.18 = -0.36. w_2 = 4.80 + (-0.36) = 4.44.

[FINAL EVALUATED ANSWER: At step 1: v_1 = -0.20, w_1 = 4.80. At step 2: v_2 = -0.36, w_2 = 4.44.]
Pedagogical Note: Even though the gradient decreased from 2.0 to 1.8, the accumulated velocity increased from -0.20 to -0.36, accelerating the parameter step toward the minimum.

Problem 2 (Advanced (KTU Exam Level)): KTU Model Paper Q13(b): GD with vs without Momentum on Plateau

Statement: Differentiate gradient descent with and without momentum. Consider a flat plateau where the gradient is constant at g = 0.05. The learning rate is eta = 0.02 and momentum factor beta = 0.9. (a) Calculate the steady-state parameter step size for vanilla GD, (b) Derive the terminal velocity step size for GD with momentum, and (c) Determine how many times faster momentum traverses the plateau compared to vanilla GD.

Given: Constant gradient g = 0.05, eta = 0.02, beta = 0.9.

Governing Formula: Vanilla step = - eta * g; Terminal velocity v_inf = - (eta * g) / (1 - beta); Speedup = v_inf / (eta * g) = 1 / (1 - beta)

Step-by-Step Calculation:

Step 1: Compute Vanilla GD step size: step_gd = eta * g = 0.02 * 0.05 = 0.0010.

Step 2: Derive Momentum recursive sequence under constant gradient g: v_1 = - eta * g v_2 = beta * v_1 - eta * g = - eta * g * (1 + beta) v_t = - eta * g * sum_{k=0}^{t-1} beta^k. As t -> infinity, sum_{k=0}^infinity beta^k = 1 / (1 - beta).

Step 3: Calculate terminal velocity: v_inf = - (0.02 * 0.05) / (1 - 0.9) = - 0.0010 / 0.10 = - 0.0100.

Step 4: Compute acceleration ratio: Ratio = |v_inf| / step_gd = 0.0100 / 0.0010 = 10.0x.

[FINAL EVALUATED ANSWER: Vanilla step = 0.0010, Momentum terminal step = 0.0100. Speedup = 10x faster.]

Pedagogical Note: On flat plateaus or persistent gradients, momentum accelerates progress tenfold by accumulating velocity over past steps.

Real-World Industry Case (Robotics & Real-Time Motion Control): Robotic Arm Inverse Kinematics Trajectory Planning

Pipeline Workflow: 6-DOF robotic manipulator computes joint angles for target end-effector coordinates -> High non-linearity creates flat plateaus and steep singularities in joint space -> Momentum-accelerated optimization prevents joint velocity stalling.

Industrial Impact & Benefit: Reduces robotic pick-and-place cycle times on automotive assembly lines by 24%.

Key Oral Viva Voce Checkpoints

Q1: What is the physical analogy behind gradient descent with momentum?

Ans: A heavy ball rolling down a potential energy surface, accumulating kinetic momentum while subject to viscous friction.

Q2: What is the typical value used for the momentum hyperparameter beta?

Ans: 0.9 (and occasionally 0.99 for deep architectures).

Q3: How does momentum resolve the problem of oscillating gradients in narrow ravines?

Ans: Oscillating components across the valley walls have opposing signs and cancel each other out over successive steps, while consistent components along the ravine floor reinforce each other.

Q4: What is the terminal velocity amplification factor of momentum?

Ans: 1 / (1 - beta); for beta = 0.9, the effective step size is amplified by a factor of 10.

Q5: Can momentum cause the optimizer to overshoot the global minimum?

Ans: Yes, if the accumulated velocity is too high, it may overshoot the minimum and oscillate around it before settling.

LECTURE #11 · MODULE 2 (Topic 2.4)

Date: 19-Sep-2025
Period: Period 1

Title: Nesterov Accelerated Gradient (NAG) and Stochastic Gradient Descent (SGD)

CO: CO2 Bloom's: Understand / Apply

Prerequisites: Gradient Descent, Momentum Algorithm, Day 10 concepts
Key Terminology: Nesterov Accelerated Gradient, Look-Ahead Step, Stochastic Gradient Descent, Mini-Batch SGD, Robbins-Monro Conditions, Gradient Noise

Nesterov Accelerated Gradient

Yurii Nesterov (1983) modified classical momentum by computing the gradient not at the current position theta_t, but at the 'look-ahead' position (theta_t + beta * v_t): NAG Equations: 1. Look-ahead projection: theta_lookahead = theta_t + beta * v_t 2. Evaluate gradient at projected point: g_t = grad_theta J(theta_lookahead) 3. Update velocity: v_{t+1} = beta * v_t - eta * g_t 4. Update parameters: theta_{t+1} = theta_t + v_{t+1} Advantage: NAG acts as an adaptive brake. If the current velocity is carrying the parameters uphill, the look-ahead gradient detects the slope increase BEFORE the step is taken and immediately applies a counter-acting brake, significantly reducing overshooting.

Stochastic Gradient Descent

Standard Batch GD computes the exact gradient over all N training examples: cost O(N) per step. For millions of samples, this is computationally prohibitive. Stochastic Gradient Descent (Robbins & Monro, 1951) approximates the true gradient using a single randomly chosen sample (or a mini-batch B of size 32 to 512): g_t = (1 / |B|) * sum_{i in B} grad_theta L(f(x_i; theta), y_i) theta_{t+1} = theta_t - eta_t * g_t Robbins-Monro Convergence Conditions: To guarantee almost-sure convergence to a local minimum under stochastic noise: 1. sum_{t=1}^infinity eta_t = infinity (learning rate must allow traversing any distance) 2. sum_{t=1}^infinity eta_t^2 < infinity (learning rate must shrink fast enough to damp noise)

Convergence Comparison

Theoretical convergence rates on convex L-smooth functions: - Standard Gradient Descent: O(1 / k) after k iterations. - Nesterov Accelerated Gradient: O(1 / k^2) — proved to be the optimal theoretical lower bound for first-order black-box optimization!

Exam Important Points

- Formulate NAG update equations and contrast with classical Polyak momentum.
- Diagram illustrating the look-ahead step in Nesterov momentum.
- Formulation of Mini-Batch SGD and its computational trade-offs.
- Robbins-Monro conditions for convergence of stochastic gradient descent.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Nesterov Look-Ahead Position and Gradient Evaluation

Statement: An optimizer using Nesterov momentum has current parameter w_t = 2.0, current velocity v_t = -0.5, and momentum factor beta = 0.9. The loss function is J(w) = 0.5 * w^2. Learning rate is eta = 0.1. (a) Calculate the look-ahead position w_ahead, (b) Compute the gradient at the look-ahead position, and (c) Compute the updated velocity v_{t+1} and new parameter w_{t+1}.

Given: w_t = 2.0, v_t = -0.5, beta = 0.9, J(w) = 0.5 * w^2, eta = 0.1.

Governing Formula: w_ahead = w_t + beta * v_t; g_t = dJ/dw(w_ahead) = w_ahead; v_{t+1} = beta * v_t - eta * g_t; w_{t+1} = w_t + v_{t+1}

Step-by-Step Calculation:

Step 1: Compute look-ahead position: w_ahead = 2.0 + (0.9 * -0.5) = 2.0 - 0.45 = 1.55.
Step 2: Evaluate gradient at look-ahead position: g_ahead = dJ/dw(1.55) = 1.55.
Step 3: Update velocity: v_{t+1} = (0.9 * -0.5) - (0.1 * 1.55) = -0.45 - 0.155 = -0.605.
Step 4: Update parameter: w_{t+1} = 2.0 + (-0.605) = 1.395.

[FINAL EVALUATED ANSWER: w_ahead = 1.55, Gradient = 1.55, v_{t+1} = -0.605, w_{t+1} = 1.395]

Pedagogical Note: Nesterov evaluated the slope where momentum was about to carry the parameter (at 1.55) rather than at the stale position (2.0), adjusting velocity dynamically.

Problem 2 (Advanced (KTU Exam Level)): Stochastic Gradient Variance Reduction with Mini-Batch Size

Statement: Let the true population gradient be g_true with variance Var(g_single) = sigma^2 = 16.0 when computed on a single random sample. (a) Calculate the variance of the mini-batch gradient estimator for batch sizes B = 1, B = 16, B = 64, and B = 256, (b) Determine the standard error of the gradient estimate for each batch size, and (c) Discuss the computational trade-off between variance reduction and matrix multiply saturation on a GPU.

Given: sigma^2 = 16.0 for B = 1.

Governing Formula: Var(g_B) = sigma^2 / B; SE(g_B) = sqrt(Var(g_B)) = sigma / sqrt(B)

Step-by-Step Calculation:

Step 1: For B = 1: Var = 16.0 / 1 = 16.0; SE = sqrt(16.0) = 4.00.
Step 2: For B = 16: Var = 16.0 / 16 = 1.0; SE = sqrt(1.0) = 1.00.
Step 3: For B = 64: Var = 16.0 / 64 = 0.25; SE = sqrt(0.25) = 0.50.
Step 4: For B = 256: Var = 16.0 / 256 = 0.0625; SE = sqrt(0.0625) = 0.25.

[FINAL EVALUATED ANSWER: B=1: SE=4.00; B=16: SE=1.00; B=64: SE=0.50; B=256: SE=0.25.]

Pedagogical Note: Increasing batch size from 1 to 64 reduces gradient standard error by a factor of 8 (from 4.0 to 0.5). Beyond B=256, returns diminish as diminishing variance gains require quadrupling the batch size to halve error, while GPU memory fills.

Real-World Industry Case (Edge AI & Drone Surveillance Systems): Real-Time Embedded Video Object Tracking

Pipeline Workflow: Drone onboard camera streams 60 fps 4K video -> High frame rate requires sub-15ms model updates on edge GPU -> Mini-Batch SGD with Nesterov momentum optimizes lightweight detector parameters on-the-fly without tracking lag.

Industrial Impact & Benefit: Enables uninterrupted drone camera lock-on for search-and-rescue target tracking in mountain terrain.

Key Oral Viva Voce Checkpoints

Q1: What is the key difference between Polyak's standard momentum and Nesterov momentum?

Ans: Polyak evaluates the gradient at the current position theta_t, whereas Nesterov evaluates the gradient at the look-ahead position (theta_t + beta*v_t).

Q2: What is the convergence rate of Nesterov Accelerated Gradient for convex smooth functions?

Ans: O(1 / k^2), which is significantly faster than standard GD's O(1 / k).

Q3: Why is pure SGD (batch size = 1) computationally inefficient on GPUs?

Ans: GPUs achieve high compute utilization through massive vector and matrix parallelization; a single sample fails to saturate thousands of CUDA cores.

Q4: State the Robbins-Monro conditions for stochastic optimization.

Ans: The learning rate sequence eta_t must satisfy: sum_{t=1}^infty eta_t = infty and sum_{t=1}^infty eta_t^2 < infty.

Q5: Why is the first Robbins-Monro condition necessary?

Ans: To ensure the optimizer has sufficient energy to travel an arbitrarily large distance in parameter space to reach the minimum regardless of initialization.

LECTURE #12 · MODULE 2 (Topic 2.5)

Date: 22-Sep-2025
Period: Period 2

Title: Adaptive Learning Rate Methods: AdaGrad, RMSProp, and Adam

CO: CO2 Bloom's: Understand / Apply

Prerequisites: Gradient Descent with Momentum, Element-wise vector operations, Day 11 concepts
Key Terminology: AdaGrad, RMSProp, Adam (Adaptive Moment Estimation), Second Moment Accumulator, Exponential Moving Average, Bias Correction, Per-Coordinate Learning Rate

Adagrad

AdaGrad (Duchi, Hazan, Singer, 2011) adapts the learning rate to each individual parameter based on the history of past squared gradients: 1. Accumulate squared gradients: s_t = s_{t-1} + g_t odot g_t 2. Parameter update: theta_{t+1} = theta_t - (eta / sqrt(s_t + epsilon)) odot g_t Merit: Automatically takes large steps for infrequent/sparse features and small steps for frequent features. Fatal Flaw: Since g_t^2 >= 0, s_t increases monotonically with every training step. Consequently, the effective learning rate eta / sqrt(s_t) strictly decays to zero, prematurely terminating learning before the network reaches the optimum!

Rmsprop

RMSProp (Geoff Hinton, Coursera Lecture 6e, 2012) fixes AdaGrad's premature freezing by replacing the monotonic sum with an Exponentially Weighted Moving Average (EWMA): 1. Update discounted second moment: s_t = beta_2 * s_{t-1} + (1 - beta_2) * (g_t odot g_t) (typically beta_2 = 0.9 or 0.99) 2. Parameter update: theta_{t+1} = theta_t - (eta / sqrt(s_t + epsilon)) odot g_t This allows RMSProp to discard stale gradient history and adapt dynamically to recent curvature.

Adam

Adam (Kingma & Ba, ICLR 2015) combines the advantages of Momentum (first moment estimation) and RMSProp (second moment estimation): 1. First moment (mean of gradient): m_t = beta_1 * m_{t-1} + (1 - beta_1) * g_t (default beta_1 = 0.9) 2. Second moment (uncentered variance): v_t = beta_2 * v_{t-1} + (1 - beta_2) * (g_t odot g_t) (default beta_2 = 0.999) 3. Bias corrections (counteracting initialization at zero): m_hat_t = m_t / (1 - beta_1^t) v_hat_t = v_t / (1 - beta_2^t) 4. Parameter update: theta_{t+1} = theta_t - (eta / (sqrt(v_hat_t) + epsilon)) odot m_hat_t (default eta = 0.001, epsilon = 1e-8)

Exam Important Points

- KTU Exam Q13(a): Mathematical explanation of AdaGrad gradient decay and why it halts training.
- Comparison table: SGD vs AdaGrad vs RMSProp vs Adam.
- Full mathematical derivation of Adam's bias correction terms 1 / (1 - beta^t).
- Why Adam is the de facto default optimizer across modern deep learning architectures.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): AdaGrad vs RMSProp Two-Step Accumulator Comparison

Statement: A parameter w has initial value 1.0. At step 1 and step 2, the observed gradients are g1 = 4.0 and g2 = 4.0. The base learning rate is eta = 0.5 and epsilon = 1e-8. For RMSProp, use decay factor beta_2 = 0.9. Calculate: (a) AdaGrad's accumulator s_1, s_2 and effective learning rates, and (b) RMSProp's accumulator s_1, s_2 and effective learning rates.

Given: g1 = 4.0, g2 = 4.0, eta = 0.5, epsilon = 1e-8, RMSProp beta_2 = 0.9.

Governing Formula: AdaGrad: s_t = s_{t-1} + g_t^2; RMSProp: s_t = beta_2 * s_{t-1} + (1 - beta_2) * g_t^2; eta_eff = eta / sqrt(s_t + eps)

Step-by-Step Calculation:

Step 1: AdaGrad calculations: s_1 = 0 + 4.0^2 = 16.0 -> eta_eff_1 = 0.5 / sqrt(16.0) = 0.5 / 4.0 = 0.1250. s_2 = 16.0 + 4.0^2 = 16.0 + 16.0 = 32.0 -> eta_eff_2 = 0.5 / sqrt(32.0) = 0.5 / 5.6569 = 0.0884.

Step 2: RMSProp calculations: s_1 = (0.9 * 0) + (1 - 0.9) * 4.0^2 = 0.1 * 16.0 = 1.60. eta_eff_1 = 0.5 / sqrt(1.60) = 0.5 / 1.2649 = 0.3953. s_2 = (0.9 * 1.60) + (0.1 * 16.0) = 1.44 + 1.60 = 3.04. eta_eff_2 = 0.5 / sqrt(3.04) = 0.5 / 1.7436 = 0.2868.

[FINAL EVALUATED ANSWER: AdaGrad: s_2 = 32.0, eta_eff = 0.0884. RMSProp: s_2 = 3.04, eta_eff = 0.2868.]

Pedagogical Note: AdaGrad's accumulator grew rapidly to 32.0, suffocating the effective step size by ~82%. RMSProp maintained a balanced discounted accumulator (3.04), allowing continued agile learning.

Problem 2 (Advanced (KTU Exam Level)): Adam Bias Correction Derivation and Step Calculation

Statement: At step t = 1, an Adam optimizer observes gradient g_1 = 2.0. Hyperparameters are eta = 0.01, beta_1 = 0.9, beta_2 = 0.999, and epsilon = 1e-8. Initial moments are m_0 = 0.0, v_0 = 0.0. (a) Calculate the raw moments m_1 and v_1, (b) Calculate the bias-corrected moments m_hat_1 and v_hat_1, (c) Explain why bias correction is mandatory at early steps, and (d) Compute the parameter update step delta_w.

Given: t = 1, g_1 = 2.0, eta = 0.01, beta_1 = 0.9, beta_2 = 0.999, eps = 1e-8, m_0 = 0, v_0 = 0.

Governing Formula: m_1 = beta_1*m_0 + (1 - beta_1)*g_1; v_1 = beta_2*v_0 + (1 - beta_2)*g_1^2; m_hat = m_1 / (1 - beta_1^1); v_hat = v_1 / (1 - beta_2^1); delta_w = - (eta / (sqrt(v_hat) + eps)) * m_hat

Step-by-Step Calculation:

Step 1: Compute raw moments: m_1 = (0.9 * 0) + (1 - 0.9) * 2.0 = 0.1 * 2.0 = 0.20. v_1 = (0.999 * 0) + (1 - 0.999) * (2.0)^2 = 0.001 * 4.0 = 0.0040.

Step 2: Compute bias correction denominators: 1 - beta_1^1 = 1 - 0.9 = 0.10. 1 - beta_2^1 = 1 - 0.999 = 0.0010.

Step 3: Compute bias-corrected moments: m_hat_1 = 0.20 / 0.10 = 2.00. v_hat_1 = 0.0040 / 0.0010 = 4.00.

Step 4: Compute parameter update step: delta_w = - (0.01 / (sqrt(4.00) + 1e-8)) * 2.00 = - (0.01 / 2.0) * 2.00 = - 0.0100.

[FINAL EVALUATED ANSWER: Raw: m_1=0.20, v_1=0.0040. Corrected: m_hat_1=2.00, v_hat_1=4.00. Step delta_w = -0.0100]

Pedagogical Note: Without bias correction, v_1 would be 0.0040, giving sqrt(v_1) = 0.0632 and resulting in a wildly distorted, inflated initial step. Bias correction normalizes both moments to reflect the true expected gradient scale.

Real-World Industry Case (Autonomous Robotics & Edge Spatial Computing): Autonomous Drone LiDAR Simultaneous Localization and Mapping (SLAM)

Pipeline Workflow: Drone flies through unmapped indoor warehouse -> LiDAR point cloud features are sparse (certain spatial bins trigger rarely while floor points trigger constantly) -> Adam optimizer adapts individual coordinate step sizes dynamically.

Industrial Impact & Benefit: Enables rapid convergence of 3D spatial pose graph estimation in sub-second real-time constraints.

Key Oral Viva Voce Checkpoints

Q1: What core problem in vanilla GD did AdaGrad aim to solve?

Ans: The reliance on a single global learning rate for all parameters, regardless of how frequently or sparsely individual features occurred.

Q2: What is the fatal flaw of the AdaGrad optimizer?

Ans: The monotonic accumulation of squared gradients in the denominator causes the effective learning rate to decay to near zero, prematurely halting training.

Q3: How does RMSProp overcome AdaGrad's learning rate decay problem?

Ans: By using an exponentially weighted moving average of squared gradients instead of an unweighted cumulative sum.

Q4: What two algorithms are combined to create Adam?

Ans: Momentum (first moment of gradients) and RMSProp (second moment of gradients).

Q5: Why are bias corrections needed in the Adam optimizer?

Ans: Because the first and second moment vectors are initialized to zero, biasing their estimates towards zero during the initial training steps; dividing by (1 - beta^t) removes this bias.

LECTURE #13 · MODULE 2 (Topic 2.6)

Date: 24-Sep-2025
Period: Period 4

Title: Regularization Techniques: L1, L2, Early Stopping, and Dataset Augmentation

CO: CO2 Bloom's: Understand / Apply

Prerequisites: Loss Formulation, Gradient Updates, Optimization Overview, Day 9 concepts
Key Terminology: L1 Regularization (Lasso), L2 Regularization (Weight Decay / Ridge), Sparsity, Weight Shrinkage, Early Stopping, Patience, Dataset Augmentation

L1 And L2 Regularization

Regularized Objective Function: J_tilde(w) = J(w) + alpha * Omega(w) 1. L2 Regularization (Weight Decay / Ridge): Omega(w) = (1/2) * ||w||_2^2 = (1/2) * sum_i w_i^2 Gradient: grad J_tilde = grad J + alpha * w Update Step: w_{t+1} = w_t - eta * (grad J + alpha * w_t) = (1 - eta * alpha) * w_t - eta * grad J Effect: Multiplies the weight vector by a shrinkage factor (1 - eta * alpha) < 1 at each step, preventing weights from growing excessively large. 2. L1 Regularization (Lasso): Omega(w) = ||w||_1 = sum_i |w_i| Gradient: grad J_tilde = grad J + alpha * sign(w) Update Step: w_{t+1} = w_t - eta * grad J - eta * alpha * sign(w_t) Geometric Sparsity: The L1 constraint forms a diamond-shaped polytope with sharp corners on the coordinate axes. The elliptical loss contours intersect these corners, driving many weights to be exactly zero (feature selection).

Early Stopping

Early Stopping is the most widely used regularization technique in deep learning. Protocol: 1. At the end of each epoch, evaluate loss on a hold-out validation set. 2. If validation loss improves, save model parameters theta* = theta. 3. If validation loss does not improve for 'patience' P consecutive epochs, terminate training and restore theta*. Mathematical Equivalence: Bishop (1995) proved that on quadratic loss surfaces, early stopping after tau iterations is mathematically equivalent to L2 regularization with weight decay parameter alpha approx 1 / (eta * tau).

Dataset Augmentation

The most reliable method to improve generalization is to train on more data. When additional data is scarce, Dataset Augmentation generates realistic synthetic transformations: - Computer Vision: Random horizontal flips, random cropping, rotations (+-15 deg), color jitter, affine shearing, CutMix, MixUp. - Speech: Time stretching, pitch shifting, background noise injection (SpecAugment). - Text: Synonym replacement, back-translation (English -> German -> English).

Exam Important Points

- Derivation of weight shrinkage equation w <- (1 - eta*alpha)*w for L2 regularization.
- Geometric explanation with sketches showing why L1 produces sparsity while L2 shrinks weights uniformly.
- Early stopping protocol, patience parameter, and its equivalence to L2 regularization.
- List of data augmentation techniques for image classification.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): L2 Weight Decay Shrinkage Step Calculation

Statement: A neural network weight currently has value w_t = 0.80. The unregularized loss gradient with respect to this weight is dJ/dw = 0.10. The learning rate is eta = 0.05 and the L2 regularization coefficient is alpha = 0.20. Calculate: (a) The effective shrinkage factor (1 - eta * alpha), (b) The weight value after decay but before gradient application, and (c) The final updated weight w_{t+1}.

Given: w_t = 0.80, dJ/dw = 0.10, eta = 0.05, alpha = 0.20

Governing Formula: Shrinkage factor = 1 - eta * alpha; w_decayed = (1 - eta * alpha) * w_t; w_{t+1} = w_decayed - eta * (dJ/dw)

Step-by-Step Calculation:

Step 1: Compute shrinkage factor: factor = 1.0 - (0.05 * 0.20) = 1.0 - 0.010 = 0.990.

Step 2: Apply weight decay: w_decayed = 0.990 * 0.80 = 0.7920.

Step 3: Apply gradient descent step: grad_step = 0.05 * 0.10 = 0.0050. w_{t+1} = 0.7920 - 0.0050 = 0.7870.

[FINAL EVALUATED ANSWER: Shrinkage factor = 0.990, Decayed weight = 0.7920, Final weight w_{t+1} = 0.7870]

Pedagogical Note: L2 regularization systematically eroded the weight by 1.0% (from 0.80 to 0.792) prior to applying the empirical gradient descent update.

Problem 2 (Advanced (KTU Exam Level)): Optimal Weight Reduction under L2 Regularization on Quadratic Loss

Statement: Consider a scalar quadratic loss function J(w) = 0.5 * H * (w - w*)^2 where Hessian curvature H = 4.0 and unregularized optimum w* = 3.0. An L2 penalty term 0.5 * alpha * w^2 is added with alpha = 1.0. (a) Derive the expression for the regularized objective J_tilde(w), (b) Find the exact regularized optimal weight w_reg by setting dJ_tilde/dw = 0, and (c) Show that w_reg = (H / (H + alpha)) * w* and compute the percentage shrinkage.

Given: H = 4.0, w* = 3.0, alpha = 1.0, J_tilde(w) = 0.5 * H * (w - w*)^2 + 0.5 * alpha * w^2.

Governing Formula: dJ_tilde/dw = H*(w - w*) + alpha*w = 0 => (H + alpha)*w = H*w* => w_reg = (H / (H + alpha)) * w*

Step-by-Step Calculation:

Step 1: Differentiate regularized objective: dJ_tilde/dw = 4.0 * (w - 3.0) + 1.0 * w = 4.0*w - 12.0 + 1.0*w = 5.0*w - 12.0.

Step 2: Set derivative to zero to solve for optimum: 5.0 * w_reg - 12.0 = 0 => w_reg = 12.0 / 5.0 = 2.40.

Step 3: Verify theoretical ratio: Ratio = H / (H + alpha) = 4.0 / (4.0 + 1.0) = 4.0 / 5.0 = 0.80. w_reg = 0.80 * 3.0 = 2.40.

Step 4: Calculate shrinkage percentage: Shrinkage = ((3.0 - 2.40) / 3.0) * 100 = (0.60 / 3.0) * 100 = 20.0%.

[FINAL EVALUATED ANSWER: Regularized optimum w_reg = 2.40 (Exact 20% shrinkage from unregularized value 3.0)]

Pedagogical Note: The L2 penalty pulls the parameter toward the origin; directions with higher curvature H resist shrinkage more strongly than flatter directions with small H.

Real-World Industry Case (Healthcare Diagnostics & Radiomics): Medical Imaging CT Scan Lung Nodule Detection

Pipeline Workflow: Patient 3D CT pulmonary scans -> Extremely small training dataset (only 500 patient scans available) -> Massive 10x dataset augmentation (random 3D rotations, intensity scaling, elastic deformation) combined with L2 weight decay.

Industrial Impact & Benefit: Prevents catastrophic overfitting to patient scanner artifacts, boosting lung cancer early detection sensitivity to 96.4%.

Key Oral Viva Voce Checkpoints

Q1: Why does L1 regularization lead to sparse weight matrices while L2 does not?

Ans: L1 has a constant gradient magnitude alpha*sign(w) right down to the origin and its constraint surface has sharp corners on the axes where elliptical loss contours typically intersect; L2 gradient alpha*w shrinks proportionally to w, pulling weights close to zero without making them exactly zero.

Q2: What is the update rule for weight decay in L2 regularization?

Ans: w_{t+1} = (1 - eta * alpha) * w_t - eta * grad J(w_t).

Q3: What is the physical meaning of the 'patience' parameter in Early Stopping?

Ans: The number of consecutive epochs to wait for validation loss improvement before terminating training.

Q4: Why does Early Stopping prevent overfitting?

Ans: It stops optimization before the model begins memorizing idiosyncratic training set noise, constraining the parameter vector close to its initial state.

Q5: Can dataset augmentation be applied to tabular data?

Ans: Yes, techniques like SMOTE (Synthetic Minority Over-sampling Technique) or adding small Gaussian jitter to continuous features perform tabular augmentation.

LECTURE #14 · MODULE 2 (Topic 2.7)

Date: 26-Sep-2025
Period: Period 1

Title: Parameter Sharing, Parameter Tying, and Injecting Noise at Input

CO: CO2 Bloom's: Understand / Apply

FACULTY SYLLABUS NOTE: Official KTU 2.7 & 2.8 duplication flagged. This lecture covers: Parameter Sharing, Parameter Tying, and Injecting Noise at Input.

Prerequisites: Convolution Operation intuition, Autoencoder concepts, Regularization, Day 13 concepts
Key Terminology: Parameter Sharing, Parameter Tying, Tied Weights, Input Noise Injection, Tikhonov Regularization, Denoising Objective

Parameter Sharing And Tying

1. Parameter Sharing: Instead of assigning unique independent weights to every connection, parameter sharing forces multiple components to share the exact same parameter value. - Primary Example: Convolutional Neural Networks (CNNs). A convolutional filter (e.g. 3x3 kernel) is shared across all spatial locations of the input image. - Benefit: Slashes parameter count from millions to hundreds, enforces translation equivariance, and drastically lowers sample complexity. 2. Parameter Tying: Parameter tying imposes hard linear constraints between different weight matrices in a network. - Classic Example: Symmetric Autoencoders where decoder weights W_d are tied to the transpose of encoder weights W_e: W_d = W_e^T - Benefit: Halves the number of parameters in the autoencoder and acts as a strong regularizer preventing the latent representation from learning identity trivialities.

Injecting Noise At Input

Adding random noise epsilon ~ N(0, sigma^2 * I) to the input vector x during training: x_tilde = x + epsilon Mathematical Analysis (Bishop, 1995): For a linear or smooth model f(x) trained with squared error loss, minimizing the expected loss over noisy inputs: E_epsilon [0.5 * (f(x + epsilon) - y)^2] is mathematically equivalent to minimizing the uncorrupted loss PLUS an L2 regularization penalty proportional to the Frobenius norm of the model's Jacobian: L_regularized = L_clean + (1/2) * sigma^2 * || grad_x f(x) ||_2^2 This penalizes rapid changes in the model's prediction with respect to input perturbations, forcing the network to learn a smooth, robust decision boundary.

Exam Important Points

- Definition and architectural benefits of Parameter Sharing in CNNs and RNNs.
- Formulation of Tied Weights in Autoencoders: W_decoder = W_encoder^T.
- Mathematical proof that injecting input Gaussian noise is equivalent to Tikhonov gradient penalty regularization.
- How input noise improves model robustness against adversarial attacks.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Parameter Reduction Calculation in Tied Autoencoder

Statement: An autoencoder maps an input vector of dimension n_in = 784 into a compressed latent bottleneck of dimension n_h = 64, and then reconstructs the 784-dimensional output. (a) Calculate the total trainable weights and biases if encoder and decoder weights are UNTIED, (b) Calculate the total parameters if weights are TIED (W_decoder = W_encoder^T), and (c) Determine the exact percentage savings in parameter storage.

Given: n_in = 784, n_h = 64, n_out = 784.

Governing Formula: Untied Params = 2 * (n_in * n_h) + n_h + n_in; Tied Params = (n_in * n_h) + n_h + n_in

Step-by-Step Calculation:

Step 1: Compute Untied parameter count: Encoder weights = 64 * 784 = 50,176. Encoder biases = 64. Decoder weights = 784 * 64 = 50,176. Decoder biases = 784. Total Untied Parameters = 50,176 + 64 + 50,176 + 784 = 101,200.

Step 2: Compute Tied parameter count: Shared weight matrix W = 64 * 784 = 50,176. Encoder biases = 64. Decoder biases = 784. Total Tied Parameters = 50,176 + 64 + 784 = 51,024.

Step 3: Compute parameter reduction: Difference = 101,200 - 51,024 = 50,176 parameters. Percentage Savings = (50,176 / 101,200) * 100 = 49.58%.

[FINAL EVALUATED ANSWER: Untied = 101,200 params. Tied = 51,024 params (49.58% parameter reduction).]

Pedagogical Note: Tying weights virtually cuts the network weight parameters in half while enforcing symmetric encoder-decoder representations.

Problem 2 (Advanced (KTU Exam Level)): Mathematical Equivalence of Input Noise to Tikhonov Regularization

Statement: Let a linear model be f(x) = w^T * x with scalar squared error loss L = 0.5 * (f(x) - y)^2. Add isotropic Gaussian noise epsilon with mean 0 and variance sigma^2 * I to input x, giving perturbed input x_tilde = x + epsilon. (a) Expand the expected loss E_epsilon [0.5 * (w^T * (x + epsilon) - y)^2], and (b) Prove algebraically that the expected noisy loss equals the clean loss plus an exact L2 penalty 0.5 * sigma^2 * ||w||_2^2.

Given: f(x) = w^T * x, x_tilde = x + epsilon, E[epsilon] = 0, E[epsilon * epsilon^T] = sigma^2 * I.

Governing Formula: w^T * (x + epsilon) - y = (w^T*x - y) + w^T*epsilon; Expand square and take expectation.

Step-by-Step Calculation:

Step 1: Express error term: e(epsilon) = (w^T * x - y) + w^T * epsilon. Let e_clean = w^T * x - y.

Step 2: Square the error term: e(epsilon)^2 = e_clean^2 + 2 * e_clean * (w^T * epsilon) + (w^T * epsilon)^2.

Step 3: Take mathematical expectation with respect to noise distribution epsilon: E[e(epsilon)^2] = E[e_clean^2] + 2 * e_clean * w^T * E[epsilon] + E[(w^T * epsilon)^2].

Step 4: Since noise is zero-mean, E[epsilon] = 0, so the middle cross-term vanishes: 2 * e_clean * w^T * 0 = 0.

Step 5: Evaluate the quadratic noise term: (w^T * epsilon)^2 = (w^T * epsilon) * (epsilon^T * w) = w^T * (epsilon * epsilon^T) * w. E[w^T * (epsilon * epsilon^T) * w] = w^T * E[epsilon * epsilon^T] * w = w^T * (sigma^2 * I) * w = sigma^2 * ||w||_2^2.

Step 6: Combine results with the 0.5 loss factor: E[L(x_tilde)] = 0.5 * e_clean^2 + 0.5 * sigma^2 * ||w||_2^2 = L_clean + 0.5 * sigma^2 * ||w||_2^2.

[FINAL EVALUATED ANSWER: E[L(x_tilde)] = L_clean + 0.5 * sigma^2 * ||w||_2^2 (Exact L2 Regularization Penalty!)]

Pedagogical Note: This rigorous proof establishes that adding input Gaussian noise with variance sigma^2 is mathematically identical to applying explicit L2 weight decay with regularization coefficient alpha = sigma^2.

Real-World Industry Case (Audio Signal Processing & In-Cabin Infotainment): Noise-Resistant Automatic Speech Recognition in High-Speed Rail

Pipeline Workflow: High-speed rail passenger speaks voice commands into cabin microphone -> Severe acoustic rumble (wind shear, steel track vibration) -> Network trained with Gaussian noise injection at input audio spectrogram frames.

Industrial Impact & Benefit: Maintains 94% command recognition accuracy without acoustic degradation or word drops under 300 km/h train speeds.

Key Oral Viva Voce Checkpoints

Q1: What is Parameter Sharing in neural networks?

Ans: Using the same set of weights across multiple computational units or time steps, rather than learning independent weights for each connection.

Q2: Give two prominent examples of parameter sharing in modern architectures.

Ans: Convolutional Neural Networks (sharing filter kernels across image space) and Recurrent Neural Networks (sharing transition matrices across time steps).

Q3: What is Parameter Tying in Autoencoders?

Ans: Constraining the decoder weight matrix to be the exact transpose of the encoder weight matrix: W_decoder = W_encoder^T.

Q4: How does parameter tying benefit an autoencoder?

Ans: It halves the model parameter count, prevents overfitting, and prevents the hidden representation from learning a trivial identity mapping.

Q5: What happens mathematically when Gaussian noise is added to model inputs during training?

Ans: It acts as an implicit regularizer mathematically equivalent to an L2 weight penalty proportional to the noise variance.

LECTURE #15 · MODULE 2 (Topic 2.8)

Date: 29-Sep-2025
Period: Period 2

Title: Ensemble Methods, Model Averaging, Bagging and Boosting in Deep Learning

CO: CO2 Bloom's: Understand / Apply

FACULTY SYLLABUS NOTE: Official KTU 2.7 & 2.8 duplication flagged. This lecture covers: Ensemble Methods, Model Averaging, Bagging and Boosting in Deep Learning.

Prerequisites: Bias-Variance Decomposition, Generalization Theory, Day 14 concepts
Key Terminology: Ensemble Learning, Model Averaging, Bootstrap Aggregation (Bagging), Boosting, Snapshot Ensembles, Variance Reduction, Syllabus Duplication Verification

Faculty Syllabus Flag

FACULTY NOTICE: In the approved KTU CST414 teaching plan (Page 12 of DEEP LEARNING.pdf), topics 2.7 and 2.8 carry the identical title: 'Parameter sharing and tying, Injecting noise at input, Ensemble methods'. To ensure complete academic coverage without redundant lecturing, Day 14 covered Parameter Sharing, Tying, and Input Noise, while Day 15 is dedicated thoroughly to Ensemble Methods, Model Averaging, Bagging, and Snapshot Ensembles.

Model Averaging Mathematics

Ensemble learning combines the predictions of k distinct models {f_1(x), f_2(x), ..., f_k(x)}: f_ens(x) = (1 / k) * sum_{i=1}^k f_i(x) Variance Reduction Theorem: Suppose each model f_i has error epsilon_i with variance E[epsilon_i^2] = v and covariance E[epsilon_i * epsilon_j] = c between different models. The expected squared error of the ensemble predictor is: E[((1/k) * sum_i epsilon_i)^2] = (1 / k^2) * sum_i sum_j E[epsilon_i * epsilon_j] = (1 / k^2) * [k * v + k * (k - 1) * c] = (1 / k) * v + ((k - 1) / k) * c Two Extreme Cases: 1. Perfectly Correlated Models (c = v): Ensemble error = (1/k)*v + ((k-1)/k)*v = v. No gain. 2. Perfectly Uncorrelated Errors (c = 0): Ensemble error = v / k. Error decreases linearly with ensemble size k! In deep learning, even if models share the same architecture, different random weight initializations and mini-batch shuffle orders lead to partially uncorrelated errors (c < v), guaranteeing that the ensemble outperforms any single constituent model.

Deep Ensemble Techniques

1. Bagging (Bootstrap Aggregation): Train k identical architectures on k bootstrap resamples of the dataset. 2. Multi-Architecture Ensembling: Average predictions of diverse models (e.g. ResNet + DenseNet + Vision Transformer). 3. Snapshot Ensembles (Huang et al., 2017): Use a cyclical learning rate schedule to converge to multiple local minima during a single training run, saving model snapshots at the base of each cycle without k-fold training cost.

Exam Important Points

- Mathematical derivation of ensemble error: E[e^2] = (1/k)*v + ((k-1)/k)*c.
- Why ensemble error strictly improves whenever model error correlation c < v.
- Comparison of Bagging vs Boosting in deep learning contexts.
- Snapshot ensembles: achieving ensemble benefits in a single training run.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Theoretical Ensemble Error Variance Calculation

Statement: An ensemble is composed of k = 5 independently trained deep neural networks. Each individual network has an expected squared error variance of v = 0.20. The average pairwise covariance between the errors of different models is c = 0.05. (a) Calculate the expected squared error of the ensemble predictor, and (b) Calculate the percentage reduction in error achieved compared to using any single individual model.

Given: k = 5 models, individual variance v = 0.20, covariance c = 0.05.

Governing Formula: E_ens = (1/k) * v + ((k - 1) / k) * c; Reduction = (v - E_ens) / v * 100

Step-by-Step Calculation:

Step 1: Compute first term (variance contribution): term1 = v / k = 0.20 / 5 = 0.040.

Step 2: Compute second term (covariance contribution): term2 = ((5 - 1) / 5) * 0.05 = (4 / 5) * 0.05 = 0.80 * 0.05 = 0.040.

Step 3: Calculate total ensemble expected error: E_ens = term1 + term2 = 0.040 + 0.040 = 0.080.

Step 4: Calculate percentage reduction: Reduction = ((0.20 - 0.080) / 0.20) * 100 = (0.120 / 0.20) * 100 = 60.0%.

[FINAL EVALUATED ANSWER: Ensemble Error = 0.080 (60% error reduction compared to individual model error 0.20)]

Pedagogical Note: Because the model errors were only weakly correlated (c = 0.05 vs v = 0.20), model averaging slashed the total error variance by 60%.

Problem 2 (Advanced (KTU Exam Level)): Ensemble Probability Voting vs Majority Hard Voting

Statement: Three classifiers (M1, M2, M3) output predicted probabilities for a binary positive class as P1 = 0.45, P2 = 0.48, and P3 = 0.95 for a true positive instance (y = 1). Using a classification threshold of 0.50: (a) Determine the individual binary classifications of M1, M2, and M3, (b) Determine the ensemble prediction under Majority Hard Voting, (c) Calculate the Soft Average ensemble probability P_soft and determine its prediction, and (d) Contrast the robustness of soft vs hard voting.

Given: P1 = 0.45, P2 = 0.48, P3 = 0.95. True class y = 1. Threshold = 0.50.

Governing Formula: Hard Vote: argmax(count(pred == 1)); Soft Vote: P_soft = (1/3) * (P1 + P2 + P3)

Step-by-Step Calculation:

Step 1: Individual binary decisions: M1: P1 = 0.45 < 0.50 -> Class 0 (Incorrect) M2: P2 = 0.48 < 0.50 -> Class 0 (Incorrect) M3: P3 = 0.95 >= 0.50 -> Class 1 (Correct).

Step 2: Majority Hard Voting: Votes for Class 0 = 2 (M1, M2). Votes for Class 1 = 1 (M3). Hard Voting Ensemble Prediction = Class 0 (WRONG!).

Step 3: Soft Probability Voting: P_soft = (0.45 + 0.48 + 0.95) / 3 = 1.88 / 3 = 0.6267. Since P_soft = 0.6267 >= 0.50, Soft Voting Ensemble Prediction = Class 1 (CORRECT!).

[FINAL EVALUATED ANSWER: Hard Vote = Class 0 (Fails); Soft Vote = Class 1 with 62.67% confidence (Succeeds)]

Pedagogical Note: Soft probability voting takes into account confidence calibration: M3 was extremely confident (95%) while M1 and M2 were barely uncertain (45% and 48%). Soft voting correctly allowed M3's strong signal to overturn the marginal majority.

Real-World Industry Case (Competitive Machine Learning & Financial Risk Modeling): Kaggle Grandmaster Competition Winning Pipeline

Pipeline Workflow: Tabular credit loan dataset -> Team trains 10 diverse models (LightGBM, XGBoost, CatBoost, 5-layer MLP, ResNet-Tabular) -> Out-of-fold probability predictions combined using weighted soft voting ensemble.

Industrial Impact & Benefit: Ensemble consistently outscores any individual model by 0.015-0.030 AUC, making the decisive difference between 1st place gold and 50th place.

Key Oral Viva Voce Checkpoints

Q1: What is the primary mathematical mechanism through which ensembling reduces test error?

Ans: Variance reduction: averaging uncorrelated error terms dampens overall prediction variance without increasing bias.

Q2: Why is ensembling particularly effective for deep neural networks?

Ans: Because deep networks have highly non-convex loss surfaces; different initializations and mini-batch sequences cause models to converge to distinct local minima that make different errors.

Q3: What is the difference between Bagging and Boosting?

Ans: Bagging trains independent models in parallel on bootstrap resamples to reduce variance; Boosting trains models sequentially, focusing each new model on earlier errors to reduce bias.

Q4: What is the key disadvantage of deploying an ensemble of 10 deep neural networks to production?

Ans: Inference latency and memory costs are multiplied by 10, which may be unacceptable for real-time edge applications.

Q5: What is a Snapshot Ensemble?

Ans: A technique using cyclical learning rate schedules to visit multiple local minima in a single training run, saving model weights at each cycle trough to form an ensemble for free.

LECTURE #16 · MODULE 2 (Topic 2.9)

Date: 03-Oct-2025
Period: Period 1

Title: Dropout and Parameter Initialization

CO: CO2 Bloom's: Understand / Apply

Prerequisites: Activation Functions, Backpropagation, Ensemble Theory, Day 15 concepts
Key Terminology: Dropout, Bernoulli Mask, Inverted Dropout, Co-Adaptation, Xavier/Glorot Initialization, He/Kaiming Initialization, Fan-In / Fan-Out, Variance Preservation

Dropout Formulation

Dropout (Srivastava et al., JMLR 2014) is a computationally efficient regularization technique that prevents feature co-adaptation. 1. Training Phase: For each training example and each training step, each hidden unit is randomly retained with probability p (or dropped with drop rate q = 1 - p): r^(l) ~ Bernoulli(p) h_tilde^(l) = r^(l) odot h^(l) 2. Inverted Dropout (Standard Modern Implementation): To ensure that the expected output during testing matches training without requiring post-hoc test-time scaling, activations are scaled by 1/p during training: h_tilde^(l) = (r^(l) odot h^(l)) / p 3. Test Phase: At test time, Dropout is deactivated completely: h^(l) is evaluated deterministically without scaling. Ensemble Interpretation: In a network with N units, Dropout samples from an ensemble of 2^N distinct 'thinned' sub-networks that all share the exact same parameter weights! At test time, using the full unmasked network computes the geometric mean of the sub-networks' predictive distributions.

Parameter Initialization

Why not initialize weights to zero? Symmetry problem: all neurons compute identical forward activations and receive identical backpropagated gradients. Why not large random weights? Activations explode or saturate (vanishing gradients in sigmoid/tanh). 1. Xavier / Glorot Initialization (Glorot & Bengio, 2010): Designed for Tanh and Sigmoid activations. Goal: maintain constant variance of activations during forward pass and gradients during backward pass: Var(W) = 2 / (fan_in + fan_out) Uniform distribution: W ~ U(- sqrt(6 / (fan_in + fan_out)), + sqrt(6 / (fan_in + fan_out))) Normal distribution: W ~ N(0, 2 / (fan_in + fan_out)) 2. He / Kaiming Initialization (He et al., ICCV 2015): Designed specifically for ReLU activations (which zero out half the distribution): Var(W) = 2 / fan_in Uniform distribution: W ~ U(- sqrt(6 / fan_in), + sqrt(6 / fan_in)) Normal distribution: W ~ N(0, sqrt(2 / fan_in))

Exam Important Points

- KTU Exam Q14(b): Effect of hidden unit capacity combined with dropout on bias and variance.
- Full algorithmic steps of Inverted Dropout vs Classical Dropout.
- Ensemble of 2^N sub-networks interpretation of Dropout.
- Formulas and derivations for Xavier/Glorot vs He/Kaiming weight initialization.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Inverted Dropout Expected Value Verification

Statement: A hidden layer produces activation vector h = [2.0, 4.0, 6.0, 8.0]^T. During training, Inverted Dropout with retention probability p = 0.5 is applied. The generated Bernoulli mask is r = [1, 0, 1, 0]^T. (a) Calculate the masked activations under classical dropout and the required test-time scaling, (b) Calculate the masked activations under Inverted Dropout, and (c) Show that the expected sum of activations is preserved under Inverted Dropout.

Given: h = [2.0, 4.0, 6.0, 8.0]^T, p = 0.5, r = [1, 0, 1, 0]^T.

Governing Formula: Classical Train: h_tilde = r * h; Classical Test: h_test = p * h; Inverted Train: h_tilde = (r * h) / p

Step-by-Step Calculation:

Step 1: Compute classical dropout activations: h_drop = [2.0*1, 4.0*0, 6.0*1, 8.0*0]^T = [2.0, 0.0, 6.0, 0.0]^T. At test time, all weights or activations must be multiplied by p = 0.5.

Step 2: Compute Inverted Dropout activations: h_inv = h_drop / p = [2.0/0.5, 0.0/0.5, 6.0/0.5, 0.0/0.5]^T = [4.0, 0.0, 12.0, 0.0]^T.

Step 3: Compare expected sums: Original sum = 2.0 + 4.0 + 6.0 + 8.0 = 20.0. Inverted Dropout sum for this mask = 4.0 + 0.0 + 12.0 + 0.0 = 16.0. Expected sum across all masks = E[r_i] / p * sum(h_i) = (0.5 / 0.5) * 20.0 = 20.0.

[FINAL EVALUATED ANSWER: Classical: [2.0, 0.0, 6.0, 0.0]^T (requires test scaling). Inverted: [4.0, 0.0, 12.0, 0.0]^T (no test scaling required).]

Pedagogical Note: Inverted dropout scales surviving activations by 1/p (here, doubles them to 4.0 and 12.0) during training, perfectly matching the expected activation magnitude at test time.

Problem 2 (Advanced (KTU Exam Level)): Xavier vs He Initialization Standard Deviation Calculation

Statement: A dense layer connects a preceding layer with fan_in = 512 neurons to a subsequent layer with fan_out = 128 neurons. (a) Calculate the standard deviation sigma_xavier for Xavier/Glorot normal initialization, (b) Calculate the uniform bounds [-a, +a] for Xavier uniform initialization, (c) Calculate the standard deviation sigma_he for He/Kaiming normal initialization, and (d) Explain why sigma_he is larger than sigma_xavier.

Given: fan_in = 512, fan_out = 128.

Governing Formula: sigma_xavier = sqrt(2 / (fan_in + fan_out)); a_xavier = sqrt(6 / (fan_in + fan_out)); sigma_he = sqrt(2 / fan_in)

Step-by-Step Calculation:

Step 1: Xavier Normal standard deviation: Var_xavier = 2.0 / (512 + 128) = 2.0 / 640 = 0.003125. sigma_xavier = sqrt(0.003125) = 0.05590.

Step 2: Xavier Uniform bounds: a_xavier = sqrt(6.0 / (512 + 128)) = sqrt(6.0 / 640) = sqrt(0.009375) = 0.09682. Bounds: [-0.09682, +0.09682].

Step 3: He Normal standard deviation: Var_he = 2.0 / fan_in = 2.0 / 512 = 0.0039062S. sigma_he = sqrt(0.0039062S) = 0.06250.

Step 4: Comparison explanation: sigma_he (0.0625) > sigma_xavier (0.0559) because ReLU sets approximately half of all activations to zero (mean activation halved). To compensate for the 50% lost energy and maintain constant activation variance, He initialization injects a factor of 2 in the numerator.

[FINAL EVALUATED ANSWER: sigma_xavier = 0.0559, a_xavier = 0.0968, sigma_he = 0.0625]

Pedagogical Note: He initialization requires higher variance (0.0625 vs 0.0559) to prevent signal attenuation caused by ReLU clipping negative activations.

Real-World Industry Case (Computational Biology & Structural Bioinformatics): Protein Folding & Biomolecular Structure Prediction

Pipeline Workflow: Deep Transformer-based architecture processes amino acid sequences -> Severe risk of memorizing known crystallized proteins -> Monte Carlo Dropout (Gal & Ghahramani, 2016) enabled at inference time to generate epistemic uncertainty confidence intervals.

Industrial Impact & Benefit: Enables drug discovery scientists to flag uncertain protein loop predictions before committing to wet-lab synthesized assays.

Key Oral Viva Voce Checkpoints

Q1: What is the primary motivation behind the Dropout technique?

Ans: To prevent complex co-adaptation of hidden units, forcing each neuron to learn robust, generalizable features that do not rely on the presence of specific other neurons.

Q2: What is Inverted Dropout and why is it preferred over classical Dropout?

Ans: Inverted Dropout scales active neurons by 1/p during training, ensuring that test-time execution requires no scaling or architectural modification.

Q3: How does Dropout approximate an ensemble of models?

Ans: A network with N units samples from an ensemble of 2^N distinct thinned sub-networks that all share parameters, combining their predictions at test time.

Q4: What is the recommended retention probability p for hidden units vs input units?

Ans: Typically p = 0.5 for hidden layers, and p = 0.8 to 0.9 (or no dropout) for input layers.

Q5: Why is Xavier initialization unsuitable for deep ReLU networks?

Ans: Because ReLU sets all negative pre-activations to zero, halving the variance at each layer; Xavier initialization fails to account for this 50% loss, causing activation variance to decay exponentially to zero in deep networks.

LECTURE #17 · MODULE 3 (Topic 3.1)

Date: 06-Oct-2025
Period: Period 2

Title: Convolutional Neural Networks: The Convolution Operation

CO: CO3 Bloom's: Understand / Apply

Prerequisites: Matrix algebra, 2D grids, Multi-Layer Perceptrons, Day 8 concepts
Key Terminology: Convolution, Cross-Correlation, Filter Kernel, Feature Map, Kernel Flipping, Receptive Field, Discrete 2D Convolution

Convolution Operation

Continuous 1D Convolution: s(t) = (x * w)(t) = integral_{-infinity}^{infinity} x(a) * w(t - a) da Discrete 2D Mathematical Convolution (with Kernel Flip): S(i, j) = (I * K)(i, j) = sum_m sum_n I(i - m, j - n) * K(m, n) Discrete 2D Cross-Correlation (Deep Learning Convention): S(i, j) = (I * K)(i, j) = sum_m sum_n I(i + m, j + n) * K(m, n) In deep learning libraries (PyTorch, TensorFlow), cross-correlation is implemented without flipping the kernel because kernel weights are learned directly from data via backpropagation, making learning flipped weights mathematically equivalent to learning unflipped weights.

Feature Map And Filters

A filter kernel is a small matrix (e.g. 3x3, 5x5) whose elements act as feature detectors: - Horizontal Sobel Filter: K = [[-1, -2, -1], [0, 0, 0], [1, 2, 1]] - Vertical Sobel Filter: K = [[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]] Sliding this across an image generates a 2D scalar activation map highlighting local directional features.

Exam Important Points

- Continuous vs discrete 2D convolution formulas.
- Why deep learning libraries implement cross-correlation rather than true flipped convolution.
- Manual calculation of 2D convolution on a 4x4 matrix with a 3x3 kernel.
- Hand-crafted kernels: Sobel horizontal and vertical edge detectors.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Manual 2D Discrete Convolution Calculation

Statement: Given a 4x4 input image I = [[1, 2, 0, 1], [0, 1, 3, 2], [2, 0, 1, 0], [1, 1, 0, 2]] and a 3x3 kernel K = [[1, 0, -1], [1, 0, -1], [1, 0, -1]]. Calculate the valid 2D cross-correlation output feature map S.

Given: Input 4x4, Kernel 3x3, stride=1, no padding.

Governing Formula: Output shape = (4 - 3 + 1) x (4 - 3 + 1) = 2x2. S(i, j) = sum(I_patch * K)

Step-by-Step Calculation:

Step 1: Output dimension is 2x2: S(0,0), S(0,1), S(1,0), S(1,1).
Step 2: S(0,0) = (1*1 + 2*0 + 0*-1) + (0*1 + 1*0 + 3*-1) + (2*1 + 0*0 + 1*-1) = 1 - 3 + 1 = -1.
Step 3: S(0,1) = (2*1 + 0*0 + 1*-1) + (1*1 + 3*0 + 2*-1) + (0*1 + 1*0 + 0*-1) = 1 - 1 + 0 = 0.
Step 4: S(1,0) = (0*1 + 1*0 + 3*-1) + (2*1 + 0*0 + 1*-1) + (1*1 + 1*0 + 0*-1) = -3 + 1 + 1 = -1.
Step 5: S(1,1) = (1*1 + 3*0 + 2*-1) + (0*1 + 1*0 + 0*-1) + (1*1 + 0*0 + 2*-1) = -1 + 0 - 1 = -2.

[FINAL EVALUATED ANSWER: Feature Map S = [[-1, 0], [-1, -2]]]

Pedagogical Note: The 3x3 vertical difference kernel extracts spatial vertical edge gradients across the 4x4 grid.

Problem 2 (Advanced (KTU Exam Level)): Mathematical Equivalence of Cross-Correlation and Convolution

Statement: Let continuous 1D convolution be (f * g)(t) = integral f(tau) * g(t - tau) d tau and cross-correlation be (f star g)(t) = integral f(tau) * g(t + tau) d tau. Prove that (f star g)(t) = (f * g_minus)(t) where g_minus(t) = g(-t), and explain why deep learning frameworks can optimize cross-correlation filters directly without explicit flipping.

Given: Definitions of convolution, cross-correlation, and flipped kernel g_minus(t) = g(-t).
Governing Formula: (f * g_minus)(t) = integral f(tau) * g_minus(t - tau) d tau

Step-by-Step Calculation:

Step 1: Write convolution: (f * g_minus)(t) = integral f(tau) * g_minus(t - tau) d tau.

Step 2: Substitute g_minus(u) = g(-u): g_minus(t - tau) = g(-(t - tau)) = g(tau - t).

Step 3: In 2D arrays: K_flipped[i, j] = K[-i, -j]. Thus cross-correlation with K equals convolution with K_flipped.

Step 4: In deep learning, weights in K are free parameters learned end-to-end to minimize loss; the optimizer learns the optimal orientation directly, eliminating the need to reverse arrays at runtime.

[FINAL EVALUATED ANSWER: Cross-correlation with kernel K is mathematically identical to true convolution with flipped kernel K_minus.]

Pedagogical Note: End-to-end learning renders the mathematical difference between convolution and cross-correlation a mere indexing convention.

Real-World Industry Case (Semiconductor Manufacturing): Silicon Wafer Defect Inspection

Pipeline Workflow: High-res microscope scans wafer -> 2D convolution applies learned micro-crack filters -> Edge-enhanced feature maps identify hairline sub-micron etch defects.

Industrial Impact & Benefit: Prevents defective silicon chips from reaching automotive and aerospace electronic engine control units.

Key Oral Viva Voce Checkpoints

Q1: What is the difference between mathematical convolution and cross-correlation?

Ans: Mathematical convolution flips the kernel by 180 degrees before computing the inner product; cross-correlation slides the kernel without flipping.

Q2: Why do deep learning frameworks implement cross-correlation while calling it convolution?

Ans: Because kernel weights are learned from data; learning flipped weights is mathematically equivalent to learning unflipped weights, so flipping is an unnecessary computational overhead.

Q3: What is a feature map?

Ans: The 2D output array produced by sliding a convolutional filter across an input image and computing local inner products.

Q4: What determines output spatial dimensions of a 2D convolution without padding?

Ans: Output Height = H_in - K_h + 1, Output Width = W_in - K_w + 1.

Q5: What does a 3x3 Sobel kernel detect?

Ans: Spatial image intensity gradients (directional edges) along horizontal or vertical orientations.

LECTURE #18 · MODULE 3 (Topic 3.2)

Date: 08-Oct-2025
Period: Period 4

Title: Motivation for CNNs and Pooling Operations

CO: CO3 Bloom's: Understand / Apply

Prerequisites: Convolution Operation, Multi-Layer Perceptrons, Day 17 concepts
Key Terminology: Sparse Interactions, Parameter Sharing, Equivariance to Translation, Max Pooling, Average Pooling, Translation Invariance, Downsampling

Three Motivations For Cnns

1. Sparse Interactions: Kernel size k << input size m. Receptive fields are local, reducing storage from O(m*n) to O(k*n) and slashing FLOPs. 2. Parameter Sharing: Reusing kernel weights across all positions slashes parameter counts and prevents overfitting. 3. Equivariant Representations: f(g(x)) = g(f(x)). Shifting the input image translates feature activations identically.

Pooling Operations

Pooling summarizes nearby outputs: - Max Pooling: S(i, j) = max_{(p,q) in R} I(i+p, j+q) (captures dominant activations). - Average Pooling: S(i, j) = (1/|R|) sum_{(p,q) in R} I(i+p, j+q) (smooth background summary). Pooling provides approximate invariance to small translations of the input.

Exam Important Points

- Sparse Interactions, Parameter Sharing, and Translation Equivariance.
- Translation Equivariance (convolution) vs Translation Invariance (pooling).
- Max Pooling vs Average Pooling manual calculation on a 4x4 matrix.
- How pooling reduces spatial dimensions and controls overfitting.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Max and Average Pooling Calculation

Statement: Given 4x4 matrix M = [[12, 20, 30, 0], [8, 16, 2, 14], [4, 6, 8, 10], [18, 22, 12, 14]], perform 2x2 pooling with stride 2: (a) Compute Max Pooling, and (b) Compute Average Pooling.

Given: 4x4 matrix, pool_size=2, stride=2.
Governing Formula: Max = max(patch); Avg = mean(patch)

Step-by-Step Calculation:

Q1 [[12, 20], [8, 16]]: Max = 20, Avg = 56/4 = 14.0.
Q2 [[30, 0], [2, 14]]: Max = 30, Avg = 46/4 = 11.5.
Q3 [[4, 6], [18, 22]]: Max = 22, Avg = 50/4 = 12.5.
Q4 [[8, 10], [12, 14]]: Max = 14, Avg = 44/4 = 11.0.

[FINAL EVALUATED ANSWER: Max Pool = [[20, 30], [22, 14]], Avg Pool = [[14.0, 11.5], [12.5, 11.0]]]
Pedagogical Note: Max pooling captures peak signals while average pooling extracts mean background response.

Problem 2 (Advanced (KTU Exam Level)): Sparse Interactions vs Dense MLP Parameter Reduction

Statement: An image with resolution 1000x1000 is processed by a hidden layer of 1000x1000 units. (a) Calculate weights in fully connected MLP, (b) Calculate weights in CNN with a 5x5 filter and 1 feature map, (c) Compute parameter reduction ratio, and (d) Calculate parameters with 16 filters.

Given: Input 10^6, Hidden 10^6, Kernel 5x5.

Governing Formula: MLP = m * n; CNN = k_h * k_w * filters

Step-by-Step Calculation:

Step 1: MLP weights = 10^6 * 10^6 = 10^12 = 1 Trillion parameters.
Step 2: CNN weights (1 filter) = 5 * 5 = 25 parameters.
Step 3: Reduction ratio = 10^12 / 25 = 40,000,000,000x.
Step 4: CNN weights (16 filters) = 16 * 25 = 400 parameters.

[FINAL EVALUATED ANSWER: MLP = 1 Trillion weights. CNN (1 filter) = 25 weights (40B x reduction). CNN (16 filters) = 400 weights.]

Pedagogical Note: CNNs make high-resolution vision computationally tractable through parameter sharing.

Real-World Industry Case (Biometrics & Airport Security): Biometric Facial Recognition Gates

Pipeline Workflow: Passenger walks past camera -> Head position varies by centimeters -> Max pooling provides translation invariance -> Produces stable face embeddings.

Industrial Impact & Benefit: Enables seamless passenger boarding without requiring rigid motionless posture alignment.

Key Oral Viva Voce Checkpoints

Q1: What are the three core motivations of CNNs?
Ans: Sparse Interactions, Parameter Sharing, and Equivariant Representations.

Q2: What is translation equivariance?
Ans: Shifting input causes output feature map to shift by the identical amount: f(g(x)) = g(f(x)).

Q3: Does max pooling contain trainable weights?
Ans: No, it is a fixed deterministic non-linear mathematical operation with zero learnable parameters.

Q4: What is Global Average Pooling?
Ans: Averaging all spatial values across an entire 2D channel to produce a single scalar per feature map.

Q5: How does pooling reduce compute in deeper layers?
Ans: Downsampling height and width by 2x reduces spatial locations for subsequent layers by 75%.

LECTURE #19 · MODULE 3 (Topic 3.3)

Date: 10-Oct-2025
Period: Period 1

Title: Convolution and Pooling as an Infinitely Strong Prior

CO: CO3 Bloom's: Understand / Analyze

Prerequisites: Bayesian Probability (priors, posteriors), Convolution and Pooling mechanics, Day 18 concepts
Key Terminology: Bayesian Prior, Infinitely Strong Prior, Inductive Bias, Parameter Constraint, Permutation Vulnerability, Spatial Locality Assumption

Bayesian Prior Perspective

In Bayesian statistics, a prior probability distribution p(theta) reflects our beliefs about which parameter configurations are plausible before observing any training data. A prior is 'weak' or 'soft' if it assigns positive probability to all parameter values, merely preferring some over others (e.g. Gaussian prior in L2 regularization with high variance). A prior is 'infinitely strong' if it assigns zero probability to certain parameter configurations, making it mathematically impossible for the learning algorithm to ever choose those weights regardless of how much training data is collected.

Convolution And Pooling As Infinitely Strong Prior

A Convolutional layer can be viewed as a fully connected layer with an infinitely strong prior specifying that: 1. All weights connecting non-spatially adjacent pixels must be identically ZERO: p(W_ij != 0) = 0 for |i - j| > kernel_radius. 2. Weights for spatially shifted connections must be EXACTLY IDENTICAL (parameter sharing): p(W_ij != W_{i+1, j+1}) = 0. Similarly, a Pooling layer is an infinitely strong prior specifying that each unit must be invariant to small local translations. Benefits and Hazards: - Benefit: If the prior assumption is TRUE (e.g. in natural images where pixels have local stationary correlations), the CNN learns with massive sample efficiency, avoiding overfitting. - Hazard: If the prior assumption is FALSE (e.g. tabular data where column ordering is arbitrary, or permuted images), an infinitely strong prior prevents the network from learning the true relationship, leading to high underfitting bias.

Permutation Experiment

If we randomly permute all pixels of MNIST images with a fixed permutation matrix P: - A fully connected MLP achieves the EXACT same accuracy on permuted MNIST as on unpermuted MNIST (MLPs have a weak, isotropic prior). - A CNN's performance collapses dramatically on permuted MNIST because the infinitely strong spatial locality prior is violently violated!

Exam Important Points

- Definition of weak vs infinitely strong Bayesian priors.
- How convolution enforces an infinitely strong prior: zero weights outside kernel + identical shifted weights.
- Explanation of the Permuted MNIST experiment comparing MLPs and CNNs.
- When infinitely strong priors lead to catastrophic underfitting.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Weight Matrix Density under Infinitely Strong Prior

Statement: Consider a 1D input of length N = 100 mapped to an output of length N = 100. (a) In a dense MLP, calculate the total number of weight connections in the 100x100 matrix, (b) In a convolutional layer with kernel size k = 5 and circular padding, calculate the number of non-zero entries in the matrix, and (c) Determine the percentage of weights forced to be ZERO by the infinitely strong spatial locality prior.

Given: N = 100, k = 5.

Governing Formula: MLP entries = N^2; Conv non-zero entries = N * k; Sparsity % = (1 - Non2ero / Total) * 100

Step-by-Step Calculation:

Step 1: Dense MLP total weight matrix entries = 100 * 100 = 10,000 entries.

Step 2: Convolutional kernel k = 5 means each row has exactly 5 non-zero entries. Total non-zero entries across 100 rows = 100 * 5 = 500 entries.

Step 3: Number of zero-constrained entries = 10,000 - 500 = 9,500 entries.

Step 4: Percentage of weights forced to zero = (9,500 / 10,000) * 100 = 95.0%.

[FINAL EVALUATED ANSWER: Non-zero entries = 500; Zero-constrained entries = 9,500 (95.0% forced to zero).]

Pedagogical Note: The infinitely strong prior zeroes out 95% of all conceivable linear connections and constrains the remaining 500 entries to share only 5 distinct parameter values.

Problem 2 (Advanced (KTU Exam Level)): Bayesian Gaussian Prior vs Infinitely Strong Prior Variance Limit

Statement: In Bayesian linear regression, weights have a zero-mean Gaussian prior p(w) = N(0, sigma_prior^2 * I). (a) Write the MAP regularized objective function, (b) What happens to the prior as sigma_prior^2 -> infinity? (c) What happens to the prior as sigma_prior^2 -> 0? and (d) Explain how an infinitely strong prior relates to setting prior variance to zero for non-local weights.

Given: p(w) = N(0, sigma_prior^2 * I), Loss = L_data - ln p(w).

Governing Formula: J_MAP(w) = J_data(w) + (1 / (2 * sigma_prior^2)) * ||w||_2^2

Step-by-Step Calculation:

Step 1: Write negative log posterior: J_MAP(w) = J_data(w) + (1 / (2 * sigma_prior^2)) * sum_i w_i^2 + const. This is exact L2 regularization with penalty alpha = 1 / sigma_prior^2.

Step 2: Limit as sigma_prior^2 -> infinity: alpha -> 0. The prior becomes an improper uniform distribution, exerting zero constraint on weights (unregularized maximum likelihood).

Step 3: Limit as sigma_prior^2 -> 0: alpha -> infinity. The penalty for any non-zero weight w_i != 0 approaches infinity, forcing w_i to be identically zero with probability 1.

Step 4: Connection to CNNs: A CNN sets sigma_prior^2 = 0 for all weights connecting pixels outside the local kernel window, creating a Dirac delta prior delta(w_ij) that permanently forbids non-local interactions.

[FINAL EVALUATED ANSWER: sigma_prior^2 -> 0 yields an infinitely strong prior forcing non-local weights to zero.]

Pedagogical Note: Architectural constraints in CNNs are equivalent to infinitely strong Bayesian priors with zero variance on non-local weights.

Real-World Industry Case (Bioinformatics & Computational Genomics): Genomic Variant Pathogenicity Modeling

Pipeline Workflow: DNA sequences have local spatial motif structure (transcription factor binding sites) -> CNNs with local 1D convolution priors detect regulatory motifs -> Tabular patient clinical data (age, lab blood values) lacks spatial ordering -> Processed via gradient boosted trees or MLPs to avoid harmful spatial priors.

Industrial Impact & Benefit: Applying the correct inductive bias prevents misinterpreting unstructured patient records while capturing sequential genomic motifs.

Key Oral Viva Voce Checkpoints

Q1: What is an infinitely strong prior in Bayesian terms?

Ans: A prior distribution that assigns zero probability to certain parameter values, completely forbidding the model from adopting those parameters regardless of the training data.

Q2: How does a convolutional layer represent an infinitely strong prior?

Ans: By forcing all weights outside the local kernel window to be identically zero and forcing weights at different spatial positions to be identical (parameter sharing).

Q3: What happens if you train a CNN on images with randomly permuted pixels?

Ans: Performance degrades severely because the spatial locality and translation invariance priors are violated.

Q4: Why does an MLP perform identically on original and permuted MNIST?

Ans: Because an MLP is permutation invariant with respect to its input features; it possesses no spatial locality prior.

Q5: When is an infinitely strong prior beneficial?

Ans: When the underlying physical process satisfies the prior assumptions (e.g. 2D natural images possessing translation invariance and local correlations).

LECTURE #20 · MODULE 3 (Topic 3.4)

Date: 13-Oct-2025
Period: Period 2

Title: Variants of the Convolution Function

CO: CO3 Bloom's: Understand / Apply

Prerequisites: Convolution Operation, Day 17 concepts
Key Terminology: Strided Convolution, Valid Padding, Same Padding, Full Padding, Locally Connected Layers, Tiled Convolution, 1x1 Convolution, Dilated (Atrous) Convolution

Strided Convolution

*Standard convolution slides the kernel by 1 pixel at each step. A strided convolution slides the kernel by s pixels (s > 1): S(i, j) = sum_m sum_n I(i*s + m, j*s + n) * K(m, n) Effect: Downsamples the feature map by a factor of s without requiring a separate pooling layer. Modern architectures (ResNet, MobileNet) frequently replace max pooling with strided convolutions.*

Zero Padding Types

*For input dimension W, kernel size K, stride s, and zero-padding p: Output Dimension: W_out = floor((W - K + 2*p) / s) + 1 1. Valid Padding (p = 0): No padding added. W_out = (W - K + 1). Feature maps shrink at every layer. 2. Same Padding (p = (K - 1) / 2 for odd K, s = 1): Zeros added such that output spatial dimensions exactly match input dimensions: W_out = W. 3. Full Padding (p = K - 1, s = 1): Zeros added such that every pixel is visited K times: W_out = W + K - 1.*

Specialized Convolution Variants

*1. 1x1 Convolution (Lin et al., Network in Network, 2013): Acts as a cross-channel linear combination of features without altering spatial resolution. Crucial for channel dimensionality reduction (e.g. Inception bottlenecks, ResNet residual blocks). 2. Dilated (Atrous) Convolution (Yu & Koltun, 2015): Inserts d - 1 zeros (holes) between kernel elements. Effective kernel size: K_eff = K + (K - 1) * (d - 1). Exponentially expands receptive field without increasing parameter count or losing spatial resolution (widely used in WaveNet and DeepLab semantic segmentation). 3. Locally Connected Layers (Unshared Convolutions): Local receptive fields without parameter sharing. Used when features are not stationary across the grid (e.g. human face alignment in DeepFace where eyes are always near the top).*

Exam Important Points

- *Master formula for output spatial dimension: floor((W - K + 2p)/s) + 1.*
- *Definitions and diagrams for Valid, Same, and Full padding.*
- *Purpose and parameter reduction of 1x1 convolutions in Inception/ResNet.*
- *Dilated convolution receptive field formula and application in semantic segmentation.*

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Convolution Output Shape Calculation across Padding Modes

Statement: An input feature map has spatial resolution W_in = 32x32. A convolutional layer uses filter size K = 5x5 with stride s = 1. Calculate the output spatial dimensions for: (a) Valid padding (p = 0), (b) Same padding, and (c) Full padding.

Given: W_in = 32, K = 5, s = 1.

Governing Formula: W_out = floor((W_in - K + 2*p)/s) + 1; Same: p = (K - 1)/2; Full: p = K - 1

Step-by-Step Calculation:

Step 1: Valid Padding (p = 0): W_out = (32 - 5 + 2*0)/1 + 1 = 27 + 1 = 28x28.

Step 2: Same Padding: p = (5 - 1) / 2 = 2 pixels on each side. W_out = (32 - 5 + 2*2)/1 + 1 = (32 - 5 + 4) + 1 = 31 + 1 = 32x32.

Step 3: Full Padding: p = K - 1 = 5 - 1 = 4 pixels on each side. W_out = (32 - 5 + 2*4)/1 + 1 = (32 - 5 + 8) + 1 = 35 + 1 = 36x36.

[FINAL EVALUATED ANSWER: Valid = 28x28, Same = 32x32, Full = 36x36]

Pedagogical Note: Valid padding shrinks the map by 4 pixels, Same padding preserves the 32x32 resolution, and Full padding expands it to 36x36.

Problem 2 (Advanced (KTU Exam Level)): KTU Model Paper Q9: Stride Effect & 1x1 Bottleneck Parameter Savings

Statement: Part 1: What happens if the stride of a convolutional layer increases? Part 2: A convolutional layer maps 256 input channels to 256 output channels using 3x3 filters. An alternative Bottleneck design uses: (i) a 1x1 convolution reducing 256 channels to 64, (ii) a 3x3 convolution with 64 channels, and (iii) a 1x1 convolution expanding 64 channels back to 256. Calculate the total weight parameters for both designs and the percentage reduction.

Given: Direct: 256 -> 256 with 3x3. Bottleneck: 256 -> 64 (1x1), 64 -> 64 (3x3), 64 -> 256 (1x1).

Governing Formula: Params = C_in * C_out * K_h * K_w

Step-by-Step Calculation:

Part 1: Increasing stride s reduces output spatial resolution by approximately 1/s, reducing memory and computation for subsequent layers while increasing the effective receptive field stride.

Part 2 (Direct Design): Weights = 256 * 256 * (3 * 3) = 65,536 * 9 = 589,824 parameters.

Part 3 (Bottleneck Design): Layer 1 (1x1): 256 * 64 * (1 * 1) = 16,384. Layer 2 (3x3): 64 * 64 * (3 * 3) = 4,096 * 9 = 36,864. Layer 3 (1x1): 64 * 256 * (1 * 1) = 16,384. Total Bottleneck Parameters = 16,384 + 36,864 + 16,384 = 69,632 parameters.

Part 4: Compute savings: Reduction = (589,824 - 69,632) / 589,824 * 100 = (520,192 / 589,824) * 100 = 88.19%.

[FINAL EVALUATED ANSWER: Direct = 589,824 parameters. Bottleneck = 69,632 parameters (88.19% parameter reduction).]

Pedagogical Note: Using 1x1 bottleneck convolutions cuts parameters by over 88% while preserving representational capacity, which is the cornerstone of modern ResNet and Inception architectures.

Real-World Industry Case (Autonomous Vehicles & Scene Parsing): DeepLab Autonomous Driving Semantic Segmentation

Pipeline Workflow: High-res 1080p vehicle camera feed -> Standard pooling loses fine boundary details of distant pedestrians -> Dilated (Atrous) Convolutions expand receptive fields to cover entire intersections without downsampling.

Industrial Impact & Benefit: Enables pixel-accurate sidewalk, curb, and pedestrian segmentation without spatial detail loss.

Key Oral Viva Voce Checkpoints

Q1: What is the mathematical formula for the output dimension of a 2D convolution?

*Ans: Output = floor((Input - Kernel + 2*Padding) / Stride) + 1.*

Q2: What is the difference between Valid padding and Same padding?

Ans: Valid padding adds zero zeros (output shrinks by K - 1); Same padding adds enough zeros such that the output spatial resolution matches the input resolution when stride=1.

Q3: What is a 1x1 convolution and what is its primary use?

Ans: A convolution with a 1x1 kernel that computes a linear combination across input channels at each pixel, used for channel dimensionality reduction (bottlenecks).

Q4: What is a Dilated (Atrous) Convolution?

Ans: A convolution where the filter kernel is expanded with spaces/zeros between weights, enlarging the receptive field without adding parameters.

Q5: What are Locally Connected Layers?

Ans: Layers with local receptive fields like convolution, but without parameter sharing; each spatial location learns its own unique set of weights.

LECTURE #21 · MODULE 3 (Topic 3.5)

Date: 15-Oct-2025
Period: Period 4

Title: Structured Outputs and Diverse Data Types in CNNs

CO: CO3 Bloom's: Understand / Apply

Prerequisites: Convolution Operations, Multi-channel Tensors, Day 20 concepts
Key Terminology: Structured Outputs, Multi-Channel Tensors, 1D Convolution, 3D Volumetric Convolution, Semantic Segmentation, Object Detection Bounding Box, Spatiotemporal Feature Learning

Data Type Taxonomy

CNNs process multi-dimensional grid data across diverse domains: 1. 1D Sequential Grids (Channels x Time): - Audio waveforms (1D amplitude sampled at 16 kHz or 44.1 kHz). - Biomedical ECG / EEG sensor telemetry. - Text sentences (sequence of word embedding vectors). 2. 2D Spatial Grids (Height x Width x Channels): - Standard RGB color photographs (H x W x 3). - Audio spectrograms (Time x Frequency x 1). - Multispectral satellite imagery (e.g. 13 spectral bands in Sentinel-2). 3. 3D Volumetric & Spatiotemporal Grids (Height x Width x Depth/Time x Channels): - Medical imaging: 3D MRI and CT volumetric anatomical voxel scans. - Video clips: 2D spatial frames concatenated over time (H x W x T x 3). - Computational fluid dynamics and weather simulation grids.

Structured Outputs

Rather than outputting a single categorical scalar y in {1, ..., K}, structured prediction tasks output high-dimensional structured arrays: 1. Semantic Segmentation (Dense Pixel Classification): - Output is a 2D label map S of shape (H x W x C) where every individual pixel is assigned a class probability distribution. - Architecture: Fully Convolutional Networks (FCN) and U-Net using encoder-decoder paths and transposed convolutions (skip connections). 2. Object Detection Bounding Boxes: - Output combines classification probabilities with regression coordinates [x_center, y_center, width, height] for multiple bounding boxes (e.g. YOLO, Faster R-CNN).

Exam Important Points

- *Dimensional structure of 1D, 2D, and 3D convolutional tensors.*
- *How 3D convolutions extract spatiotemporal features from video clips.*
- *Difference between image classification and dense semantic segmentation.*
- *U-Net architecture with skip connections for medical image segmentation.*

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): 3D Volumetric Convolution Output Dimension and Parameter Count

Statement: A medical MRI brain scan has volumetric resolution 128x128x64 voxels (Height x Width x Depth) with 1 input channel. A 3D convolutional layer applies 32 filters of size 3x3x3 with stride s = 1 and no padding (valid). (a) Calculate the output volumetric dimensions, and (b) Calculate the total number of trainable weight and bias parameters in the layer.

Given: Input: 128x128x64, C_in = 1, Filters = 32, K = 3x3x3, s = 1, p = 0.

Governing Formula: Out = floor((Dim - K + 2*p)/s) + 1; Weights = N_filters * C_in * (K_h * K_w * K_d); Biases = N_filters

Step-by-Step Calculation:

Step 1: Compute output dimensions: H_out = 128 - 3 + 1 = 126. W_out = 128 - 3 + 1 = 126. D_out = 64 - 3 + 1 = 62. Output volume = 126 x 126 x 62 with 32 channels.

Step 2: Compute weights: Weights = 32 * 1 * (3 * 3 * 3) = 32 * 27 = 864 weights.

Step 3: Compute biases = 32. Total parameters = 864 + 32 = 896 parameters.

[FINAL EVALUATED ANSWER: Output dimensions = 126 x 126 x 62 (32 channels); Total parameters = 896]

Pedagogical Note: 3D convolutions slide across height, width, and depth simultaneously, capturing volumetric anatomical spatial relationships.

Problem 2 (Advanced (KTU Exam Level)): Memory Footprint in Dense Semantic Segmentation vs Image Classification

Statement: An image with resolution 512x512x3 is processed for: (Case A) 1000-class Image Classification using Global Average Pooling, and (Case B) 20-class Semantic Segmentation producing a full-resolution dense prediction map. Both outputs use 32-bit floats. Calculate: (a) Output tensor dimensions for Case A and Case B, (b) Output memory consumption for a single image in bytes and kilobytes, and (c) Discuss the gradient backpropagation memory challenge in dense segmentation.

Given: Image: 512x512x3. Case A: 1000 classes. Case B: 512x512x20 classes. Float32 (4 bytes).

Governing Formula: Memory = Total_Elements * 4 bytes

Step-by-Step Calculation:

Step 1: Case A (Classification): Output shape = 1 x 1000. Elements = 1000. Memory = 1000 * 4 bytes = 4,000 bytes = 3.91 KB.

Step 2: Case B (Semantic Segmentation): Output shape = 512 x 512 x 20. Elements = 512 * 512 * 20 = 262,144 * 20 = 5,242,880 elements. Memory = 5,242,880 * 4 bytes = 20,971,520 bytes = 20.0 MB!

Step 3: Comparison ratio: Ratio = 20,971,520 / 4,000 = 5,242.88x more output memory.

Step 4: Backpropagation implications: Storing full-resolution intermediate feature activations for a batch of 16 segmentation maps requires gigabytes of GPU VRAM, requiring gradient checkpointing or smaller mini-batch sizes.

[FINAL EVALUATED ANSWER: Classification = 3.91 KB. Segmentation = 20.0 MB (5,243x larger memory footprint).]

Pedagogical Note: Dense structured prediction tasks require immense GPU VRAM to cache high-resolution multi-channel tensors during the backward pass.

Real-World Industry Case (Medical Radiology & Neurosurgical Navigation): Brain Glioblastoma Tumor Segmentation in 3D MRI

Pipeline Workflow: Patient multi-parametric 3D MRI scans (T1, T1-contrast, T2, FLAIR) -> 3D U-Net processes 4-channel volumetric voxels -> Produces voxel-wise segmented 3D tumor boundary mesh.

Industrial Impact & Benefit: Enables neurosurgeons to achieve millimeter-accurate tumor resection while avoiding critical motor cortex brain tissue.

Key Oral Viva Voce Checkpoints

Q1: What is the difference between image classification and semantic segmentation?

Ans: Classification assigns a single label to the entire image; semantic segmentation assigns a categorical class label to every individual pixel in the image.

Q2: What is a 1D convolution and where is it applied?

Ans: A convolution where the kernel slides across a single spatial or temporal dimension, used in audio waveforms, text sequences, and ECG signals.

Q3: What is a 3D convolution and where is it applied?

Ans: A convolution where a 3D kernel slides across height, width, and depth/time, used in 3D MRI/CT scans and video action recognition.

Q4: What is a U-Net architecture?

Ans: An encoder-decoder architecture with skip connections between corresponding contracting and expanding levels, widely used for biomedical image segmentation.

Q5: Why are skip connections critical in semantic segmentation architectures?

Ans: They bypass deep bottleneck downsampling, transferring high-resolution spatial boundary details directly to the decoder for precise pixel localization.

LECTURE #22 · MODULE 3 (Topic 3.6)

Date: 17-Oct-2025
Period: Period 1

Title: Efficient Convolution Algorithms

CO: CO3 Bloom's: Understand / Apply

Prerequisites: Convolution Operations, Matrix Multiplication, Complexity Theory, Day 21 concepts
Key Terminology: Efficient Convolutions, Depthwise Separable Convolution, Pointwise Convolution, Fast Fourier Transform (FFT), Convolution Theorem, Winograd Algorithm, MobileNet

Computational Bottleneck

Standard 2D Convolution on an input of size D_F x D_F with M channels, producing N channels with kernel size D_K x D_K requires: Total Multiplications = D_F * D_F * M * N * D_K * D_K For a 56x56 feature map with M=256, N=512, and 3x3 kernel: 3.71 BILLION multiplications per forward pass! This is completely prohibitive on mobile edge hardware.

Depthwise Separable Convolutions

MobileNet (Howard et al., 2017) factorizes standard convolution into two independent steps: 1. Depthwise Convolution: Applies a single spatial filter per input channel (M filters of size D_K x D_K x 1). Cost = D_F * D_F * M * D_K * D_K 2. Pointwise Convolution: A 1x1 convolution combining channels linearly across depth (N filters of size 1 x 1 x M). Cost = D_F * D_F * M * N Total Cost = D_F * D_F * M * (D_K^2 + N) Computation Reduction Ratio: Ratio = [D_F^2 * M * (D_K^2 + N)] / [D_F^2 * M * N * D_K^2] = (D_K^2 + N) / (N * D_K^2) = (1 / N) + (1 / D_K^2) For standard 3x3 filters (D_K = 3) and large N (e.g. N = 512): Ratio approx 1 / 3^2 = 1 / 9 approx 0.111 (8 to 9 times faster, an 88-89% computational reduction with negligible accuracy loss!).

Fft And Winograd Algorithms

1. Fast Fourier Transform (FFT) Convolution: By the Convolution Theorem: Convolution in the spatial domain equals point-wise multiplication in the frequency domain: I * K = IFFT(FFT(I) odot FFT(K)) Complexity drops from O(N_pixels * K^2) to O(N_pixels * log N_pixels). Highly efficient for large kernels (K >= 7x7). 2. Winograd Minimal Filtering Algorithm (F(2x2, 3x3)): Lavin & Gray (2016) showed that computing 2x2 outputs with a 3x3 kernel requires only 16 multiplications instead of standard 36 multiplications: a 2.25x speedup! Modern GPU libraries (cuDNN) automatically dispatch Winograd for 3x3 convolutions.

Exam Important Points

- Full derivation of Depthwise Separable Convolution computational savings ratio: (1/N) + (1/D_K^2).
- Two-step decomposition: Depthwise (spatial) + Pointwise (cross-channel).
- Convolution Theorem and FFT-based convolution principle.
- Winograd algorithm multiplication reduction for 3x3 filters.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): MobileNet Depthwise Separable FLOP Calculation

Statement: A convolutional layer operates on an input feature map of size 28x28 with M = 64 input channels and produces N = 128 output channels using 3x3 filters (stride = 1, same padding). (a) Calculate the multiplication operations for a standard convolution, (b) Calculate the operations for depthwise convolution, (c) Calculate the operations for pointwise convolution, and (d) Determine the total operations and reduction percentage.

Given: D_F = 28, M = 64, N = 128, D_K = 3.

Governing Formula: Std = D_F^2 * M * N * D_K^2; DW = D_F^2 * M * D_K^2; PW = D_F^2 * M * N; Total_Sep = DW + PW

Step-by-Step Calculation:

Step 1: Compute Standard Convolution operations: Std = (28 * 28) * 64 * 128 * (3 * 3) = 784 * 64 * 128 * 9 = 57,802,752 multiplications.

Step 2: Compute Depthwise Convolution operations: DW = (28 * 28) * 64 * (3 * 3) = 784 * 64 * 9 = 451,584 multiplications.

Step 3: Compute Pointwise (1x1) Convolution operations: PW = (28 * 28) * 64 * 128 * (1 * 1) = 784 * 64 * 128 * 1 = 6,422,528 multiplications.

Step 4: Total Separable Operations = 451,584 + 6,422,528 = 6,874,112 multiplications.

Step 5: Compute reduction percentage: Ratio = 6,874,112 / 57,802,752 = 0.11892 (11.89%). Reduction = (1 - 0.11892) * 100 = 88.11% reduction!

[FINAL EVALUATED ANSWER: Standard = 57.8M ops. Separable = 6.87M ops (88.11% computation reduction, ~8.4x speedup).]

Pedagogical Note: Depthwise separable convolution reduces operations from 57.8 million to 6.87 million while retaining identical channel depth and spatial resolution.

Problem 2 (Advanced (KTU Exam Level)): Winograd Minimal Filtering Multiplications Reduction Proof

Statement: The Winograd minimal filtering algorithm F(m, r) computes m output values using an r-tap FIR filter with m + r - 1 multiplications, whereas standard direct convolution requires m * r multiplications. For 1D convolution F(2, 3) computing 2 outputs with a 3-tap filter: (a) Compare multiplications required by direct vs Winograd convolution, (b) Extend to 2D convolution F(2x2, 3x3) which computes 2x2 outputs using a 3x3 filter, and (c) Calculate the exact theoretical speedup factor achieved by Winograd on modern GPU engines.

Given: 1D: m = 2, r = 3; 2D: F(2x2, 3x3).

Governing Formula: 1D Direct = m * r; 1D Winograd = m + r - 1; 2D Direct = m^2 * r^2; 2D Winograd = (m + r - 1)^2

Step-by-Step Calculation:

Step 1: 1D Convolution: Direct multiplications = m * r = 2 * 3 = 6 multiplications. Winograd multiplications = m + r - 1 = 2 + 3 - 1 = 4 multiplications. 1D savings = 6 / 4 = 1.5x speedup.

Step 2: 2D Convolution F(2x2, 3x3): Direct multiplications = m^2 * r^2 = (2^2) * (3^2) = 4 * 9 = 36 multiplications. Winograd multiplications = (m + r - 1)^2 = (2 + 3 - 1)^2 = 4^2 = 16 multiplications.

Step 3: Calculate 2D speedup factor: Speedup = 36 / 16 = 2.25x speedup! Multiplication reduction = ((36 - 16) / 36) * 100 = 55.56% reduction.

[FINAL EVALUATED ANSWER: 1D Winograd saves 33.3% (1.5x). 2D Winograd reduces multiplications from 36 to 16 (2.25x speedup, 55.56% reduction).]

Pedagogical Note: Winograd minimal filtering cuts 3x3 convolution multiplications by more than half, explaining why NVIDIA cuDNN utilizes Winograd by default for 3x3 kernels.

Real-World Industry Case (Consumer Electronics & Edge Computer Vision): Smart Home Edge Camera Face Unlock

Pipeline Workflow: Battery-powered doorbell camera detects motion -> Microcontroller runs MobileNet with depthwise separable convolutions -> Evaluates live facial biometrics in 30ms on a 1-Watt edge processor.

Industrial Impact & Benefit: Enables instant biometric door unlock without draining battery or sending private home video streams to the cloud.

Key Oral Viva Voce Checkpoints

Q1: What is a Depthwise Separable Convolution?

Ans: A factorized convolution that breaks the standard operation into two steps: a depthwise spatial convolution applied independently per channel, followed by a 1x1 pointwise convolution across channels.

Q2: What is the computational reduction factor achieved by MobileNet depthwise separable convolution?

Ans: (1 / N) + (1 / D_K^2); for 3x3 filters, this yields an 8 to 9-fold computational speedup.

Q3: How does the Convolution Theorem enable fast convolution via FFT?

*Ans: Convolution in the spatial domain corresponds to point-wise multiplication in the frequency domain: I * K = IFFT(FFT(I) odot FFT(K)).*

Q4: For what kernel sizes is FFT convolution faster than direct convolution?

Ans: For large kernels (typically K >= 7x7 or 11x11); for small 3x3 kernels, FFT transform overhead exceeds direct computation.

Q5: What is the Winograd minimal filtering algorithm?

Ans: An algebraic algorithm that computes small convolutions with the minimum possible number of multiplications, speeding up 3x3 convolutions by 2.25x.

LECTURE #23 · MODULE 4 (Topic 4.1)

Date: 27-Oct-2025
Period: Period 2

Title: Recurrent Neural Networks: Computational Graphs and RNN Design

CO: CO4 Bloom's: Understand / Apply

Prerequisites: Backpropagation Algorithm, Sequence modeling intuition, Matrix multiplication, Day 4 concepts
Key Terminology: Recurrent Neural Network (RNN), Hidden State, Computational Graph Unfolding, Backpropagation Through Time (BPTT), Parameter Sharing Across Time, Temporal Receptive Field

Recurrent Equations And Unfolding

A Recurrent Neural Network processes sequential data x^(1), x^(2), ..., x^(tau) by maintaining an internal hidden state memory vector h^(t) that summarizes past history. State Update Equation: h^(t) = tanh(W_hh * h^(t-1) + W_xh * x^(t) + b_h) Output Equation: o^(t) = W_hy * h^(t) + b_y y_hat^(t) = softmax(o^(t)) (or linear / sigmoid for regression) Computational Graph Unfolding: Unfolding (unrolling) maps the recursive loop representation into a directed acyclic computational graph spanning tau discrete time steps. Key insight: The weight matrices W_hh, W_xh, and W_hy are SHARED across all time steps, enabling the network to generalize to arbitrary sequence lengths.

Backpropagation Through Time

Total sequence loss is the sum of losses at each time step: L = sum_{t=1}^tau L^(t). By the multivariable chain rule, the gradient of loss at step tau with respect to hidden state at early step t is a product of Jacobian matrices: del L^(tau) / del h^(t) = (del L^(tau) / del h^(tau)) * product_{j=t+1}^tau (del h^(j) / del h^(j-1)) Each temporal Jacobian is: del h^(j) / del h^(j-1) = W_hh^T * diag(1 - (h^(j))^2) Vanishing / Exploding Problem: If the largest eigenvalue of W_hh is lambda < 1 and tanh' <= 1: The product contracts as (lambda)^T -> 0, causing gradients from distant past steps to vanish completely. The standard RNN cannot bridge long-term dependencies (typically failing beyond 10-15 steps).

Exam Important Points

- Full state update and output equations for simple vanilla RNN.
- Diagram showing recurrent loop representation vs unfolded computational graph.
- Derivation of Backpropagation Through Time (BPTT) chain rule product.
- Why parameter sharing across time is mandatory for variable-length sequence modeling.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Vanilla RNN Hidden State and Output Calculation

Statement: A simple RNN has parameters W_xh = [0.5, 0.2], W_hh = 0.8, b_h = 0.1, W_hy = 1.2, and b_y = -0.3. The activation function for the hidden state is Tanh, and the output neuron is linear. The initial hidden state is h_0 = 0.0. The input sequence has two time steps: x_1 = [1.0, 2.0]^T and x_2 = [0.5, -1.0]^T. Calculate: (a) Hidden state h_1 and output y_1, and (b) Hidden state h_2 and output y_2.

Given: h_0 = 0.0, W_xh = [0.5, 0.2], W_hh = 0.8, b_h = 0.1, W_hy = 1.2, b_y = -0.3.

Governing Formula: h_t = tanh(W_hh * h_{t-1} + W_xh * x_t + b_h); y_t = W_hy * h_t + b_y

Step-by-Step Calculation:

Step 1: Time step t = 1: W_xh * x_1 = (0.5 * 1.0) + (0.2 * 2.0) = 0.5 + 0.4 = 0.9. z_1 = (0.8 * 0.0) + 0.9 + 0.1 = 1.0. h_1 = tanh(1.0) = 0.76159. y_1 = (1.2 * 0.76159) + (-0.3) = 0.9139 - 0.3 = 0.6139.

Step 2: Time step t = 2: W_xh * x_2 = (0.5 * 0.5) + (0.2 * -1.0) = 0.25 - 0.20 = 0.05. W_hh * h_1 = 0.8 * 0.76159 = 0.60927. z_2 = 0.60927 + 0.05 + 0.1 = 0.75927. h_2 = tanh(0.75927) = 0.64082. y_2 = (1.2 * 0.64082) + (-0.3) = 0.7690 - 0.3 = 0.4690.

[FINAL EVALUATED ANSWER: Step 1: h_1 = 0.7616, y_1 = 0.6139. Step 2: h_2 = 0.6408, y_2 = 0.4690.]

Pedagogical Note: The hidden state h_1 retained memory of x_1 and propagated it forward into step 2, combining with x_2 to produce output y_2.

Problem 2 (Advanced (KTU Exam Level)): BPTT Gradient Decay Factor over 10 Time Steps

Statement: In a recurrent neural network, the scalar recurrent weight is W_hh = 0.85. At each time step, the hidden activation derivative is approximately tanh'(z) = 1 - h^2 approx 0.80. (a) Calculate the single-step temporal Jacobian factor gamma = W_hh * tanh'(z), (b) Calculate the total gradient attenuation factor across 10 unrolled time steps: gamma^10, and (c) Determine what percentage of the gradient signal survives backpropagating 10 steps into the past.

Given: W_hh = 0.85, tanh'(z) = 0.80, horizon T = 10 steps.

Governing Formula: Single step gamma = W_hh * tanh'(z); 10-step attenuation = gamma^10; Survival % = gamma^10 * 100

Step-by-Step Calculation:

Step 1: Calculate single-step contraction factor: gamma = 0.85 * 0.80 = 0.680.

Step 2: Calculate attenuation across 10 steps: Attenuation = (0.680)^10 = 0.021139.

Step 3: Calculate percentage of surviving gradient: Percentage = 0.021139 * 100 = 2.11%.

[FINAL EVALUATED ANSWER: Single-step factor gamma = 0.680. 10-step attenuation = 0.0211 (Only 2.11% gradient survives).]

Pedagogical Note: Over 97.8% of the error gradient disappears after just 10 time steps, demonstrating why vanilla RNNs cannot learn long-term temporal dependencies.

Real-World Industry Case (Renewable Energy & Power Distribution): Smart Power Grid Wind Energy Output Forecasting

Pipeline Workflow: Hourly anemometer wind speed and barometric sensor readings over past 24 hours -> Unrolled RNN processes sequential wind gusts -> Predicts mega-watt turbine power generation for next 4 hours.

Industrial Impact & Benefit: Enables power grid operators to balance battery storage and spin up backup generation without blackouts.

Key Oral Viva Voce Checkpoints

Q1: What is the primary architectural difference between an MLP and an RNN?

Ans: An RNN contains cyclic recurrent connections that feed the previous hidden state back into the current step, allowing it to process variable-length sequential data with memory.

Q2: What does 'unfolding' a recurrent computational graph mean?

Ans: Drawing the computational graph across discrete time steps as a sequence of repeated identical layers sharing the same parameters.

Q3: What is Backpropagation Through Time (BPTT)?

Ans: The application of the backpropagation algorithm to an unfolded recurrent network, computing gradients by summing contributions across all time steps.

Q4: Why is parameter sharing essential in RNNs?

Ans: It allows the model to process sequences of arbitrary length and recognize patterns regardless of where they appear in time, using a fixed number of parameters.

Q5: Why do vanilla RNNs suffer from vanishing gradients?

Ans: Because backpropagation over long horizons involves repeatedly multiplying by the recurrent weight matrix W_hh and tanh derivatives (< 1), causing gradients to decay exponentially.

LECTURE #24 · MODULE 4 (Topic 4.2)

Date: 29-Oct-2025
Period: Period 4

Title: Encoder-Decoder Sequence-to-Sequence Architectures

CO: CO4 Bloom's: Understand / Apply

Prerequisites: Vanilla RNN, Hidden states, Day 23 concepts
Key Terminology: Sequence-to-Sequence (Seq2Seq), Encoder RNN, Decoder RNN, Context Vector, Information Bottleneck, Teacher Forcing, Autoregressive Generation, Beam Search

Seq2Seq Architecture

Standard RNNs require input length T_x and output length T_y to be strictly equal or synchronized. Many tasks require mapping a sequence of length T_x to an output sequence of different length T_y (e.g. Machine Translation: 'How are you?' [3 words] -> 'Wie geht es Ihnen?' [4 words]). 1. Encoder RNN: Processes input sequence x^(1), ..., x^(T_x) sequentially: h_enc^(t) = f_enc(h_enc^(t-1), x^(t)) The final hidden state represents the summary Context Vector c = h_enc^(T_x). 2. Decoder RNN: Initializes hidden state with the context vector: h_dec^(0) = c. Autoregressively generates target tokens y^(1), ..., y^(T_y): h_dec^(t) = f_dec(h_dec^(t-1), y^(t-1), c) p(y^(t) | y^(1), ..., y^(t-1), x) = softmax(W_out * h_dec^(t)) Generation terminates when the special <EOS> (End-of-Sequence) token is emitted.

Information Bottleneck And Teacher Forcing

1. Information Bottleneck Problem: Compressing an entire source sentence of arbitrary length (e.g. 50 words) into a single fixed-dimensional vector c (e.g. 512 floats) creates an extreme bottleneck. Model performance deteriorates sharply for sentences longer than 20-30 words (motivating Bahdanau's Attention mechanism). 2. Teacher Forcing: During training, the decoder receives the ground-truth target token y_true^(t-1) as input at step t, rather than its own potentially erroneous previous prediction y_pred^(t-1). - Advantage: Stabilizes and drastically accelerates training. - Drawback (Exposure Bias): At test time, ground truth is absent; early mistakes can compound catastrophically.

Exam Important Points

- Full architectural diagram of Encoder-Decoder Seq2Seq framework.
- How context vector c transfers semantic information between encoder and decoder.
- Definition, merits, and exposure bias drawback of Teacher Forcing.
- Why fixed-length context vectors fail on long sequences.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Context Vector Compression Ratio in Seq2Seq

Statement: An English sentence of 40 words is processed by an Encoder RNN. Each word is represented by a 300-dimensional word embedding vector. The Encoder condenses the sentence into a context vector c of dimension 512. Assuming 32-bit floating point numbers (4 bytes/float): (a) Calculate the raw input memory size in bytes, (b) Calculate the context vector memory size in bytes, and (c) Determine the compression ratio achieved by the bottleneck.

Given: 40 words, 300 dims/word, context c = 512 floats, 4 bytes/float.

Governing Formula: Input_mem = Words * Dims * 4; Context_mem = Dims_c * 4; Ratio = Input_mem / Context_mem

Step-by-Step Calculation:

Step 1: Compute input sentence memory: Total input floats = 40 * 300 = 12,000 floats. Input memory = 12,000 * 4 bytes = 48,000 bytes = 46.88 KB.

Step 2: Compute context vector memory: Context memory = 512 * 4 bytes = 2,048 bytes = 2.0 KB.

Step 3: Calculate compression ratio: Ratio = 48,000 / 2,048 = 23.4375x (over 23-fold compression!).

[FINAL EVALUATED ANSWER: Input = 48,000 bytes. Context = 2,048 bytes. Compression ratio = 23.44x.]

Pedagogical Note: The encoder must compress 48 KB of nuanced semantic and syntactic linguistic relationships into just 2 KB, highlighting why fixed context vectors suffer severe information loss on long documents.

Problem 2 (Advanced (KTU Exam Level)): Beam Search Path Probability vs Greedy Decoding

Statement: At step 1 of decoding, a vocabulary has 3 words: {A, B, C} with probabilities P(A) = 0.6, P(B) = 0.3, P(C) = 0.1. At step 2, conditional probabilities are: If step 1 was A: P(A|A) = 0.2, P(B|A) = 0.3, P(C|A) = 0.5. If step 1 was B: P(A|B) = 0.8, P(B|B) = 0.1, P(C|B) = 0.1. (a) Determine the 2-step sequence selected by Greedy Decoding, (b) Calculate the total sequence probability of all 9 possible combinations, and (c) Determine the optimal sequence found by Beam Search with beam width B = 2.

Given: P(y1), P(y2|y1) as specified.

Governing Formula: P(y1, y2) = P(y1) * P(y2|y1); Greedy: argmax at each step; Beam Search: maintain top B paths at each step

Step-by-Step Calculation:

Step 1: Greedy Decoding: Step 1: max(0.6, 0.3, 0.1) -> selects 'A' (P=0.6). Step 2 (given A): max(0.2, 0.3, 0.5) -> selects 'C' (P=0.5). Greedy sequence: 'A C' with total probability = 0.6 * 0.5 = 0.30.

Step 2: Evaluate alternative candidate paths starting from B: P('B A') = P(B) * P(A|B) = 0.3 * 0.8 = 0.24. P('B B') = 0.3 * 0.1 = 0.03. P('B C') = 0.3 * 0.1 = 0.03.

Step 3: Evaluate paths starting from A: P('A A') = 0.6 * 0.2 = 0.12. P('A B') = 0.6 * 0.3 = 0.18. P('A C') = 0.6 * 0.5 = 0.30.

Step 4: Beam Search with Beam Width B = 2: Step 1: Top 2 candidates retained: Path 'A' (0.6) and Path 'B' (0.3). Step 2 expansions: 'A A' = 0.12, 'A B' = 0.18, 'A C' = 0.30, 'B A' = 0.24, 'B B' = 0.03, 'B C' = 0.03. Top 2 pruned beam paths: 'A C' (0.30) and 'B A' (0.24). Optimal path = 'A C' (0.30).

[FINAL EVALUATED ANSWER: Greedy = 'A C' (P = 0.30). Beam Search (B=2) retains both 'A C' (0.30) and 'B A' (0.24), confirming 'A C' as optimal.]

Pedagogical Note: Greedy search makes locally optimal decisions that can miss globally superior sequences, whereas Beam Search explores multiple plausible trajectories concurrently.

Real-World Industry Case (Medical Informatics & Cross-Lingual Healthcare): Automated Clinical Discharge Summary Translation

Pipeline Workflow: Doctor dictates English clinical notes -> Encoder RNN ingests medical diagnosis jargon -> Produces latent context embedding -> Decoder RNN generates patient-friendly regional language discharge instructions.

Industrial Impact & Benefit: Eliminates language barriers in rural hospital discharge, cutting medication dosing misunderstandings by 35%.

Key Oral Viva Voce Checkpoints

Q1: What is the core capability that Seq2Seq provides over standard vanilla RNNs?

Ans: Mapping an input sequence of length T_x to an output sequence of a completely different length T_y.

Q2: What is the Context Vector c in Seq2Seq?

Ans: The final hidden state vector of the Encoder RNN that summarizes the entire source sentence and serves as the seed memory for the Decoder.

Q3: What is the Information Bottleneck problem in Seq2Seq?

Ans: Forcing an entire source sentence of arbitrary length into a single fixed-size vector causes severe performance degradation on long sequences.

Q4: What is Teacher Forcing?

Ans: A training procedure where the ground-truth target token from the previous step is fed into the decoder, rather than the model's own predicted token.

Q5: What is Exposure Bias in sequence generation?

Ans: The discrepancy between training (where ground truth tokens are provided) and inference (where the model must consume its own predictions), allowing early generation errors to compound.

LECTURE #25 · MODULE 4 (Topic 4.3)

Date: 31-Oct-2025
Period: Period 1

Title: Deep Recurrent Networks and Recursive Neural Networks

CO: CO4 Bloom's: Understand / Apply

Prerequisites: RNN Unfolding, Seq2Seq, Tree data structures, Day 24 concepts
Key Terminology: Deep Recurrent Network, Stacked RNN, Bidirectional RNN (BiRNN), Forward State, Backward State, Recursive Neural Network, Tree-Structured Graph, Constituency Parse Tree

Deep Stacked Rnns

A standard RNN is deep in time (across unrolled sequence steps), but shallow in spatial representation (only one hidden layer per time step). A Deep (Stacked) RNN stacks multiple recurrent layers vertically: - Layer 1 receives input sequence x^(t): h_1^(t) = f(W_1 * x^(t) + U_1 * h_1^(t-1)) - Layer 2 receives hidden state sequence of Layer 1 as its input: h_2^(t) = f(W_2 * h_1^(t) + U_2 * h_2^(t-1)) - Layer L computes: h_L^(t) = f(W_L * h_{L-1}^(t) + U_L * h_L^(t-1)) Hierarchical Temporal Abstraction: Lower recurrent layers capture fast, fine-grained temporal patterns (phonemes, characters), while higher recurrent layers capture slow, abstract semantic structures (syntax, topics).

Bidirectional Rnns

Schuster & Paliwal (1997) introduced Bidirectional RNNs (BiRNNs) to resolve the limitation that standard causal RNNs only know past context. Equations: 1. Forward RNN: h_fwd^(t) = f(W_xf * x^(t) + W_ff * h_fwd^(t-1)) (processes sequence left-to-right) 2. Backward RNN: h_bwd^(t) = f(W_xb * x^(t) + W_bb * h_bwd^(t+1)) (processes sequence right-to-left) 3. Concatenated Representation: h^(t) = [h_fwd^(t) ; h_bwd^(t)] Output y^(t) is conditioned simultaneously on past history AND future context! Critical for speech recognition, POS tagging, and NER.

Recursive Neural Networks

While an RNN operates over a 1D chain graph of sequence length T, a Recursive Neural Network (Pollack 1990; Socher et al. 2011) operates over an arbitrary Directed Tree-Structured Graph. Tree Propagation: For a node with children c_1 and c_2: p = f(W * [c_1 ; c_2] + b) The parent representation p is computed recursively up to the root node. Advantage: Drastically reduces the maximum path length between any two words from O(T) in a linear RNN to O(log T) in a balanced tree, massively mitigating vanishing gradients and naturally matching the hierarchical syntax of natural language sentences and molecular graphs.

Exam Important Points

- Architectural equations and diagram for Bidirectional RNNs.
- Comparison: Recurrent (1D sequential chain) vs Recursive (hierarchical tree graph).
- Why BiRNNs cannot be used for online real-time causal generation.
- Logarithmic path length O(log T) in Recursive networks.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Trainable Parameter Count in a 3-Layer Deep Stacked RNN

Statement: A deep stacked RNN has an input dimension of x_dim = 128. It contains 3 stacked recurrent hidden layers, each having hidden dimension h_dim = 256. The top hidden layer connects to an output layer of dimension y_dim = 10. Assuming each layer has full input-to-hidden and hidden-to-hidden matrices plus biases: (a) Calculate the parameters in Layer 1, (b) Calculate the parameters in Layer 2 and Layer 3, (c) Calculate output layer parameters, and (d) Determine the total trainable parameters.

Given: x_dim = 128, h1 = 256, h2 = 256, h3 = 256, y_dim = 10.

Governing Formula: Layer 1 = (x_dim * h) + (h * h) + h; Middle Layer = (h_prev * h) + (h * h) + h; Out = (h * y) + y

Step-by-Step Calculation:

Step 1: Layer 1 (Input 128 -> Hidden 256): W_x1 = 256 * 128 = 32,768. W_h1 = 256 * 256 = 65,536. b_1 = 256. Total Layer 1 = 32,768 + 65,536 + 256 = 98,560 parameters.

Step 2: Layer 2 (Hidden 256 -> Hidden 256): W_x2 = 256 * 256 = 65,536. W_h2 = 256 * 256 = 65,536. b_2 = 256. Total Layer 2 = 65,536 + 65,536 + 256 = 131,328 parameters.

Step 3: Layer 3 (Hidden 256 -> Hidden 256): Total Layer 3 = 131,328 parameters.

Step 4: Output Layer (Hidden 256 -> Output 10): W_out = 10 * 256 = 2,560. b_out = 10. Total Output = 2,560 + 10 = 2,570 parameters.

Step 5: Total Trainable Parameters = 98,560 + 131,328 + 131,328 + 2,570 = 363,786 parameters.

[FINAL EVALUATED ANSWER: Layer 1 = 98,560; Layer 2 = 131,328; Layer 3 = 131,328; Output = 2,570. Total = 363,786 parameters.]

Pedagogical Note: Stacking layers creates multi-tier hierarchical representations with moderate parameter growth compared to widening.

Problem 2 (Advanced (KTU Exam Level)): Path Length and Gradient Propagation: Recurrent Chain vs Recursive Tree

Statement: Consider a sequence of length T = 64 tokens. (a) In a standard 1D recurrent chain, calculate the maximum number of sequential transitions required for an error signal to propagate from token 64 to token 1, (b) In a balanced binary recursive parse tree, calculate the maximum path length from the root to any leaf token, and (c) Assuming a per-step contraction factor gamma = 0.8, compare the gradient attenuation in the Recurrent chain vs the Recursive tree.

Given: T = 64 tokens, gamma = 0.8.

Governing Formula: Recurrent Path = T - 1; Recursive Balanced Tree Path = 2 * log2(T); Attenuation = gamma^(Path)

Step-by-Step Calculation:

Step 1: Recurrent 1D Chain Path Length: Path = 64 - 1 = 63 transitions.

Step 2: Recursive Balanced Binary Tree Path Length: Tree height = log2(64) = 6 levels. Maximum path between two leaves through root = 2 * 6 = 12 transitions.

Step 3: Compute Recurrent gradient attenuation: Attenuation_RNN = (0.8)^63 = 1.07 x 10^(-6) (Completely vanished!).

Step 4: Compute Recursive gradient attenuation: Attenuation_Recursive = (0.8)^12 = 0.0687 (6.87% signal preserved!).

Step 5: Comparison ratio: Ratio = 0.0687 / (1.07 x 10^(-6)) = ~64,000x stronger gradient signal!

[FINAL EVALUATED ANSWER: Recurrent Path = 63 steps (Atten = 1.07e-6). Recursive Tree Path = 12 steps (Atten = 0.0687, ~64,000x stronger signal).]

Pedagogical Note: The tree topology of Recursive Neural Networks compresses sequential path lengths from O(T) to O(log T), dramatically preserving gradient signals across distant words.

Real-World Industry Case (Clinical NLP & Biomedical Information Extraction): Named Entity Recognition (NER) in Electronic Health Records (EHR)

Pipeline Workflow: Clinical doctor discharge note -> Ingested by a 2-layer Bidirectional LSTM -> Forward state understands preceding symptoms, Backward state understands succeeding medication dosages -> Accurately tags entities (Disease, Drug, Dosage).

Industrial Impact & Benefit: Extracts structured clinical adverse drug event records from unstructured physician notes with 93.8% F1-score.

Key Oral Viva Voce Checkpoints

Q1: What is the primary motivation for Bidirectional RNNs?

Ans: Standard RNNs only possess information about past context; BiRNNs process the sequence in both forward and backward directions, giving each token full access to both past and future context.

Q2: Why can Bidirectional RNNs NOT be used for real-time online speech generation or autoregressive language modeling?

Ans: Because the backward pass requires access to future tokens, which have not yet occurred or been spoken in real time.

Q3: What is a Deep (Stacked) RNN?

Ans: An architecture where the output hidden states of one recurrent layer serve as the input sequence to a second recurrent layer above it.

Q4: What is the primary difference between a Recurrent and a Recursive Neural Network?

Ans: A Recurrent network operates over a 1D linear chain of time steps, whereas a Recursive network operates over an arbitrary hierarchical tree graph (e.g. syntax parse tree).

Q5: What is the theoretical advantage of a Recursive Neural Network for natural language?

Ans: It matches the innate hierarchical constituency grammar of human language and reduces the gradient propagation distance between words from O(T) to O(log T).

LECTURE #26 · MODULE 4 (Topic 4.4)

Date: 03-Nov-2025
Period: Period 2

Title: Modern RNNs: Long Short-Term Memory (LSTM)

CO: CO4 Bloom's: Understand / Apply

Prerequisites: Vanilla RNN, Backpropagation Through Time, Vanishing Gradients, Day 23 concepts
Key Terminology: Long Short-Term Memory (LSTM), Cell State, Constant Error Carousel (CEC), Forget Gate, Input Gate, Output Gate, Candidate Memory State, Additive Gradient Highway

Lstm Architecture And Equations

Hochreiter & Schmidhuber (1997) designed the LSTM to maintain a linear memory cell state c^(t) where error gradients can flow backward across time without exponential decay. Given input x^(t) and previous hidden state h^(t-1): 1. Forget Gate (Gers et al., 2000): f^(t) = sigma(W_f * [h^(t-1) ; x^(t)] + b_f) Decides what fraction of the old cell state to discard (0 = forget completely, 1 = retain entirely). 2. Input Gate & Candidate State: i^(t) = sigma(W_i * [h^(t-1) ; x^(t)] + b_i) (determines which values to update) c_tilde^(t) = tanh(W_c * [h^(t-1) ; x^(t)] + b_c) (creates new candidate values) 3. Cell State Update (The Additive Highway): c^(t) = f^(t) odot c^(t-1) + i^(t) odot c_tilde^(t) Key Insight: This is a LINEAR, ADDITIVE combination! If the forget gate f^(t) = 1, the old memory passes through with a derivative of exactly 1.0, enabling error signals to flow back hundreds of steps without decaying. 4. Output Gate & Hidden State: o^(t) = sigma(W_o * [h^(t-1) ; x^(t)] + b_o) h^(t) = o^(t) odot tanh(c^(t)) Filters the cell state through the output gate to produce the external hidden state.

Mathematical Resolution Of Vanishing Gradients

In a vanilla RNN, del h^(t) / del h^(t-1) = W_hh^T * diag(1 - h^2). Repeated multiplication of matrices with eigenvalues < 1 causes exponential decay. In an LSTM, the gradient flowing along the cell state is: del c^(t) / del c^(t-1) = diag(f^(t)) If the network learns that forget gate f^(t) approx 1.0, the Jacobian is the IDENTITY matrix: del c^(tau) / del c^(t) = product_{j=t+1}^tau f^(j) approx 1.0 Gradients bypass the destructive multiplication by weight matrices entirely, flowing uninterrupted across hundreds of steps.

Exam Important Points

- Full mathematical formulas for all four LSTM gates/states: f, i, c_tilde, c, o, h.
- Architectural block diagram showing cell state conveyor belt and pointwise operations.
- Mathematical explanation of why del c^(t) / del c^(t-1) = f^(t) resolves vanishing gradients.
- KTU Exam Q18(a): Applications of LSTM in Natural Language Processing (Machine Translation, Text Generation, Sentiment Analysis).

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Manual Step-by-Step LSTM Gate and State Calculation

Statement: An LSTM has scalar dimensions (x_dim = 1, h_dim = 1). At step t = 1, previous hidden state is h_0 = 0.5 and previous cell state is c_0 = 2.0. The input is x_1 = 1.0. Parameters are: Forget gate: W_f = 0.5, U_f = 0.2, b_f = 0.1 (Sigmoid) Input gate: W_i = 0.4, U_i = 0.3, b_i = -0.2 (Sigmoid) Candidate state: W_c = 0.8, U_c = 0.1, b_c = 0.0 (Tanh) Output gate: W_o = 0.6, U_o = -0.1, b_o = 0.2 (Sigmoid) Calculate: (a) Forget gate f_1, (b) Input gate i_1 and candidate c_tilde_1, (c) Updated cell state c_1, and (d) Updated hidden state h_1.

Given: h_0 = 0.5, c_0 = 2.0, x_1 = 1.0. Parameters as stated.

Governing Formula: f = sigma(W_f*x + U_f*h + b_f); i = sigma(W_i*x + U_i*h + b_i); c_tilde = tanh(W_c*x + U_c*h + b_c); c = f*c_prev + i*c_tilde; o = sigma(W_o*x + U_o*h + b_o); h = o * tanh(c)

Step-by-Step Calculation:

Step 1: Compute Forget Gate f_1: z_f = (0.5 * 1.0) + (0.2 * 0.5) + 0.1 = 0.5 + 0.1 + 0.1 = 0.70. f_1 = 1 / (1 + e^(-0.70)) = 1 / (1 + 0.49659) = 1 / 1.49659 = 0.6682.

Step 2: Compute Input Gate i_1 and Candidate c_tilde_1: z_i = (0.4 * 1.0) + (0.3 * 0.5) + (-0.2) = 0.4 + 0.15 - 0.2 = 0.35. i_1 = 1 / (1 + e^(-0.35)) = 1 / (1 + 0.70469) = 1 / 1.70469 = 0.5866. z_c = (0.8 * 1.0) + (0.1 * 0.5) + 0.0 = 0.8 + 0.05 = 0.85. c_tilde_1 = tanh(0.85) = 0.6911.

Step 3: Update Cell State c_1: c_1 = (f_1 * c_0) + (i_1 * c_tilde_1) = (0.6682 * 2.0) + (0.5866 * 0.6911) = 1.3364 + 0.4054 = 1.7418.

Step 4: Compute Output Gate o_1 and Hidden State h_1: z_o = (0.6 * 1.0) + (-0.1 * 0.5) + 0.2 = 0.6 - 0.05 + 0.2 = 0.75. o_1 = 1 / (1 + e^(-0.75)) = 1 / (1 + 0.47237) = 1 / 1.47237 = 0.6792. tanh(c_1) = tanh(1.7418) = 0.9404. h_1 = o_1 * tanh(c_1) = 0.6792 * 0.9404 = 0.6387.

[FINAL EVALUATED ANSWER: f_1 = 0.6682, i_1 = 0.5866, c_tilde_1 = 0.6911, Cell State c_1 = 1.7418, Hidden State h_1 = 0.6387]

Pedagogical Note: The forget gate retained 66.8% of past memory c_0, added new information scaled by i_1, and exposed 63.8% of the transformed memory to the hidden state.

Problem 2 (Advanced (KTU Exam Level)): Trainable Parameter Count in Standard Multi-Class LSTM Layer

Statement: An LSTM layer receives sequential word embeddings of dimension x_dim = 300 and contains h_dim = 512 hidden units. It connects to an output classification vocabulary of size V = 10,000 words. (a) Derive the parameter formula for an LSTM layer containing 4 internal gates/states, (b) Calculate total weights and biases in the recurrent LSTM layer, (c) Calculate parameters in the final projection layer, and (d) Determine the grand total trainable parameters.

Given: x_dim = 300, h_dim = 512, V = 10,000.

Governing Formula: LSTM Params = 4 * [h_dim * (x_dim + h_dim) + h_dim]; Projection = (h_dim * V) + V

Step-by-Step Calculation:

Step 1: Understand LSTM 4-fold parameter structure: The LSTM contains 4 linear transformations: Forget gate, Input gate, Candidate cell, Output gate. Each gate has input weights (h x x_dim), recurrent weights (h x h), and bias (h).

Step 2: Parameters per single gate: W_x = 512 * 300 = 153,600. W_h = 512 * 512 = 262,144. b = 512. Per-gate parameters = 153,600 + 262,144 + 512 = 416,256.

Step 3: Total LSTM Layer parameters (4 gates): LSTM_total = 4 * 416,256 = 1,665,024 parameters.

Step 4: Output Projection Layer: W_proj = 10,000 * 512 = 5,120,000. b_proj = 10,000. Total Projection = 5,120,000 + 10,000 = 5,130,000 parameters.

Step 5: Grand Total Parameters = 1,665,024 + 5,130,000 = 6,795,024 parameters.

[FINAL EVALUATED ANSWER: LSTM Layer = 1,665,024 params. Output Layer = 5,130,000 params. Grand Total = 6,795,024 parameters.]

Pedagogical Note: An LSTM has exactly 4 times the recurrent parameters of a vanilla RNN of the same hidden width due to its four internal gating projections.

Real-World Industry Case (Critical Care Medicine & Early Warning Clinical Systems): Long-Term ICU Patient Sepsis Onset Prediction

Pipeline Workflow: Patient vital signs recorded every minute over a 72-hour ICU stay (4,320 time steps) -> LSTM cell state conveyor belt maintains long-term baseline organ history -> Predicts septic shock 6 hours before clinical manifestation.

Industrial Impact & Benefit: Enables preemptive broad-spectrum antibiotic intervention, reducing ICU sepsis mortality by 28%.

Key Oral Viva Voce Checkpoints

Q1: Who invented the LSTM architecture and in what year?
Ans: Sepp Hochreiter and Jürgen Schmidhuber in 1997.

Q2: What are the three gates present in a standard modern LSTM?
Ans: The Forget Gate, the Input Gate, and the Output Gate.

Q3: What is the mathematical role of the Forget Gate?
Ans: To compute a probability vector f in [0, 1] using a Sigmoid function that determines which components of the previous cell state c^(t-1) are preserved or erased.

Q4: Why is the cell state update c^(t) = f*c^(t-1) + i*c_tilde called an additive conveyor belt?
Ans: Because information is added linearly without passing through a matrix multiplication, allowing error gradients to propagate unchanged with derivative ~ 1.0.

Q5: Why should the forget gate bias b_f be initialized to 1.0 or 2.0 at the start of training?
Ans: To ensure that f^(t) approx 1.0 initially, preventing the network from forgetting historical context during early training epochs.

LECTURE #27 · MODULE 4 (Topic 4.5)

Date: 05-Nov-2025
Period: Period 4

Title: Gated Recurrent Unit (GRU) and Modern Sequence Architectures

CO: CO4 Bloom's: Understand / Apply

Prerequisites: LSTM Architecture, Gating mechanisms, Day 26 concepts
Key Terminology: Gated Recurrent Unit (GRU), Reset Gate, Update Gate, Merged Hidden State, Convex Linear Interpolation, Parameter Efficiency, LSTM vs GRU Trade-off

Gru Architecture And Equations

Kyunghyun Cho et al. (2014) introduced the Gated Recurrent Unit (GRU) as a streamlined, computationally efficient alternative to LSTM. Key Structural Differences from LSTM: 1. No separate Cell State: GRU merges cell state c^(t) and hidden state h^(t) into a single state vector h^(t). 2. Two Gates instead of Three: Eliminates the separate output gate. Uses only a Reset Gate r^(t) and an Update Gate z^(t). 3. Coupled Retention and Injection: In LSTM, forget gate f and input gate i are independent. In GRU, the update gate z controls both simultaneously via convex linear interpolation (1 - z) and z. Mathematical Formulation: 1. Update Gate: z^(t) = sigma(W_z * [h^(t-1) ; x^(t)] + b_z) Decides how much of past state to carry forward vs new state. 2. Reset Gate: r^(t) = sigma(W_r * [h^(t-1) ; x^(t)] + b_r) Decides how much of past state to ignore when computing candidate. 3. Candidate Hidden State: h_tilde^(t) = tanh(W_h * [r^(t) odot h^(t-1) ; x^(t)] + b_h) 4. Final Hidden State (Linear Interpolation): h^(t) = (1 - z^(t)) odot h^(t-1) + z^(t) odot h_tilde^(t)

Lstm Vs Gru Comprehensive Comparison

Comparison Dimension | LSTM | GRU Gates | 3 (Forget, Input, Output) | 2 (Reset, Update) Internal States | 2 (Cell c + Hidden h) | 1 (Merged Hidden h) Parameters per Unit | 4 * (d_x + d_h + 1) * d_h | 3 * (d_x + d_h + 1) * d_h (25% fewer parameters!) Computation Speed | Slower (more matrix multiplies) | Faster (~25-30% faster per epoch) Memory Usage | Higher (stores c and h) | Lower Long Dependencies | Superior on massive data / long sequences | Comparable on moderate sequences, less prone to overfitting on small datasets.

Exam Important Points

- KTU Exam Q18(b): Complete architectural discussion and equations of GRU.
- Equations for Update Gate z, Reset Gate r, Candidate h_tilde, and Final h.
- How GRU couples forgetting and updating via (1 - z)*h_prev + z*h_tilde.
- Detailed comparison table: LSTM vs GRU.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Manual Step-by-Step GRU State Calculation

Statement: A GRU cell has scalar dimensions (x_dim = 1, h_dim = 1). At step t = 1, the previous hidden state is h_0 = 0.6. The input is x_1 = 0.5. Parameters are: Reset gate: W_xr = 0.4, W_hr = 0.5, b_r = -0.1 (Sigmoid) Update gate: W_xz = 0.3, W_hz = 0.2, b_z = 0.1 (Sigmoid) Candidate state: W_xh = 0.6, W_hh = 0.8, b_h = 0.0 (Tanh) Calculate: (a) Reset gate r_1, (b) Update gate z_1, (c) Candidate state h_tilde_1, and (d) Final updated hidden state h_1.

Given: h_0 = 0.6, x_1 = 0.5. Parameters as specified.

Governing Formula: r = sigma(W_xr*x + W_hr*h + b_r); z = sigma(W_xz*x + W_hz*h + b_z); h_tilde = tanh(W_xh*x + W_hh*(r*h) + b_h); h = (1 - z)*h_prev + z*h_tilde

Step-by-Step Calculation:

Step 1: Compute Reset Gate r_1: z_r = (0.4 * 0.5) + (0.5 * 0.6) + (-0.1) = 0.20 + 0.30 - 0.10 = 0.40. r_1 = 1 / (1 + e^(-0.40)) = 1 / (1 + 0.67032) = 1 / 1.67032 = 0.5987.

Step 2: Compute Update Gate z_1: z_z = (0.3 * 0.5) + (0.2 * 0.6) + 0.10 = 0.15 + 0.12 + 0.10 = 0.37. z_1 = 1 / (1 + e^(-0.37)) = 1 / (1 + 0.69073) = 1 / 1.69073 = 0.5915.

Step 3: Compute Candidate State h_tilde_1: Reset-masked previous state = r_1 * h_0 = 0.5987 * 0.6 = 0.3592. z_h = (0.6 * 0.5) + (0.8 * 0.3592) + 0.0 = 0.30 + 0.2874 = 0.5874. h_tilde_1 = tanh(0.5874) = 0.5281.

Step 4: Compute Final Hidden State h_1: h_1 = (1.0 - z_1) * h_0 + (z_1 * h_tilde_1) = (1.0 - 0.5915) * 0.6 + (0.5915 * 0.5281) = (0.4085 * 0.6) + 0.3124 = 0.2451 + 0.3124 = 0.5575.

[FINAL EVALUATED ANSWER: Reset gate r_1 = 0.5987, Update gate z_1 = 0.5915, Candidate h_tilde_1 = 0.5281, Final Hidden State h_1 = 0.5575]

Pedagogical Note: The update gate z_1 = 0.5915 blended 40.85% of old state h_0 with 59.15% of the newly computed candidate h_tilde_1.

Problem 2 (Advanced (KTU Exam Level)): Exact Parameter Reduction Ratio: GRU vs LSTM

Statement: An embedded edge speech recognition system has an input dimension of x_dim = 80 (filterbank features) and a recurrent hidden dimension of h_dim = 256. (a) Calculate the total weights and biases in an LSTM layer, (b) Calculate the total weights and biases in a GRU layer, (c) Determine the exact number of parameters saved and the percentage reduction, and (d) Calculate the GPU inference memory saved for a batch size of 64 over 100 time steps.

Given: x_dim = 80, h_dim = 256, batch = 64, T = 100.

Governing Formula: LSTM = 4 * [h*(x + h) + h]; GRU = 3 * [h*(x + h) + h]; Ratio = 3 / 4 = 0.75 (25% reduction)

Step-by-Step Calculation:

Step 1: Compute single-gate parameters: Single gate = 256 * (80 + 256) + 256 = 256 * 336 + 256 = 86,016 + 256 = 86,272 parameters.

Step 2: LSTM Parameters (4 gates): LSTM_total = 4 * 86,272 = 345,088 parameters.

Step 3: GRU Parameters (3 gates): GRU_total = 3 * 86,272 = 258,816 parameters.

Step 4: Compute parameter reduction: Difference = 345,088 - 258,816 = 86,272 parameters. Percentage Reduction = (86,272 / 345,088) * 100 = 25.00% exact reduction!

Step 5: Compute state cache memory savings at inference: LSTM caches both cell state c and hidden state h: 2 * (64 * 256) floats = 32,768 floats = 128 KB. GRU caches only hidden state h: 1 * (64 * 256) floats = 16,384 floats = 64 KB (50% state memory savings!).

[FINAL EVALUATED ANSWER: LSTM = 345,088 params. GRU = 258,816 params (Exact 25% parameter reduction; 50% state cache savings).]

Pedagogical Note: GRU slashes parameter count by exactly 25% and halves state memory requirements, making it ideal for edge microcontrollers and low-latency audio devices.

Real-World Industry Case (Smart Home IoT & Ultra-Low-Power Edge Audio): On-Device Voice Assistant Wake-Word Detection

Pipeline Workflow: Always-on microphone samples ambient audio at 16 kHz -> 2-layer GRU consumes streaming 80-band Mel filterbanks -> Slashes memory and battery by 25% compared to LSTM -> Triggers main CPU only when wake-word ('Hey Google') is detected.

Industrial Impact & Benefit: Maintains 98.7% wake-word detection accuracy while drawing less than 15 milli-Watts on battery power.

Key Oral Viva Voce Checkpoints

Q1: Who invented the Gated Recurrent Unit (GRU) and in what year?

Ans: Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio in 2014.

Q2: What are the two gates in a GRU?

Ans: The Reset Gate and the Update Gate.

Q3: What is the function of the Reset Gate in a GRU?

Ans: It determines how much of the previous hidden state h^(t-1) should be ignored when computing the new candidate hidden state h_tilde^(t).

Q4: What is the function of the Update Gate in a GRU?

Ans: It controls the linear interpolation balance between the previous hidden state h^(t-1) and the candidate hidden state h_tilde^(t).

Q5: How does GRU combine the functions of the LSTM Forget and Input gates?

Ans: It couples them into a single update gate z: the old state is scaled by (1 - z) while the candidate state is scaled by z.

LECTURE #28 · MODULE 5 (Topic 5.1)

Date: 07-Nov-2025
Period: Period 1

Title: Deep Learning Applications: Computer Vision

CO: CO5 Bloom's: Understand / Apply

Prerequisites: CNN fundamentals, Convolutions, Pooling, Day 18 & 21 concepts
Key Terminology: Computer Vision, Object Detection, Two-Stage Detectors, Single-Stage Detectors (YOLO), Intersection over Union (IoU), Non-Maximum Suppression (NMS), Mean Average Precision (mAP)

Computer Vision Taxonomy

Deep learning has transformed computer vision across four primary task tiers: 1. Image Classification: Predict a single discrete category label for the entire image (e.g. ResNet on ImageNet 1k). 2. Object Localization: Predict class label AND a single bounding box [x_min, y_min, x_max, y_max]. 3. Object Detection: Identify and localize ALL object instances across multiple categories with multiple bounding boxes. 4. Semantic & Instance Segmentation: Pixel-level dense category assignment (Semantic) or per-object mask extraction (Instance, e.g. Mask R-CNN).

Object Detection Architectures

1. Two-Stage Detectors (Faster R-CNN, Ren et al., 2015): - Stage 1: Region Proposal Network (RPN) slides across convolutional feature maps to generate candidate object proposals with anchor boxes. - Stage 2: RoI Pooling/Align extracts fixed-size feature vectors per proposal for classification and bounding box regression. - Advantage: High precision and small object recall. 2. One-Stage Detectors (YOLO - Redmon et al., 2016; SSD): - Divides image into an S x S spatial grid. Each grid cell directly predicts B bounding boxes, objectness confidences, and C class probabilities in a single forward pass. - Advantage: Ultra-fast real-time inference (45 to 150+ fps), enabling live edge deployment.

Evaluation Metrics

1. Intersection over Union (IoU): IoU = Area(Box_pred cap Box_gt) / Area(Box_pred cup Box_gt) A prediction is counted as True Positive if IoU >= threshold (typically 0.50 or 0.75). 2. Non-Maximum Suppression (NMS): Sorts candidate boxes by confidence score and iteratively suppresses lower-scoring overlapping boxes with IoU > threshold to eliminate redundant detections. 3. Mean Average Precision (mAP): Computes the area under the Precision-Recall curve for each class across IoU thresholds, averaged across all categories.

Exam Important Points

- Contrast Two-Stage (Faster R-CNN) with One-Stage (YOLO) detectors.
- Mathematical formulation and manual calculation of Intersection over Union (IoU).
- Non-Maximum Suppression (NMS) algorithm steps.
- Definitions of Precision, Recall, and Mean Average Precision (mAP).

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Manual Intersection over Union (IoU) Calculation

Statement: Two bounding boxes are defined on an image grid: Ground Truth Box A: [x1 = 20, y1 = 20, x2 = 80, y2 = 80] Predicted Box B: [x1 = 40, y1 = 40, x2 = 100, y2 = 100] Calculate: (a) Area of Box A, (b) Area of Box B, (c) Area of Intersection, (d) Area of Union, and (e) The exact Intersection over Union (IoU).

Given: Box A = [20, 20, 80, 80], Box B = [40, 40, 100, 100].

Governing Formula: Area = (x2 - x1) * (y2 - y1); Inter = max(0, x_right - x_left) * max(0, y_bottom - y_top); Union = Area_A + Area_B - Inter; IoU = Inter / Union

Step-by-Step Calculation:

Step 1: Compute Area of Box A: Width_A = 80 - 20 = 60; Height_A = 80 - 20 = 60. Area_A = 60 * 60 = 3,600 sq pixels.

Step 2: Compute Area of Box B: Width_B = 100 - 40 = 60; Height_B = 100 - 40 = 60. Area_B = 60 * 60 = 3,600 sq pixels.

Step 3: Compute Intersection Coordinates: x_left = max(20, 40) = 40. y_top = max(20, 40) = 40. x_right = min(80, 100) = 80. y_bottom = min(80, 100) = 80. Inter_width = 80 - 40 = 40; Inter_height = 80 - 40 = 40. Area_Inter = 40 * 40 = 1,600 sq pixels.

Step 4: Compute Area of Union: Area_Union = Area_A + Area_B - Area_Inter = 3,600 + 3,600 - 1,600 = 5,600 sq pixels.

Step 5: Compute IoU: IoU = 1,600 / 5,600 = 16 / 56 = 2 / 7 = 0.2857.

[FINAL EVALUATED ANSWER: Area_A = 3600, Area_B = 3600, Inter = 1600, Union = 5600, IoU = 0.2857 (28.57%)]

Pedagogical Note: Since IoU = 0.2857 is below the standard 0.50 threshold, this prediction is classified as a False Positive in detection benchmark scoring.

Problem 2 (Advanced (KTU Exam Level)): Mean Average Precision (mAP) 11-Point Interpolation Calculation

Statement: An object detector outputs 5 ranked predictions for a class with 4 ground truth instances. Ranked detections have precision and recall values as follows: Rank 1: Precision = 1.00, Recall = 0.25 (TP) Rank 2: Precision = 0.50, Recall = 0.25 (FP) Rank 3: Precision = 0.67, Recall = 0.50 (TP) Rank 4: Precision = 0.50, Recall = 0.50 (FP) Rank 5: Precision = 0.60, Recall = 0.75 (TP) Using the standard Pascal VOC 11-point interpolation method at recall levels r in {0.0, 0.1, 0.2, ..., 1.0} where p_interp(r) = max_{r_tilde >= r} p(r_tilde): Calculate the Average Precision (AP).

Given: 4 ground truth objects, 5 ranked detections with (P, R) pairs.

Governing Formula: AP_11 = (1 / 11) * sum_{r in {0.0, 0.1, ..., 1.0}} p_interp(r)

Step-by-Step Calculation:

Step 1: Maximum precision values achievable for r_tilde >= r: For r in [0.0, 0.25]: Max precision = max(1.00, 0.67, 0.60) = 1.00. For r in (0.25, 0.50]: Max precision = max(0.67, 0.60) = 0.67. For r in (0.50, 0.75]: Max precision = 0.60. For r in (0.75, 1.00]: Since max recall reached is 0.75, precision = 0.00.

Step 2: Evaluate 11 recall thresholds: r = 0.0: p = 1.00 r = 0.1: p = 1.00 r = 0.2: p = 1.00 r = 0.3: p = 0.67 r = 0.4: p = 0.67 r = 0.5: p = 0.67 r = 0.6: p = 0.60 r = 0.7: p = 0.60 r = 0.8: p = 0.00 r = 0.9: p = 0.00 r = 1.0: p = 0.00.

Step 3: Sum interpolated precisions: Sum = 1.00*3 + 0.67*3 + 0.60*2 + 0.00*3 = 3.00 + 2.01 + 1.20 = 6.21.

Step 4: Compute Average Precision: AP = 6.21 / 11 = 0.5645.

[FINAL EVALUATED ANSWER: 11-point interpolated Average Precision AP = 0.5645 (56.45%)]

Pedagogical Note: The 11-point interpolation method smooths out precision fluctuations, penalizing the model for failing to reach 100% recall (precision drops to 0 at r >= 0.8).

Real-World Industry Case (Automotive Safety & Self-Driving Vehicles): Autonomous Highway Collision Avoidance & Emergency Braking

Pipeline Workflow: Triple front-facing camera array streams 60 fps 4K video -> Real-time YOLOv8 detector localizes oncoming vehicles, lane dividers, and pedestrians -> IoU tracker associates objects across frames -> Triggers automatic emergency braking if collision time < 1.2 seconds.

Industrial Impact & Benefit: Slashes forward vehicle collision accidents by 73% in poor visibility rain and fog conditions.

Key Oral Viva Voce Checkpoints

Q1: What is the primary difference between image classification and object detection?

Ans: Classification identifies what object is in the image; detection identifies what objects are present AND where they are located using bounding box coordinates.

Q2: What is Intersection over Union (IoU)?

Ans: The ratio of the area of overlap between the predicted box and ground truth box to the total area of their union.

Q3: What is the typical IoU threshold for counting a detection as a True Positive?

Ans: Typically IoU >= 0.50 (or 0.75 for strict benchmark scoring like COCO mAP@[.5:.95]).

Q4: What is Non-Maximum Suppression (NMS)?

Ans: A post-processing algorithm that eliminates redundant, overlapping bounding boxes by iteratively keeping the highest scoring box and discarding neighboring boxes with IoU above a threshold.

Q5: What is the key architectural difference between Faster R-CNN and YOLO?

Ans: Faster R-CNN is a two-stage detector (Region Proposal Network followed by classifier); YOLO is a single-stage detector that predicts boxes and classes directly from feature grids in one forward pass.

LECTURE #29 · MODULE 5 (Topic 5.2)

Date: 10-Nov-2025
Period: Period 2

Title: Deep Learning Applications: Speech Recognition

CO: CO5 Bloom's: Understand / Apply

Prerequisites: Audio Signal Processing, Recurrent Networks, CTC loss intuition, Day 23 & 26 concepts
Key Terminology: Automatic Speech Recognition (ASR), Mel Frequency Cepstral Coefficients (MFCC), Mel Filterbank, Connectionist Temporal Classification (CTC), Blank Token, Word Error Rate (WER)

Speech Representation

Speech is an analog acoustic pressure wave sampled at high frequencies (16 kHz = 16,000 samples/sec). Acoustic Feature Extraction: 1. Short-Time Fourier Transform (STFT): Apply a Hamming window of 25ms with 10ms hop size to extract local frequency spectrum. 2. Mel Scale Filterbank: Warps linear frequency into the perceptual Mel scale, matching human ear sensitivity (dense filters at low frequencies, wide filters at high frequencies): m = 2595 * log10(1 + f / 700) Yields an 80-channel log-Mel spectrogram tensor (Time x 80).

Connectionist Temporal Classification

Challenge: Audio frames T (e.g. 1,000 frames) far exceed target character length U (e.g. 50 characters), and exact frame-level alignments are unknown. CTC Solution (Graves et al., 2006): 1. Introduces a special 'Blank' token epsilon into the vocabulary: V_prime = V cup {epsilon}. 2. The network outputs a probability distribution over V_prime at every audio frame t. 3. Collapse Mapping B: Collapses consecutive duplicate characters and then removes blank tokens: B('a' 'a' epsilon 'b' 'b') = 'a' 'b' B('c' epsilon 'c') = 'c' 'c' (blank allows generating identical consecutive letters!). 4. Loss Function: Sums conditional probabilities across all valid alignment paths pi that map to target y: P(y | x) = sum_{pi in B^(-1)(y)} product_{t=1}^T P(pi_t | x) L_CTC = - ln P(y | x), optimized via dynamic programming (forward-backward algorithm).

Evaluation Metrics

Word Error Rate (WER): WER = (S + D + I) / N where S is Substitutions, D is Deletions, I is Insertions, and N is total ground truth words.

Exam Important Points

- The role of the Mel scale filterbank in acoustic preprocessing.
- Why CTC loss is necessary for unaligned audio-to-text sequence modeling.
- How the CTC blank token enables spelling repeated letters (e.g. 'hello').
- Formulation and manual calculation of Word Error Rate (WER).

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Word Error Rate (WER) Calculation

Statement: An Automatic Speech Recognition (ASR) system transcribes a recorded meeting sentence. The reference transcript contains N = 10 words: Reference: 'the weather in southern kerala is hot and humid today' Hypothesis: 'the weather in kerala is very hot humid today' Identify the number of Substitutions (S), Deletions (D), and Insertions (I), and calculate the Word Error Rate (WER).

Given: Reference: 10 words. Hypothesis text as given.

Governing Formula: WER = (S + D + I) / N

Step-by-Step Calculation:

Step 1: Align reference and hypothesis words: 'the' -> 'the' (Match) 'weather' -> 'weather' (Match) 'in' -> 'in' (Match) 'southern' -> [DELETED] (Deletion: D = 1) 'kerala' -> 'kerala' (Match) 'is' -> 'is' (Match) [NONE] -> 'very' (Insertion: I = 1) 'hot' -> 'hot' (Match) 'and' -> [DELETED] (Deletion: D = 2) 'humid' -> 'humid' (Match) 'today' -> 'today' (Match).

Step 2: Total error counts: Substitutions (S) = 0. Deletions (D) = 2 ('southern', 'and'). Insertions (I) = 1 ('very'). Total Errors = 0 + 2 + 1 = 3 errors.

Step 3: Calculate WER: WER = (S + D + I) / N = 3 / 10 = 0.3000 (30.0%).

[FINAL EVALUATED ANSWER: S = 0, D = 2, I = 1. Total Errors = 3. Word Error Rate (WER) = 30.0%.]

Pedagogical Note: Two deleted words and one hallucinated insertion yield an overall word error rate of 30% against the 10-word reference.

Problem 2 (Advanced (KTU Exam Level)): Mel Frequency Calculation from Hertz Audio Pitch

Statement: The Mel scale formula relating acoustic physical frequency f in Hertz (Hz) to perceived psychological pitch m in Mels is m = 2595 * log10(1 + f / 700). (a) Calculate the Mel pitch for frequencies f1 = 300 Hz, f2 = 1000 Hz, and f3 = 4000 Hz, (b) Calculate the delta in Hertz between f2 and f1 vs between f3 and f2, and (c) Compare the corresponding deltas on the Mel scale to illustrate human ear non-linearity.

Given: m = 2595 * log10(1 + f / 700). Frequencies: 300 Hz, 1000 Hz, 4000 Hz.

Governing Formula: m = 2595 * log10(1 + f / 700)

Step-by-Step Calculation:

Step 1: For f1 = 300 Hz: m1 = 2595 * log10(1 + 300 / 700) = 2595 * log10(1 + 0.42857) = 2595 * log10(1.42857) = 2595 * 0.15490 = 401.97 Mels.

Step 2: For f2 = 1000 Hz: m2 = 2595 * log10(1 + 1000 / 700) = 2595 * log10(1 + 1.42857) = 2595 * log10(2.42857) = 2595 * 0.38535 = 1000.00 Mels.

Step 3: For f3 = 4000 Hz: m3 = 2595 * log10(1 + 4000 / 700) = 2595 * log10(1 + 5.71429) = 2595 * log10(6.71429) = 2595 * 0.82700 = 2146.06 Mels.

Step 4: Hertz spacing vs Mel spacing: Delta_Hz(f2 - f1) = 1000 - 300 = 700 Hz -> Delta_Mel = 1000 - 401.97 = 598.03 Mels. Delta_Hz(f3 - f2) = 4000 - 1000 = 3000 Hz -> Delta_Mel = 2146.06 - 1000 = 1146.06 Mels. In Hertz, interval 2 is 3000/700 = 4.29x wider, but in Mel space it is only 1146/598 = 1.92x wider!

[FINAL EVALUATED ANSWER: m(300Hz) = 402.0 Mels, m(1000Hz) = 1000.0 Mels, m(4000Hz) = 2146.1 Mels.]

Pedagogical Note: Human hearing has high perceptual resolution at lower frequencies (under 1 kHz) and compressed logarithmic sensitivity at high frequencies, which the Mel filterbank preserves.

Real-World Industry Case (Public Safety & Telecommunications): Real-Time Emergency Dispatch Voice Call Transcription

Pipeline Workflow: Distressed citizen calls 911/112 -> Audio stream contains high acoustic background siren noise -> 80-band Mel filterbank fed into Conformer CTC ASR engine -> Transcribes street address and emergency details in sub-second latency.

Industrial Impact & Benefit: Enables emergency first responders to dispatch ambulances 45 seconds faster, saving critical lives.

Key Oral Viva Voce Checkpoints

Q1: What is the physical sampling rate of standard high-fidelity voice audio?

Ans: 16 kHz (16,000 samples per second) for speech recognition, or 44.1 kHz for studio music.

Q2: Why is the Mel scale preferred over linear frequency in speech recognition?

Ans: The Mel scale matches human auditory perception, which has high resolution for low frequencies and compressed sensitivity for high frequencies.

Q3: What is Connectionist Temporal Classification (CTC)?

Ans: A loss function designed for sequence tasks where the input sequence is longer than the output sequence and exact temporal alignments are unknown.

Q4: What is the function of the blank token epsilon in CTC?

Ans: To indicate silence/transitions and to allow the model to emit repeated characters (e.g. 'b l a n k' vs 'l o o k').

Q5: How does the CTC collapse function decode 'c c _ a a a _ t t'?

Ans: First it collapses consecutive duplicates: 'c _ a _ t'; then it removes blanks: 'cat'.

LECTURE #30 · MODULE 5 (Topic 5.3)

Date: 12-Nov-2025
Period: Period 4

Title: Deep Learning Applications: Natural Language Processing

CO: COS Bloom's: Understand / Apply

Prerequisites: Seq2Seq, Word Embeddings intuition, Cross-Entropy Loss, Day 24 concepts
Key Terminology: Natural Language Processing (NLP), Autoregressive Language Modeling, Perplexity (PPL), Sentiment Classification, Tokenization, Cross-Entropy Loss, Transformer Transition

Nlp Task Taxonomy

Deep learning models natural language across diverse granularities: 1. Document / Sentence-Level Classification: - Sentiment Analysis: Input text -> Output sentiment label (Positive, Neutral, Negative). - Topic Categorization: Input news article -> Output genre (Sports, Tech, Politics). 2. Token-Level Sequence Tagging: - Named Entity Recognition (NER): Tagging Person, Organization, Location per token. - Part-of-Speech (POS) Tagging: Noun, Verb, Adjective. 3. Autoregressive Language Modeling: - Decomposes the joint probability of a sequence of tokens w_1, ..., w_T using the chain rule of probability: P(w_1, w_2, ..., w_T) = product_{t=1}^T P(w_t | w_1, w_2, ..., w_{t-1}) - Foundation for modern generative LLMs (GPT, Llama, Gemini).

Perplexity Metric

Perplexity (PPL) is the standard quantitative metric for evaluating language models. It is defined as the exponentiated average negative log-likelihood per token: PPL = exp(- (1 / T) * sum_{t=1}^T ln P(w_t | w_{<t})) = 2^(Cross_Entropy_Loss_in_bits) Interpretation: Perplexity represents the weighted average branching factor of the language model — the effective number of equally likely words the model is uncertain between at each prediction step. Lower perplexity indicates superior predictive certainty!

Exam Important Points

- Chain rule decomposition of joint sequence probability in language modeling.
- Mathematical formulation and physical interpretation of Perplexity (PPL).
- Deep learning architecture for Sentiment Analysis using Bidirectional LSTMs.
- Limitations of recurrent NLP models (sequential O(T) latency, lack of parallel training).

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Perplexity Calculation from Cross-Entropy Loss

Statement: A neural language model evaluates a test document containing T = 1,000 words. The total negative log-likelihood of the document under natural logarithm (base e) is sum_{t=1}^T - ln P(w_t | w_{<t}) = 3,500 nats. (a) Calculate the average cross-entropy loss per token in nats, (b) Calculate the Perplexity (PPL) of the model, and (c) If a competing model achieves Perplexity PPL = 20, calculate its average loss per token.

Given: T = 1,000 words, Total Loss = 3,500 nats.

Governing Formula: Avg_Loss = Total_Loss / T; PPL = exp(Avg_Loss); Avg_Loss = ln(PPL)

Step-by-Step Calculation:

Step 1: Compute average cross-entropy loss per token: Avg_Loss = 3,500 / 1,000 = 3.50 nats/token.

Step 2: Calculate Perplexity: PPL = e^(3.50) = 33.115.

Step 3: Competing model with PPL = 20: Loss = ln(20) = 2.9957 nats/token.

Step 4: Comparison: The competing model has lower perplexity (20.0 vs 33.1) and lower loss (3.00 vs 3.50), indicating stronger language modeling ability.

[FINAL EVALUATED ANSWER: Average Loss = 3.50 nats, Perplexity PPL = 33.12. Competing Model Loss = 3.00 nats.]

Pedagogical Note: A perplexity of 33.12 means the language model is effectively as uncertain as guessing from a uniformly distributed vocabulary of 33 words at each step.

Problem 2 (Advanced (KTU Exam Level)): Sentiment Classifier Binary Cross-Entropy and Gradient Update

Statement: A sentiment analysis model processes a movie review and outputs probability y_hat = 0.85 for Positive class (ground truth y = 1.0). The output layer computes y_hat = sigma(z) where z = w^T * h + b. Current weight is w_1 = 0.50, hidden activation is h_1 = 0.80, and learning rate is eta = 0.10. Loss is Binary Cross-Entropy L = - [y*ln(y_hat) + (1-y)*ln(1-y_hat)]. (a) Calculate the loss value, (b) Compute the gradient dL/dz and dL/dw_1, and (c) Compute the updated weight w_1_new.

Given: y = 1.0, y_hat = 0.85, w_1 = 0.50, h_1 = 0.80, eta = 0.10.

Governing Formula: L = - ln(y_hat); dL/dz = y_hat - y; dL/dw_1 = (dL/dz) * h_1; w_new = w - eta * (dL/dw)

Step-by-Step Calculation:

Step 1: Compute Binary Cross-Entropy Loss: L = - [1.0 * ln(0.85) + 0] = - (-0.1625) = 0.1625.

Step 2: Compute gradient with respect to logit z: dL/dz = y_hat - y = 0.85 - 1.00 = -0.15.

Step 3: Compute gradient with respect to weight w_1: dL/dw_1 = (dL/dz) * h_1 = (-0.15) * 0.80 = -0.120.

Step 4: Update weight: w_1_new = w_1 - eta * (dL/dw_1) = 0.50 - 0.10 * (-0.120) = 0.50 + 0.012 = 0.5120.

[FINAL EVALUATED ANSWER: Loss L = 0.1625, Gradient dL/dw_1 = -0.120, Updated weight w_1_new = 0.5120]

Pedagogical Note: Because the model underpredicted slightly (0.85 vs 1.0), the negative gradient increased weight w_1 to reinforce positive sentiment confidence in future passes.

Real-World Industry Case (Fintech & Quantitative Regulatory Analysis): Automated Financial SEC 10-K Filing Risk Assessment

Pipeline Workflow: Public corporate annual 10-K financial disclosures (100+ pages) -> Deep bidirectional NLP model scans Risk Factors section -> Categorizes litigation, supply chain, and cyber risks -> Generates portfolio risk scores.

Industrial Impact & Benefit: Enables asset managers to detect emerging corporate insolvency risks weeks before quarterly earnings announcements.

Key Oral Viva Voce Checkpoints

Q1: What is an Autoregressive Language Model?

Ans: A model that predicts the probability of the next word conditioned on all previously generated context words: P(w_t | w_1, ..., w_{t-1}).

Q2: What is Perplexity in language modeling?

Ans: The exponentiated cross-entropy loss, representing the effective branching factor (number of uncertain choices) per token.

Q3: What is the relationship between Perplexity and Cross-Entropy Loss?

Ans: PPL = exp(Loss_nats) or PPL = 2^(Loss_bits). Lower perplexity indicates superior predictive performance.

Q4: What is Named Entity Recognition (NER)?

Ans: A token-level sequence classification task that identifies mentions of predefined real-world entities (persons, locations, organizations) in text.

Q5: How does subword tokenization (Byte-Pair Encoding, BPE) resolve the Out-of-Vocabulary (OOV) problem?

Ans: By decomposing rare and unseen words into frequent subword units (e.g. 'unpredictable' -> 'un' + 'predict' + 'able'), allowing any arbitrary string to be represented.

LECTURE #31 · MODULE 5 (Topic 5.4)

Date: 14-Nov-2025
Period: Period 1

Title: Common Word Embedding: CBOW and Word2Vec

CO: CO5 Bloom's: Understand / Apply

Prerequisites: Vector representations, Softmax, Cross-Entropy, Day 3 concepts
Key Terminology: Word Embeddings, One-Hot Encoding, Word2Vec, Continuous Bag of Words (CBOW), Continuous Skip-Gram, Negative Sampling (SGNS), Vector Arithmetic

One Hot Vs Distributed Embeddings

One-Hot Encoding: - Represents each word as a sparse binary vector of vocabulary size |V| (e.g. 100,000 dimensions). - Fatal Flaw: All word vectors are orthogonal: v_cat^T * v_dog = 0. No notion of semantic distance or similarity! Distributed Word Embeddings: - Maps words to dense, low-dimensional continuous vectors in R^d (d = 100 to 300). - Words appearing in similar contexts acquire similar geometric representations (Distributional Hypothesis: Firth, 1957).

Word2Vec Cbow Vs Skipgram

Tomas Mikolov et al. (Google, 2013) introduced Word2Vec with two complementary architectures: 1. Continuous Bag-of-Words (CBOW): - Objective: Predict a target center word w_t given context words in a window: {w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2}}. - Mechanism: Averages context word vectors: v_context = (1 / 2C) * sum_{c in context} v_c. - Characteristics: Faster training, slightly better on frequent words. 2. Continuous Skip-Gram: - Objective: Given a single center word w_t, predict surrounding context words w_{t+j}. - Characteristics: Slower training, but superior representations for rare words. Skip-Gram with Negative Sampling (SGNS): Evaluating standard Softmax requires summing exp(v^T * v') over the entire vocabulary |V| (costly O(|V|)). Negative Sampling replaces multiclass classification with k binary logistic regressions: L_SGNS = - ln sigma(v'_{w_O}^T * v_{w_I}) - sum_{i=1}^k E_{w_i ~ P_n(w)} [ln sigma(- v'_{w_i}^T * v_{w_I})] where negative words are drawn from unigram distribution raised to the 3/4 power: P_n(w) propto f(w)^(0.75).

Vector Arithmetic

Linear compositionality: Word vectors capture semantic and grammatical analogies via simple vector arithmetic: vec('King') - vec('Man') + vec('Woman') approx vec('Queen') vec('Paris') - vec('France') + vec('Germany') approx vec('Berlin')

Exam Important Points

- Architectural diagrams comparing CBOW (context -> target) vs Skip-Gram (target -> context).
- Why negative sampling is used instead of standard Softmax (computational complexity).
- Unigram distribution exponent 3/4 rationale for negative sampling.
- Semantic vector arithmetic: King - Man + Woman = Queen.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Cosine Similarity Calculation Between Word Vectors

Statement: Two word embedding vectors in 3-dimensional semantic space are given as: v_doctor = [0.8, 0.6, 0.0]^T v_hospital = [0.6, 0.8, 0.0]^T v_banana = [0.0, 0.1, 0.9]^T Calculate: (a) Euclidean norms of all three vectors, (b) Cosine similarity between 'doctor' and 'hospital', and (c) Cosine similarity between 'doctor' and 'banana'.

Given: v_doc = [0.8, 0.6, 0], v_hosp = [0.6, 0.8, 0], v_ban = [0, 0.1, 0.9].

Governing Formula: ||v|| = sqrt(sum v_i^2); cos_sim(u, v) = (u . v) / (||u|| * ||v||)

Step-by-Step Calculation:

Step 1: Compute Euclidean norms: ||v_doc|| = sqrt(0.8^2 + 0.6^2 + 0) = sqrt(0.64 + 0.36) = sqrt(1.0) = 1.00. ||v_hosp|| = sqrt(0.6^2 + 0.8^2 + 0) = sqrt(0.36 + 0.64) = sqrt(1.0) = 1.00. ||v_ban|| = sqrt(0 + 0.1^2 + 0.9^2) = sqrt(0.01 + 0.81) = sqrt(0.82) = 0.9055.

Step 2: Compute dot products: v_doc . v_hosp = (0.8 * 0.6) + (0.6 * 0.8) + (0 * 0) = 0.48 + 0.48 = 0.96. v_doc . v_ban = (0.8 * 0) + (0.6 * 0.1) + (0 * 0.9) = 0.06.

Step 3: Compute Cosine Similarities: cos_sim(doc, hosp) = 0.96 / (1.00 * 1.00) = 0.9600 (96.0% similarity). cos_sim(doc, ban) = 0.06 / (1.00 * 0.9055) = 0.0663 (6.63% similarity).

[FINAL EVALUATED ANSWER: cos(doc, hosp) = 0.9600 (Strongly Related). cos(doc, ban) = 0.0663 (Unrelated).]

Pedagogical Note: Cosine similarity successfully captures high semantic relatedness between clinical concepts while showing near-zero alignment with unrelated entities.

Problem 2 (Advanced (KTU Exam Level)): Skip-Gram Computational Complexity: Full Softmax vs Negative Sampling

Statement: A corpus has vocabulary size |V| = 200,000 words. Word embedding dimension is d = 300. (a) Calculate the multiplication operations required per training step to evaluate full Softmax over the vocabulary, (b) Calculate the operations required under Negative Sampling with k = 5 negative samples, and (c) Determine the exact computational speedup factor achieved by SGNS.

Given: |V| = 200,000, d = 300, k = 5 negative samples.

Governing Formula: Full Softmax Ops = |V| * d (for logits) + |V| (for exp/sum); SGNS Ops = (1 + k) * d

Step-by-Step Calculation:

Step 1: Compute Full Softmax operations: Computing inner products for all vocabulary words: 200,000 * 300 = 60,000,000 multiplications. Exponentiation and normalizer sum: ~200,000 ops. Total Full Softmax ops approx 60,000,000 operations per word update.

Step 2: Compute Negative Sampling (SGNS) operations: SGNS evaluates 1 true positive pair + k negative pairs = 1 + 5 = 6 dot products. Total multiplications = 6 * 300 = 1,800 operations.

Step 3: Calculate computational speedup: Speedup = 60,000,000 / 1,800 = 33,333.33x (~33,333 times faster!).

[FINAL EVALUATED ANSWER: Full Softmax = 60M ops. SGNS = 1,800 ops (Over 33,000x faster!).]

Pedagogical Note: Negative sampling replaces a massive 60-million operation vocabulary sweep with just 1,800 simple multiplications, transforming Word2Vec from an intractable theoretical model into an ultra-fast algorithm trainable on billions of words in minutes.

Real-World Industry Case (Retail E-Commerce & Product Discovery): E-Commerce Semantic Search & Recommendation Engine

Pipeline Workflow: Customer searches 'running shoes' on shopping app -> Catalog items lack literal string match for 'sneakers' -> Word2Vec item embeddings compute high cosine similarity between 'running shoes' and 'sneakers' -> Recommends relevant footwear.

Industrial Impact & Benefit: Increases search conversion rates by 19% by overcoming the vocabulary mismatch problem.

Key Oral Viva Voce Checkpoints

Q1: What is the primary limitation of one-hot word representations?

Ans: High dimensionality, extreme sparsity, and complete lack of semantic distance or similarity between words (all one-hot vectors are orthogonal).

Q2: What is the Distributional Hypothesis in linguistics?

Ans: 'Words that occur in similar contexts tend to have similar meanings' (J.R. Firth, 1957).

Q3: What is the difference between CBOW and Skip-Gram?

Ans: CBOW predicts the center target word from surrounding context words; Skip-Gram predicts the surrounding context words given the center target word.

Q4: Why is standard Softmax computationally prohibitive in Word2Vec?

Ans: Because computing the denominator requires calculating inner products and exponentials over the entire vocabulary $|V|$ (hundreds of thousands of words) for every training step.

Q5: How does Negative Sampling (SGNS) solve the Softmax bottleneck?

Ans: It reformulates the task as binary logistic regression, updating only the positive word pair and k randomly chosen negative words, cutting complexity from $O(|V|)$ to $O(k)$.

LECTURE #32 · MODULE 5 (Topic 5.5)

Date: 17-Nov-2025
Period: Period 2

Title: Common Word Embedding: Global Vectors for Word Representation (GloVe)

CO: CO5 Bloom's: Understand / Apply

Prerequisites: Matrix Factorization, Word2Vec, Log-bilinear models, Day 31 concepts
Key Terminology: Global Vectors (GloVe), Co-occurrence Matrix, Co-occurrence Probability Ratio, Log-Bilinear Model, Weighted Least Squares, Weighting Function f(X_ij)

Glove Motivation

Prior to GloVe (2014), representation learning was divided into two distinct paradigms: 1. Global Matrix Factorization Methods (LSA): Leveraged global corpus statistics, but performed poorly on local word analogy tasks. 2. Local Context Window Methods (Word2Vec): Excellent on analogical reasoning, but failed to directly leverage vast global co-occurrence statistics. GloVe combines the benefits of both by directly fitting a log-bilinear model to the global word-word co-occurrence matrix X.

Mathematical Foundation And Cooccurrence Ratios

Let X_ij be the count of times word j appears in the context of word i in the corpus. Let P_ij = P(j | i) = X_ij / X_i be the probability that word j appears in the context of word i. The core insight of GloVe is that semantic meaning is captured by the RATIO of co-occurrence probabilities: Consider i = 'ice', j = 'steam', and probe words k: - k = 'solid': P(solid | ice) is high, P(solid | steam) is low -> Ratio P(solid|ice) / P(solid|steam) >> 1 - k = 'gas': P(gas | ice) is low, P(gas | steam) is high -> Ratio P(gas|ice) / P(gas|steam) << 1 - k = 'water': P(water | ice) is high, P(water | steam) is high -> Ratio approx 1 - k = 'fashion': P(fashion | ice) is low, P(fashion | steam) is low -> Ratio approx 1 GloVe models this ratio using dot products of word vectors: w_i^T * w_tilde_j + b_i + b_tilde_j = ln(X_ij)

Weighted Least Squares Objective

GloVe minimizes the weighted squared error between the dot product of word vectors and the log co-occurrence counts: J = sum_{i=1}^V sum_{j=1}^V f(X_ij) * (w_i^T * w_tilde_j + b_i + b_tilde_j - ln(X_ij))^2 Weighting Function f(X_ij): f(x) = { (x / x_max)^alpha if x < x_max { 1 if x >= x_max (typically x_max = 100, alpha = 0.75). Properties: 1. f(0) = 0: Ensures ln(0) is never evaluated, avoiding divergence on non-co-occurring pairs. 2. Non-decreasing: Frequent words carry appropriate statistical weight. 3. Saturating at 1: Prevents extremely common stop-words ('the', 'of') from dominating the objective.

Exam Important Points

- The co-occurrence probability ratio table (ice vs steam with solid, gas, water, fashion).
- Derivation of the log-bilinear equation: w_i^T * w_j + b_i + b_j = ln(X_ij).
- Full GloVe weighted least-squares objective function and properties of weighting function f(x).
- Comparison: Word2Vec vs GloVe.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): GloVe Weighting Function f(x) Calculation

Statement: The GloVe weighting function is defined as f(x) = (x / x_max)^alpha if x < x_max else 1.0, with x_max = 100 and alpha = 0.75. Calculate the weight f(x) assigned to: (a) A word pair with zero co-occurrences (x = 0), (b) A moderately frequent pair with x = 16 co-occurrences, (c) A frequent pair with x = 81 co-occurrences, and (d) An extremely frequent stop-word pair with x = 5,000 co-occurrences.

Given: x_max = 100, alpha = 0.75.

Governing Formula: f(x) = (x / 100)^0.75 if x < 100 else 1.0

Step-by-Step Calculation:

Step 1: For x = 0: f(0) = 0.0 (Eliminates log(0) singularity in objective function!).

Step 2: For x = 16: f(16) = (16 / 100)^0.75 = (0.16)^0.75 = (0.16^(3/4)) = (0.16^(1/4))^3 = (0.63245)^3 = 0.25298.

Step 3: For x = 81: f(81) = (81 / 100)^0.75 = (0.81)^0.75 = (0.81^(1/4))^3 = (0.94868)^3 = 0.8538.

Step 4: For x = 5,000: Since x = 5,000 >= x_max = 100, f(5,000) = 1.0 (Caps runaway stop-word weights).

[FINAL EVALUATED ANSWER: f(0) = 0.0, f(16) = 0.2530, f(81) = 0.8538, f(5000) = 1.0000]

Pedagogical Note: The weighting function smooths out frequency disparities, preventing zero-count crashes while preventing stop-word co-occurrences from dominating optimization.

Problem 2 (Advanced (KTU Exam Level)): GloVe Log-Bilinear Prediction and Loss Gradient

Statement: In a GloVe model, word vector w_i = [0.5, 0.4] and context vector w_tilde_j = [0.6, 0.2] with scalar biases b_i = 0.10 and b_tilde_j = 0.05. The observed co-occurrence count is X_ij = 2.0. The weight assigned by f(X_ij) is 0.05. (a) Calculate the dot product plus biases: z_ij = w_i^T * w_tilde_j + b_i + b_tilde_j, (b) Calculate the target log count ln(X_ij), (c) Compute the error e_ij = z_ij - ln(X_ij), and (d) Calculate the gradient of the loss with respect to bias b_i.

Given: w_i = [0.5, 0.4], w_tilde_j = [0.6, 0.2], b_i = 0.10, b_tilde_j = 0.05, X_ij = 2.0, f(X_ij) = 0.05.

Governing Formula: z = w_i^T * w_tilde_j + b_i + b_tilde_j; e = z - ln(X_ij); J_ij = f(X_ij) * e^2; dJ/db_i = 2 * f(X_ij) * e

Step-by-Step Calculation:

Step 1: Compute linear dot product plus biases: dot = (0.5 * 0.6) + (0.4 * 0.2) = 0.30 + 0.08 = 0.38. z_ij = 0.38 + 0.10 + 0.05 = 0.530.

Step 2: Compute target log count: ln(X_ij) = ln(2.0) = 0.69315.

Step 3: Compute prediction error: e_ij = z_ij - ln(X_ij) = 0.530 - 0.69315 = -0.16315.

Step 4: Compute gradient with respect to bias b_i: dJ/db_i = 2 * f(X_ij) * e_ij = 2 * 0.05 * (-0.16315) = 0.10 * (-0.16315) = -0.016315.

[FINAL EVALUATED ANSWER: z_ij = 0.530, Target ln(X_ij) = 0.6931, Error = -0.1631, Gradient dJ/db_i = -0.0163]

Pedagogical Note: Because the predicted log count (0.53) was smaller than the empirical count (0.69), the negative gradient will increase bias b_i to better match corpus statistics.

Real-World Industry Case (Legal Tech & Corporate Compliance): Legal Contract Discovery & Precedent Retrieval

Pipeline Workflow: Corporate legal repository contains 20 years of court filings -> GloVe embeddings trained on corpus capture domain-specific legal terminology associations ('plaintiff', 'tort', 'indemnity') -> Enables semantic case law search.

Industrial Impact & Benefit: Slashes paralegal legal discovery search time from 8 hours to 15 minutes per case.

Key Oral Viva Voce Checkpoints

Q1: What does the acronym GloVe stand for?

Ans: Global Vectors for Word Representation.

Q2: Who invented GloVe and in what year?

Ans: Jeffrey Pennington, Richard Socher, and Christopher D. Manning at Stanford University in 2014.

Q3: How does GloVe combine global matrix factorization and local context windows?

Ans: It constructs a global word-word co-occurrence matrix from local context windows and trains a log-bilinear regression model directly on this global matrix.

Q4: What is the core insight behind GloVe's relationship to word meaning?

Ans: The ratio of co-occurrence probabilities of two words with various probe words encodes semantic differences much more cleanly than raw probabilities.

Q5: What is the GloVe objective function?

*Ans: J = sum_{i,j} f(X_ij) * (w_i^T * w_tilde_j + b_i + b_tilde_j - ln(X_ij))^2.*

LECTURE #33 · MODULE 5 (Topic 5.6)

Date: 19-Nov-2025
Period: Period 4

Title: Current Research Areas: Autoencoders and Representation Learning

CO: CO5 Bloom's: Understand / Apply

Prerequisites: Encoder-Decoder mechanics, Unsupervised learning, Regularization, Day 2 & 14 concepts
Key Terminology: Autoencoder, Undercomplete Autoencoder, Latent Bottleneck, Reconstruction Loss, Sparse Autoencoder, Kullback-Leibler (KL) Divergence, Denoising Autoencoder (DAE), Contractive Autoencoder (CAE)

Autoencoder Architecture

An Autoencoder is a self-supervised neural network trained to reconstruct its own input: 1. Encoder: Maps input x in R^d to a latent code h in R^k: h = f(x) = sigma(W_e * x + b_e) 2. Decoder: Reconstructs input from latent code: r = g(h) = sigma(W_d * h + b_d) 3. Objective: Minimize reconstruction error L(x, g(f(x))) (e.g. MSE for continuous data, BCE for normalized pixels). If an autoencoder succeeds in learning g(f(x)) = x everywhere, it would simply memorize the identity function! To learn meaningful representations, constraints must be imposed.

Autoencoder Variants

1. Undercomplete Autoencoders: - Constrains latent dimension k < d (information bottleneck). - Theorem: If the encoder and decoder are linear and loss is MSE, the undercomplete autoencoder learns the exact same subspace as Principal Component Analysis (PCA)! - With non-linear activations, it learns a non-linear manifold generalization of PCA. 2. Sparse Autoencoders: - Overcomplete latent space (k > d), but penalizes average hidden unit activations rho_hat_j using KL divergence: L_sparse = L_recon + beta * sum_{j=1}^k KL(rho || rho_hat_j) where KL(rho || rho_hat) = rho * ln(rho / rho_hat) + (1 - rho) * ln((1 - rho) / (1 - rho_hat)). 3. Denoising Autoencoders (DAE, Vincent et al., 2008): - Corrupts input x with noise: x_tilde ~ q(x_tilde | x). - Trains the network to reconstruct the CLEAN input x from noisy x_tilde: L = L(x, g(f(x_tilde))). - Forces the model to learn the data manifold's score matching vector field pointing from noise back to the data manifold! 4. Contractive Autoencoders (CAE, Rifai et al., 2011): - Adds an explicit Frobenius norm penalty on the encoder's Jacobian matrix: || J_f(x) ||_F^2 = sum_i sum_j (del h_i / del x_j)^2. - Forces latent representations to be invariant (contractive) to small perturbations in the input space.

Exam Important Points

- KTU Exam Q10: Definition of autoencoder and one application (anomaly detection, dimensionality reduction).
- KTU Exam Q19(b): Merits and demerits of using Autoencoders in Computer Vision.
- Proof that linear autoencoder under MSE loss is equivalent to PCA.
- Formulation of Sparse Autoencoders and KL divergence sparsity penalty.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Kullback-Leibler (KL) Divergence Sparsity Penalty Calculation

Statement: In a sparse autoencoder, the target average activation is set to rho = 0.05. A hidden neuron j has computed average activation rho_hat_j = 0.10 across a training batch. Calculate: (a) The Kullback-Leibler divergence penalty KL(rho || rho_hat_j) for this neuron, and (b) The penalty if the neuron had perfectly satisfied the target (rho_hat = 0.05).

Given: Target rho = 0.05, Actual rho_hat = 0.10.

Governing Formula: KL(rho || rho_hat) = rho * ln(rho / rho_hat) + (1 - rho) * ln((1 - rho) / (1 - rho_hat))

Step-by-Step Calculation:

Step 1: Term 1: rho * ln(rho / rho_hat): rho / rho_hat = 0.05 / 0.10 = 0.50. ln(0.50) = -0.69315. Term 1 = 0.05 * (-0.69315) = -0.03466.

Step 2: Term 2: (1 - rho) * ln((1 - rho) / (1 - rho_hat)): 1 - rho = 1 - 0.05 = 0.95. 1 - rho_hat = 1 - 0.10 = 0.90. (1 - rho) / (1 - rho_hat) = 0.95 / 0.90 = 1.05556. ln(1.05556) = 0.05407. Term 2 = 0.95 * 0.05407 = 0.05137.

Step 3: Combine terms: KL(0.05 || 0.10) = -0.03466 + 0.05137 = 0.01671.

Step 4: If rho_hat = 0.05: KL(0.05 || 0.05) = 0.05 * ln(1) + 0.95 * ln(1) = 0.0000.

[FINAL EVALUATED ANSWER: KL(0.05 || 0.10) = 0.01671. Target penalty = 0.0000.]

Pedagogical Note: The positive KL divergence (0.0167) penalizes the neuron for being overly active (10% firing rate vs 5% target), pushing its bias negative to restore sparsity.

Problem 2 (Advanced (KTU Exam Level)): Reconstruction Error vs Dimensionality Reduction in Linear Autoencoder

Statement: An input vector x = [3.0, 4.0]^T is projected into a 1-dimensional latent subspace using linear encoder weight w = [0.8, 0.6]^T (unit norm ||w|| = 1) with zero bias: h = w^T * x. The decoder reconstructs the input via tied weights: x_hat = w * h. (a) Compute the scalar latent code h, (b) Compute the reconstructed vector x_hat, (c) Calculate the reconstruction error vector e = x - x_hat, and (d) Verify that the reconstruction error vector is orthogonal to the weight vector w (e^T * w = 0).

Given: x = [3.0, 4.0]^T, w = [0.8, 0.6]^T.

Governing Formula: h = w^T * x; x_hat = w * h; e = x - x_hat; Orthogonality check: e^T * w = 0

Step-by-Step Calculation:

Step 1: Compute latent code h: h = (0.8 * 3.0) + (0.6 * 4.0) = 2.4 + 2.4 = 4.80.

Step 2: Compute reconstructed vector x_hat: x_hat = h * w = 4.80 * [0.8, 0.6]^T = [3.84, 2.88]^T.

Step 3: Compute reconstruction error vector e: e = x - x_hat = [3.0 - 3.84, 4.0 - 2.88]^T = [-0.84, 1.12]^T.

Step 4: Compute squared reconstruction loss: Loss = ||e||^2 = (-0.84)^2 + (1.12)^2 = 0.7056 + 1.2544 = 1.960.

Step 5: Verify orthogonality: e^T * w = (-0.84 * 0.8) + (1.12 * 0.6) = -0.672 + 0.672 = 0.0000.

[FINAL EVALUATED ANSWER: h = 4.80, x_hat = [3.84, 2.88]^T, Error e = [-0.84, 1.12]^T (Orthogonal: e^T * w = 0.0000).]

Pedagogical Note: The linear autoencoder performs an exact orthogonal projection onto the 1D principal component subspace, discarding the orthogonal component as reconstruction error.

Real-World Industry Case (Railway Safety & Predictive Infrastructure): Industrial High-Speed Rail Track Flaw Ultrasonic Anomaly Detection

Pipeline Workflow: Ultrasonic inspection cars record rail acoustic waveforms -> Undercomplete Autoencoder trained exclusively on thousands of kilometers of normal, undamaged steel rail -> When a microscopic sub-surface rail crack passes, autoencoder fails to reconstruct it -> High reconstruction error triggers immediate track repair alarm.

Industrial Impact & Benefit: Detects internal rail fatigue fractures before structural derailment failures occur.

Key Oral Viva Voce Checkpoints

Q1: What is an Autoencoder?

Ans: A neural network trained in a self-supervised manner to copy its input to its output through an internal latent representation.

Q2: What is the primary danger of training an unconstrained autoencoder?

Ans: It can learn to perfectly memorize the trivial identity function $g(f(x)) = x$ without extracting useful semantic features.

Q3: What is an Undercomplete Autoencoder?

Ans: An autoencoder whose latent representation dimension k is strictly smaller than the input dimension d.

Q4: What classical statistical technique is equivalent to a linear autoencoder trained with MSE loss?

Ans: Principal Component Analysis (PCA).

Q5: What is a Denoising Autoencoder (DAE)?

Ans: An autoencoder trained to reconstruct the clean original data from an intentionally corrupted or noisy input.

LECTURE #34 · MODULE 5 (Topic 5.7)

Date: 21-Nov-2025
Period: Period 1

Title: Current Research Areas: Representation Learning

CO: CO5 Bloom's: Understand / Analyze

Prerequisites: Autoencoders, Latent variables, Manifolds, Day 33 concepts
Key Terminology: Representation Learning, Manifold Hypothesis, Disentangled Representations, Self-Supervised Learning, Contrastive Learning, InfoNCE Loss, Transfer Learning

Manifold Hypothesis

The Manifold Hypothesis states that real-world high-dimensional data (e.g. 1000x1000 pixel images in R^1,000,000) concentrates its probability mass almost entirely on low-dimensional manifolds embedded within the high-dimensional ambient space. Key Observations: 1. If pixels were uniformly distributed in R^1,000,000, random noise would look like natural scenes. In reality, natural images occupy a tiny fraction of configuration space. 2. Variations in images (lighting, camera pose, object scale, facial expression) correspond to smooth continuous movements along low-dimensional manifold coordinates. 3. Deep networks learn coordinate charts that unfold, unroll, and flatten these complex curved manifolds into Euclidean spaces where linear classifiers succeed.

Disentangled Representations

Bengio, Courville, and Vincent (2013) define an ideal representation as a 'Disentangled Representation'. Properties: - Each coordinate axis of the latent representation h corresponds to an independent real-world explanatory factor of variation (e.g. h_1 = object rotation, h_2 = lighting intensity, h_3 = object identity). - Invariance: A change in lighting modifies only h_2 while leaving identity h_3 completely invariant. - Linear Separability: Complex non-linear decision boundaries on the original data manifold become flat linear hyperplanes in the disentangled representation space.

Self Supervised Learning

Modern Representation Learning utilizes Self-Supervised Learning: 1. Contrastive Learning (SimCLR, Chen et al., 2020): Pulls together representations of augmented views of the same image (positives) while pushing apart representations of different images (negatives) using InfoNCE loss: L_InfoNCE = - ln (exp(sim(z_i, z_j) / tau) / sum_k exp(sim(z_i, z_k) / tau)) 2. Masked Autoencoding (MAE, He et al., 2021): Masks 75% of image patches and trains a Vision Transformer to reconstruct missing pixels, learning rich global representations.

Exam Important Points

- KTU Exam Q20(a): Illustrate the use of representation learning in object classification.
- Definition and physical justification of the Manifold Hypothesis.
- Properties of Disentangled Representations (invariance, linearity, factor independence).
- Formulation of InfoNCE loss in contrastive self-supervised learning.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): InfoNCE Contrastive Loss Calculation

Statement: In a self-supervised contrastive learning step, an anchor representation z_a is compared against 1 positive augmented sample z_p and 2 negative samples {z_n1, z_n2}. Cosine similarities are computed as: sim(z_a, z_p) = 0.80 sim(z_a, z_n1) = 0.20 sim(z_a, z_n2) = -0.10 The temperature parameter is tau = 0.50. Calculate: (a) Exponentiated scaled similarities for all pairs, and (b) The InfoNCE loss value.

Given: sim_pos = 0.80, sim_neg1 = 0.20, sim_neg2 = -0.10, tau = 0.50.

Governing Formula: exp_sim = exp(sim / tau); L = - ln(exp_pos / (exp_pos + sum(exp_neg)))

Step-by-Step Calculation:

Step 1: Compute scaled similarities (sim / tau): pos: 0.80 / 0.50 = 1.60. neg1: 0.20 / 0.50 = 0.40. neg2: -0.10 / 0.50 = -0.20.

Step 2: Compute exponentials: exp_pos = e^(1.60) = 4.9530. exp_neg1 = e^(0.40) = 1.4918. exp_neg2 = e^(-0.20) = 0.8187.

Step 3: Compute denominator sum: Denom = exp_pos + exp_neg1 + exp_neg2 = 4.9530 + 1.4918 + 0.8187 = 7.2635.

Step 4: Compute Softmax Probability: P_pos = 4.9530 / 7.2635 = 0.6819 (68.19%).

Step 5: Compute InfoNCE Loss: L = - ln(0.6819) = 0.3829.

[FINAL EVALUATED ANSWER: P_pos = 0.6819, InfoNCE Loss L = 0.3829]

Pedagogical Note: The model successfully assigned 68.2% of probability mass to the true positive pair, yielding a low contrastive loss of 0.3829.

Problem 2 (Advanced (KTU Exam Level)): Intrinsic Manifold Dimension vs Ambient Pixel Dimension

Statement: An automotive camera produces RGB images of resolution 1024x1024 pixels. The car drives along a single straight highway where camera images vary solely based on 3 physical degrees of freedom: (1) Vehicle forward displacement along road x, (2) Sun elevation angle theta, and (3) Vehicle lane lateral offset y. (a) Calculate the ambient vector space dimension D_ambient, (b) State the intrinsic manifold dimension d_manifold, and (c) Calculate the dimensional compression factor D_ambient / d_manifold.

Given: Image: 1024x1024x3. Physical variations: 3 degrees of freedom.

Governing Formula: D_ambient = H * W * C; d_manifold = Degrees of Freedom; Compression = D_ambient / d_manifold

Step-by-Step Calculation:

Step 1: Compute ambient space dimensionality: D_ambient = 1024 * 1024 * 3 = 1,048,576 * 3 = 3,145,728 dimensions.

Step 2: State intrinsic manifold dimensionality: The underlying physical state space is governed entirely by 3 variables (x, theta, y). Therefore, the intrinsic manifold dimension d_manifold = 3.

Step 3: Compute dimensional ratio: Ratio = 3,145,728 / 3 = 1,048,576x (over 1 million to 1!).

[FINAL EVALUATED ANSWER: D_ambient = 3,145,728. d_manifold = 3. Compression factor = 1,048,576x.]

Pedagogical Note: Despite residing in a 3.1-million-dimensional ambient pixel space, the image data manifold is strictly 3-dimensional. Representation learning isolates these 3 intrinsic axes.

Real-World Industry Case (Digital Pathology & Oncology Diagnostics): Self-Supervised Medical Pathology Slide Pre-Training

Pipeline Workflow: Hospital acquires 1 million unlabeled high-resolution cancer biopsy tissue slides -> Self-Supervised Contrastive Learning (SimCLR/MAE) pre-trains a deep ResNet/ViT backbone -> Fine-tuned on only 50 labeled rare sarcoma slides.

Industrial Impact & Benefit: Achieves 95.2% diagnostic accuracy on ultra-rare cancers where obtaining hundreds of labeled slides is medically impossible.

Key Oral Viva Voce Checkpoints

Q1: What is the Manifold Hypothesis?

Ans: The proposition that high-dimensional real-world data concentrates near low-dimensional continuous sub-manifolds embedded within the high-dimensional ambient space.

Q2: What is a Disentangled Representation?

Ans: A representation where distinct latent coordinates correspond to independent real-world explanatory factors of variation (e.g. azimuth, elevation, shape).

Q3: Why is linear separability a goal of representation learning?

Ans: Because if representation learning maps complex curved manifolds into a space where classes are linearly separable, a simple linear model can achieve optimal classification without overfitting.

Q4: What is Self-Supervised Learning?

Ans: A learning paradigm where supervisory training signals are automatically generated from the data itself (e.g. predicting masked patches, contrastive view invariance) without human labels.

Q5: What is Contrastive Learning?

Ans: A self-supervised approach that pulls representations of positive pairs (augmented views of same sample) together while pushing negative pairs apart.

LECTURE #35 · MODULE 5 (Topic 5.8)

Date: 24-Nov-2025
Period: Period 2

Title: Current Research Areas: Boltzmann Machines and Deep Belief Networks

CO: CO5 Bloom's: Understand / Analyze

Prerequisites: Probability Theory, Energy-based models, Statistical Physics basics, Day 33 concepts
Key Terminology: Energy-Based Model (EBM), Boltzmann Distribution, Partition Function Z, Restricted Boltzmann Machine (RBM), Bipartite Graph, Gibbs Sampling, Conditional Independence

Energy Based Models

An Energy-Based Model associates a scalar energy E(x) with each configuration of variables. High-probability states correspond to low energy levels. The Boltzmann Distribution defines probability via the Gibbs / Boltzmann distribution: P(x) = exp(- E(x)) / Z where Z = sum_x exp(- E(x)) is the normalizing Partition Function. Intractability: Evaluating Z requires summing over 2^D possible binary states, which is computationally intractable for D > 40.

General Boltzmann Machine

A Boltzmann Machine (1985) is an undirected graphical model containing visible units v (observed data) and hidden units h (latent representations): Energy Function: E(v, h) = - v^T * W * h - (1/2) * v^T * L * v - (1/2) * h^T * J * h - b^T * v - c^T * h Intractable Learning: Because units within the same layer are interconnected (matrices L and J), computing P(h|v) requires extensive Markov Chain Monte Carlo (MCMC) sampling, making training extremely slow.

Restricted Boltzmann Machine

Smolensky (1986) introduced the Restricted Boltzmann Machine (RBM), which forbids intra-layer connections (L = 0 and J = 0). Bipartite Graph Structure: Connections exist strictly between visible units v and hidden units h. Energy Function of RBM: E(v, h) = - v^T * W * h - b^T * v - c^T * h = - sum_i sum_j v_i * W_ij * h_j - sum_i b_i * v_i - sum_j c_j * h_j Conditional Independence: Given visible layer v, all hidden units h_j are mutually independent: P(h_j = 1 | v) = sigma(sum_i W_ij * v_i + c_j) Given hidden layer h, all visible units v_i are mutually independent: P(v_i = 1 | h) = sigma(sum_j W_ij * h_j + b_i) This allows parallel, exact block Gibbs sampling in a single step!

Exam Important Points

- KTU Exam Q20(b): Compare Boltzmann Machine with Deep Belief Network.
- Diagram of general Boltzmann machine (with intra-layer connections) vs RBM (bipartite graph).
- Formulation of RBM energy function E(v, h).
- Derivation of conditional probabilities P(h_j=1|v) and P(v_i=1|h) showing conditional independence.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): RBM Energy and Hidden Activation Probability Calculation

Statement: An RBM has 2 visible units and 2 hidden units. Parameters are: Weight matrix W = [[0.8, -0.4], [0.5, 0.6]] Visible bias b = [0.1, -0.2]^T Hidden bias c = [-0.1, 0.3]^T Given visible vector v = [1, 0]^T: (a) Calculate the activation probabilities P(h_1 = 1 | v) and P(h_2 = 1 | v), and (b) If hidden vector is sampled as h = [1, 1]^T, calculate the energy E(v, h).

Given: v = [1, 0]^T, W = [[0.8, -0.4], [0.5, 0.6]], b = [0.1, -0.2]^T, c = [-0.1, 0.3]^T.

Governing Formula: P(h_j = 1 | v) = sigma(sum_i v_i * W_ij + c_j); E(v, h) = - v^T * W * h - b^T * v - c^T * h

Step-by-Step Calculation:

Step 1: Compute input to hidden units: z_h1 = (v_1 * W_11 + v_2 * W_21) + c_1 = (1 * 0.8 + 0 * 0.5) + (-0.1) = 0.8 - 0.1 = 0.70. z_h2 = (v_1 * W_12 + v_2 * W_22) + c_2 = (1 * -0.4 + 0 * 0.6) + 0.3 = -0.4 + 0.3 = -0.10.

Step 2: Compute conditional probabilities: P(h_1 = 1 | v) = sigma(0.70) = 1 / (1 + e^(-0.70)) = 1 / (1 + 0.49659) = 0.6682. P(h_2 = 1 | v) = sigma(-0.10) = 1 / (1 + e^(0.10)) = 1 / (1 + 1.10517) = 0.4750.

Step 3: Compute Energy for v = [1, 0]^T and h = [1, 1]^T: Term 1: v^T * W = [1, 0] * [[0.8, -0.4], [0.5, 0.6]] = [0.8, -0.4]. (v^T * W) * h = [0.8, -0.4] * [1, 1]^T = 0.8 - 0.4 = 0.40. Term 2: b^T * v = 0.1 * 1 + (-0.2) * 0 = 0.10. Term 3: c^T * h = (-0.1 * 1) + (0.3 * 1) = 0.20. Total Energy E(v, h) = - 0.40 - 0.10 - 0.20 = -0.70.

[FINAL EVALUATED ANSWER: P(h_1=1|v) = 0.6682, P(h_2=1|v) = 0.4750, Energy E(v, h) = -0.70]

Pedagogical Note: The negative energy value (-0.70) indicates a favorable, low-energy configuration under the learned Boltzmann distribution.

Problem 2 (Advanced (KTU Exam Level)): Exact Partition Function and Joint Probability in 2x2 Boltzmann Distribution

Statement: A tiny toy RBM has 1 visible unit v in {0, 1} and 1 hidden unit h in {0, 1}. The energy function is E(v, h) = - 2.0 * v * h. (a) Enumerate all 4 possible joint states (v, h) and compute their energies, (b) Calculate the unnormalized Boltzmann weights exp(-E) for all states, (c) Calculate the exact Partition Function Z, and (d) Calculate the exact joint probability distribution P(v, h).

Given: v in {0, 1}, h in {0, 1}, E(v, h) = - 2.0 * v * h.

Governing Formula: E(v, h) = - Z * v * h; Weight = e^(-E); Z = sum exp(-E); P(v, h) = exp(-E) / Z

Step-by-Step Calculation:

Step 1: Enumerate 4 states and energies: State 1: (0, 0) -> E = -2 * 0 * 0 = 0.0 -> exp(-0) = 1.000. State 2: (0, 1) -> E = -2 * 0 * 1 = 0.0 -> exp(-0) = 1.000. State 3: (1, 0) -> E = -2 * 1 * 0 = 0.0 -> exp(-0) = 1.000. State 4: (1, 1) -> E = -2 * 1 * 1 = -2.0 -> exp(-(-2)) = e^(2.0) = 7.3891.

Step 2: Calculate Partition Function Z: Z = 1.000 + 1.000 + 1.000 + 7.3891 = 10.3891.

Step 3: Calculate joint probabilities: P(0, 0) = 1.0 / 10.3891 = 0.0963 (9.63%) P(0, 1) = 1.0 / 10.3891 = 0.0963 (9.63%) P(1, 0) = 1.0 / 10.3891 = 0.0963 (9.63%) P(1, 1) = 7.3891 / 10.3891 = 0.7112 (71.12%).

[FINAL EVALUATED ANSWER: Partition Function Z = 10.3891. P(0,0)=P(0,1)=P(1,0)=0.0963, P(1,1)=0.7112.]

Pedagogical Note: Because state (1, 1) has the lowest energy (-2.0), the Boltzmann distribution concentrates over 71% of the total probability mass onto that single state.

Real-World Industry Case (Recommender Systems & E-Commerce): Collaborative Filtering Netflix Movie Recommendation

Pipeline Workflow: User movie rating history -> Modeled as visible units of a Conditional RBM -> Hidden units extract latent genre preferences (sci-fi, romance) -> Gibbs sampling reconstructs missing movie ratings.

Industrial Impact & Benefit: Was a core winning algorithmic component of the famous million-dollar Netflix Prize competition (Salakhutdinov et al., 2007).

Key Oral Viva Voce Checkpoints

Q1: What is an Energy-Based Model (EBM)?

Ans: A probabilistic model that defines probability distributions by assigning a scalar energy to configurations, where low energy corresponds to high probability.

Q2: What is the Partition Function Z in a Boltzmann Machine?

Ans: The normalization denominator Z = sum exp(-E(x)) that ensures probabilities sum to 1.0 across all possible configurations.

Q3: Why is training a general Boltzmann Machine computationally intractable?

Ans: Because intra-layer connections prevent conditional independence, making the exact partition function and posterior inference require prohibitive exponential sums.

Q4: What structural restriction defines a Restricted Boltzmann Machine (RBM)?

Ans: No connections are permitted between units within the same layer (no visible-to-visible or hidden-to-hidden connections), forming a bipartite graph.

Q5: What major mathematical benefit arises from the RBM bipartite restriction?

Ans: Conditional independence: all hidden units are conditionally independent given the visible units, and all visible units are conditionally independent given the hidden units.

LECTURE #36 · MODULE 5 (Topic 5.9)

Date: 26-Nov-2025
Period: Period 4

Title: Current Research Areas: Deep Belief Networks (DBNs) and Course Synthesis

CO: CO5 Bloom's: Understand / Evaluate

Prerequisites: Restricted Boltzmann Machines, Energy-Based Models, Day 35 concepts
Key Terminology: Deep Belief Network (DBN), Hybrid Graphical Model, Contrastive Divergence (CD-k), Greedy Layer-Wise Pre-training, Positive Phase, Negative Phase, Fine-Tuning, Course Synthesis

Dbn Architecture

A Deep Belief Network (Hinton et al., 2006) is a deep generative hybrid graphical model formed by stacking multiple Restricted Boltzmann Machines (RBMs). Hybrid Graphical Structure: - Top two layers form an UNDIRECTED bipartite graph (an RBM) that acts as an associative memory. - All lower layers form DIRECTED acyclic generative connections pointing downward from hidden layers to visible layers: P(x, h^1, h^2, ..., h^L) = P(h^{L-1}, h^L) * product_{l=1}^{L-2} P(h^l | h^{l+1}) * P(x | h^1)

Contrastive Divergence

The log-likelihood gradient for an RBM is: del ln P(v) / del W = E_data [v * h^T] - E_model [v * h^T] - Positive Phase (Data-dependent expectation): Easily evaluated using training sample v and P(h|v). - Negative Phase (Model distribution expectation): Requires running an MCMC chain to thermal equilibrium (infinity steps). Contrastive Divergence (CD-k, Hinton 2002): Replaces infinite MCMC sampling by running only k steps of Gibbs sampling (typically k = 1) starting from the data vector v_0: 1. Sample h_0 ~ P(h | v_0) 2. Sample v_1 ~ P(v | h_0) (Reconstruction) 3. Sample h_1 ~ P(h | v_1) Weight update: delta W = eta * (v_0 * h_0^T - v_1 * h_1^T) Astonishingly fast, robust, and revolutionized deep unsupervised training!

Greedy Layerwise Pretraining Protocol

The 2006 breakthrough protocol that launched the deep learning era: 1. Step 1: Train RBM 1 on raw data v to learn feature representation h^1 using CD-1. 2. Step 2: Freeze RBM 1. Use its hidden states h^1 as input data to train RBM 2. 3. Step 3: Repeat greedily layer by layer for L layers. 4. Step 4: Unroll into a feedforward deep network, attach a supervised Softmax classification layer, and perform end-to-end backpropagation Fine-Tuning! Why it worked: Greedy pre-training initialized deep weights into a basin of attraction close to a good local minimum, completely bypassing the vanishing gradient barrier of 2006.

Course Synthesis

Complete KTU CST414 Course Synthesis: - Module 1: Foundations (Perceptrons, MLPs, Backpropagation, Optimization pathologies). - Module 2: Deep Learning Engine (Advanced optimizers: Momentum, NAG, Adam; Regularization: L1/L2, Dropout, Initialization). - Module 3: Spatial Intelligence (CNNs, Pooling, Equivariance, Bottlenecks, Winograd, MobileNet). - Module 4: Temporal & Sequence Intelligence (RNNs, Seq2Seq, BPTT, LSTMs, GRUs). - Module 5: Frontiers & Applications (Vision, Speech, NLP, Word2Vec, GloVe, Autoencoders, DBNs).

Exam Important Points

- Hybrid directed/undirected structure of Deep Belief Networks.
- Contrastive Divergence CD-1 algorithm steps: positive phase vs negative phase.
- Greedy layer-wise unsupervised pre-training and supervised fine-tuning procedure.
- Complete comparison between Boltzmann Machine, RBM, and DBN.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Single Step Contrastive Divergence (CD-1) Weight Update Calculation

Statement: An RBM has 2 visible units and 1 hidden unit with scalar weights W = [0.2, -0.1]^T (biases are zero). A training sample is v_0 = [1.0, 1.0]. Learning rate is eta = 0.5. (a) In the positive phase, compute hidden activation probability p_h0, (b) In the negative phase, reconstruct visible vector probabilities p_v1 using hidden activation h_0 = p_h0, and (c) Compute the negative hidden probability p_h1 and the resulting CD-1 weight gradient update delta_W.

Given: v_0 = [1, 1], W = [0.2, -0.1]^T, eta = 0.5.

Governing Formula: p_h0 = sigma(v_0 . W); p_v1 = sigma(h_0 * W^T); p_h1 = sigma(p_v1 . W); delta_W = eta * (v_0^T * p_h0 - p_v1^T * p_h1)

Step-by-Step Calculation:

Step 1: Positive Phase: z_h0 = (1.0 * 0.2) + (1.0 * -0.1) = 0.10. p_h0 = 1 / (1 + e^(-0.10)) = 1 / (1 + 0.90484) = 0.5250. Positive correlation: v_0^T * p_h0 = [1, 1]^T * 0.5250 = [0.5250, 0.5250]^T.

Step 2: Negative Phase (Reconstruction): z_v1_1 = 0.5250 * 0.2 = 0.1050 -> p_v1_1 = sigma(0.1050) = 0.5262. z_v1_2 = 0.5250 * (-0.1) = -0.0525 -> p_v1_2 = sigma(-0.0525) = 0.4869. p_v1 = [0.5262, 0.4869].

Step 3: Negative Hidden Activation: z_h1 = (0.5262 * 0.2) + (0.4869 * -0.1) = 0.1052 - 0.0487 = 0.0565. p_h1 = sigma(0.0565) = 0.5141. Negative correlation: p_v1^T * p_h1 = [0.5262 * 0.5141, 0.4869 * 0.5141]^T = [0.2705, 0.2503]^T.

Step 4: Compute CD-1 Weight Update: delta_W = 0.5 * ([0.5250, 0.5250]^T - [0.2705, 0.2503]^T) = 0.5 * [0.2545, 0.2747]^T = [0.1272, 0.1374]^T.

Step 5: Compute updated weights: W_new = [0.2 + 0.1272, -0.1 + 0.1374]^T = [0.3272, 0.0374]^T.

[FINAL EVALUATED ANSWER: Positive stats = [0.525, 0.525]^T, Negative stats = [0.271, 0.250]^T, Updated W = [0.3272, 0.0374]^T]

Pedagogical Note: Contrastive Divergence pushed weights in the direction of the empirical data distribution and away from the 1-step model hallucination.

Problem 2 (Advanced (KTU Exam Level)): KTU Model Paper Q20(b): Comparative Architectural Analysis of BM vs DBN

Statement: Compare Boltzmann Machines (BM), Restricted Boltzmann Machines (RBM), and Deep Belief Networks (DBN) across: (1) Graph topology and directionality, (2) Parameter count scaling, (3) Training complexity and algorithm, and (4) Generative capability.

Given: Architectures: Boltzmann Machine, RBM, DBN.

Governing Formula: Tabular synthesis and structural contrast.

Step-by-Step Calculation:

Step 1: Graph Topology and Directionality: - Boltzmann Machine: Undirected graph with intra-layer connections within visible and hidden units. - RBM: Undirected bipartite graph with zero intra-layer connections. - DBN: Hybrid model; top two layers are an undirected RBM; all lower layers are directed top-down generative connections.

Step 2: Training Algorithm and Complexity: - Boltzmann Machine: Intractable MCMC requiring infinite Gibbs sampling; exponential complexity O(2^D). - RBM: Contrastive Divergence (CD-k), linear complexity O(k * D_v * D_h). - DBN: Greedy layer-wise unsupervised pre-training using CD-k on stacked RBMs, followed by supervised backpropagation fine-tuning.

Step 3: Inference and Feature Extraction: - Boltzmann Machine: Slow stochastic inference. - RBM: Exact, one-step parallel block Gibbs sampling: P(h|v) in O(1) MCMC steps. - DBN: Feedforward bottom-up inference pass for feature extraction; top-down directed ancestral sampling for generation.

Step 4: Historical Significance: - DBN demonstrated in 2006 that deep networks with many layers could be trained effectively, breaking the 15-year neural network winter.

[FINAL EVALUATED ANSWER: BM: Undirected with intra-layer loops (intractable). RBM: Undirected bipartite (CD-k). DBN: Hybrid directed/undirected (Greedy pre-training + fine-tuning).]

Pedagogical Note: DBNs revolutionized AI by introducing layer-wise pre-training on stacked RBMs, bridging statistical physics with modern deep learning.

Real-World Industry Case (Heavy Industry & Vibration Telemetry Analysis): Industrial Turbine Fault Diagnosis using Deep Belief Networks

Pipeline Workflow: High-dimensional multi-sensor vibration readings from power plant turbines -> 3-layer DBN pre-trained on unlabelled baseline vibrations via CD-1 -> Top layer fine-tuned with 50 labeled bearing failure events -> Detects sub-surface rotor imbalances.

Industrial Impact & Benefit: Enables predictive maintenance that avoids multi-million-dollar turbine rotor catastrophic blade fracture.

Key Oral Viva Voce Checkpoints

Q1: What is a Deep Belief Network (DBN)?

Ans: A deep generative hybrid graphical model consisting of stacked RBMs, where the top two layers form an undirected associative memory and lower layers have directed top-down generative connections.

Q2: Who invented Deep Belief Networks and in what year?

Ans: Geoffrey Hinton, Simon Osindero, and Yee-Whye Teh in 2006.

Q3: What is Contrastive Divergence (CD-k)?

Ans: An approximation algorithm for training RBMs that replaces full MCMC sampling by running only k steps of Gibbs sampling initialized at the data vector.

Q4: What are the two phases of Contrastive Divergence?

Ans: The Positive Phase (measures correlations on true data v0*h0^T) and the Negative Phase (measures correlations on reconstructed data v_k*h_k^T).

Q5: What is Greedy Layer-Wise Unsupervised Pre-training?

Ans: Training each layer of a deep network one by one from bottom to top as an independent RBM on the representations of the previous layer, before unrolling into a supervised network.